GwenLaya v4: a pre-registered study of answering only when a check can back the answer

Interim report: registered design, deviations, a claim audit, exploratory full-E baselines on TPU, a math gate, a 9B tier, and a trained Laya tested on a fresh math and Python set with three deviations and a failed control, then on a fresh Rust set where its advantage did not appear, and then once more, unchanged, on fresh Python, math and Lean 4 sets

Xavier Callens
AutoevolveAI / ANSE open-source project

11 October 2026 (revision 3.10.0, branch `release/3.10.0`; supersedes 3.9.0, 3.8.0, 3.7.1, 3.7.0 and 3.6.0)

Abstract

GwenLaya is a system design in which a small open model (Qwen3.5 family) answers a programming or mathematics question only when an executed check or a calibrated confidence score supports the answer, and otherwise abstains or escalates to a larger tier. We pre-registered fifteen hypotheses (two primary: fewer confident-wrong answers than log-probability selective prediction at matched coverage, H1; lower cost per correct answered item than the largest tier with the gate, H3; **as tested on E′, H1 is the difference in area under the risk-coverage curve against raw log-prob and H3 is the difference in selective risk against the gate alone at the gate's coverage, not the registered cost-per-correct metric**) before any GwenLaya evaluation data were generated. **Revision 3.6.0 added a deviated test of the two primaries on a fresh math and Python evaluation set E′ (Section 7): on that set H1 is supported and H3 is not. Revision 3.7.0 ran the same frozen Laya on a fresh Rust set R′ (Section 7.7): neither primary is supported there, so the E′ result is a math and Python result and has not been shown to carry over to Rust.** It carries deviations D48 (split), D49 (Laya is trained on E) and D50 (the registered shuffled-label control failed and was withdrawn; applied as written, E′ would not have been generated). No other registered hypothesis has been tested, and no test ran in the registered form.

Revision 3.10.0. The Laya, its isotonic map and τ were not changed or re-chosen. Six additions; the Laya result is a replication of an A13b estimand (a deviated test, as in 3.6.0 and 3.7.0), the rest is descriptive. (1) **Fresh sets F** (addendum A21; Section 7.9), committed after the task files were built and before any model generated for F: 407 Python tasks (LeetCodeDataset 205, FullStackBench 112, BigCodeBench 90), 501 math tasks (MATH test 200, GSM8K 130, 171 competition problems) and 229 Lean 4 statements (miniF2F validation), screened against everything the Laya was trained, calibrated or evaluated on (E, pool P, R′ and the A18 test split; 202 MATH-test, 12 competition and 34 FullStackBench candidates were removed). Pooled Python and math accuracy (908 tasks, greedy, 1,024 new tokens) is 0.313, 0.427 and 0.472 for the 2B, 4B and 9B; the competition stratum is a floor (0.012 to 0.029, 96.5 to 98.2% of generations cut at the limit). The executed gate has a lower confident-wrong rate than answering the same number of most-confident tasks by log-prob on every tier (9B: Python 0.147 against 0.224; math 0.004 against 0.078; marginal intervals), and the 2B→4B→9B cascade again does not pay off against always-9B (pooled cost ratio 1.737, CI 1.649 to 1.832, accuracy difference −0.007, CI −0.026 to +0.012). (2) **The frozen Laya on F** (Section 7.10; 9B, Python and math pooled, 908 tasks): **H1 is supported** (AURC difference against the raw mean log-prob −0.104, 95% CI −0.132 to −0.078, Holm $p = 0.0002$; 3.6.0

on E′: −0.078; within Python alone the 9B interval includes zero, −0.036 [−0.092, +0.019], and the pooled result is carried by math), **H3 is not** (selective risk against the gate alone at the gate's coverage −0.002, CI −0.022 to +0.016; Holm $p = 0.43$). Exploratory: Laya is not distinguishable from the hand-feature logistic regression on AUROC (−0.004 [−0.015, +0.006]) and its AURC is slightly worse (+0.010 [+0.003, +0.019]); on Python alone it is below the regression on every tier (4B and 9B intervals exclude zero; 2B, 4B, 9B: −0.018, −0.040, −0.033); the router at $\tau = 0.5$ has the accuracy of always-9B (0.476 against 0.472) at 0.889 (CI 0.864 to 0.915) of its cost, against 0.79 on E′ (the first-chunk-included reading in both). These are F-wide statements; they do not extend to Rust (R′ stands), to Lean (no Laya score on Lean was computed) or to the generators' pre-training exposure. (3) **First Lean 4 results** (Section 7.11; greedy, one proof per statement, hardened gate): on the 223 miniF2F test statements the 0.8B, 2B, 4B and 9B prove 22, 41, 59 and 69 (0.099, 0.184, 0.265, 0.309); on the 229 validation statements of F the 2B, 4B and 9B prove 33, 65 and 76 (0.144, 0.284, 0.332). Every VERIFIED proof matched the reference statement exactly (0 definitional-only matches). The gate's known limits (binder annotations not compared, definitional matches accepted, external red-team review not completed) still apply, and 4 to 24 checks per tier and set (timeouts at 300 s per Lean stage) are counted as not proved. (4) **A15.3** (Section 7.12): of four rules for choosing which visible Rust tests the gate sees, none lowers the confident-wrong rate relative to the first test at $k = 2$ or 3 with an interval excluding zero on E; on R′ (68 tasks) several rules do on the 2B and 9B, a comparison whose first-versus-first-and-last count was known before the plan. (5) **A22** (Section 7.14): the candidate-first layout is *not* better than the frozen layout under the registered rule (E′ AUROC difference −0.010 [−0.016, −0.004]; R′ +0.004 [−0.027, +0.034]); one seed, no shuffled-label control, and the planned token-window fractions were not computed (no tokenizer in the analysis environment, D74). (6) **System integration, results only for A19 and A20** (Section 7.13): the frozen Laya, the registered program-of-thought math gate (without a gold answer) and a llama.cpp tier with log-probs are wired into `GwenLaya`; a replay of the registered E′ policy through that loop agrees on all 828 tasks (A20); on 28 Python pairs the Laya's calibrated p moves by 0.162 on average (CI 0.054 to 0.287) between bf16 and Q4_K_M answers of the 2B (A19). **Not validated:** accuracy, calibration, thresholds and cost of the integrated system on Q4_K_M tiers; no end-to-end evaluation exists.

Revision 3.6.0. Laya here is Answer.AI's `ModernBERT-large` with a rank-16 LoRA adapter and a router and a calibrator head, trained on a TPU v5e (torch_xla) on 3,081 tasks (E, 1,536, plus 1,545 from a fresh pool P) with labels from Qwen3.5-2B, 4B and 9B. The data-lake "laya" weights were not used. A calibration split C (283 tasks) fixes an isotonic map; the artifacts were frozen in commit `d217f1e` before E′ (828 tasks: 659 math, 169 Python) was generated. On E′, with the 9B as primary tier, the calibrated Laya P(correct) has a lower area under the risk-coverage curve than the raw mean log-prob (difference −0.078, 95% CI −0.106 to −0.053; Holm $p = 0.0002$ over H1 and H3), and the same holds for the 2B and 4B (H1). Against the executed gate alone, at the gate's coverage, the difference in selective risk is −0.012 (CI −0.023 to +0.011), and H3 is not supported on any tier. Exploratory: a logistic regression on hand-built features (gate, mean and min log-prob, domain, tier) cannot be told apart from Laya on AURC (the LR's point estimate is slightly better), and Laya's AUROC advantage over it is 0.013 (CI 0.001 to 0.026) on the 9B; the gain over the raw log-prob and the gate alone may come largely from combining them (a reading; the text-only ablation of 3.8.0, Section 7.18, removes the numeric inputs, not the text, so it does not settle this). **The shuffled-label controls failed the registered 0.4 to 0.6 criterion on C** (calibrator 0.693; router 0.572 / 0.637 / 0.389), the untrained twin scored 0.480 pooled and 0.239 to 0.514 within strata, and the criterion was withdrawn (D50) and replaced by a leakage check (E′: 0 overlap by cluster, task id and prompt hash); our reading of the failure is not proven. An exploratory router (threshold chosen on C) reaches accuracy 0.705 against 0.696 for always-9B, with a confident-wrong rate of 4.0% against 30.4% (the policies answer different sets of tasks) and a cost of 0.79 of always-9B's (0.89 with each run's first compile chunk zeroed). Qwen3.5-27B did not load in 33 minutes on a four-chip slice, a generator LoRA probe finished no step in 25 minutes, and a first Laya run was lost (D52). E′ has 169 Python tasks and mostly easy math, and no Rust.

Revision 3.8.0. One exploratory ablation (A16, one seed, descriptive, no hypothesis tested): the same Laya calibrator retrained with only the domain, tier, prompt and candidate text (no gate or log-prob input). On E′ it reaches AUROC 0.839, 0.869 and 0.870 (2B, 4B,

9B; CIs above 0.5), above the raw log-prob and below the hand-feature regression, with the full Laya above it by about 0.08; on the fresh Rust set R′ it is at chance (AUROC 0.405, 0.555, 0.588; CIs include 0.5). Section 7.18 gives the numbers and the hypotheses we did not test.

Revision 3.9.0. Four additions; none is a hypothesis test, and none required new generations or GCP spend. (1) A17, the Python visible-test depth dial on the stored E answers (524 tasks; Section 7.15): giving the gate the first $k = 1, 2, 4, 8$ hidden test cases instead of one lowers the confident-wrong rate from 0.139 [0.111, 0.170] to 0.036 [0.021, 0.053] (4B) and from 0.147 [0.116, 0.177] to 0.046 [0.029, 0.065] (9B), while gate coverage falls from 0.815 to 0.712 and from 0.872 to 0.771; the pattern is that of the Rust dial of 3.7.0. The A17 plan said these answers came from prompts that showed no tests. **That is false**: every MBPP prompt (373 of the 529 Python tasks) prints the gate's assert verbatim, and 71 of the 156 HumanEval prompts carry doctest examples (D63). (2) **A corrected headline.** The 4B math gate's confident-wrong rate of 0.026 was set against 0.388 "when everything is answered"; 0.388 counts the 160 math tasks with no boxed answer (mostly generations cut at 1,024 tokens) as confident-wrong. Against answering whenever a boxed answer exists, the comparison is 0.026 against 0.068 (registered scorer). (3) The registered math scorer missed notational equivalences (D62). An exploratory, post-hoc re-score of the stored answers raises math accuracy from 0.514, 0.612 and 0.642 to 0.558, 0.664 and 0.692 (2B, 4B, 9B) and lowers the math gate's VERIFIED-but-wrong count from 7, 13 and 10 to 2, 0 and 0 of 500; under it the 4B and 9B math gates have no confident-wrong answer on E (rule-of-three 95% upper bound 0.006), against 0.016 and 0.002 for answer-if-boxed. The registered numbers remain the primary record; the re-score was not propagated into A4, A5, A11, A12 or the Laya labels. (4) System changes, with no results: an opt-in weak-gate verdict policy (off by default; every VERIFIED of the stored E gate rests on one visible test), two Lean verdict rules, and a hardened Lean 4 gate (a trusted second Lean process compares the elaborated theorem type with the reference statement and audits axioms with `collectAxioms`) written after an internal audit found soundness holes in the earlier gate (Sections 7.16, 7.17). A Lean miniF2F run (A18) is planned and pre-registered, with 223 of 244 statements passing the sanity check. Its generation started after the A18 amendment and was in progress; **no Lean result was reported in 3.9.0** (the results are in 3.10.0, Section 7.11; the $k = 8$ sampling never produced rows).

Revision 3.7.0. R′ is built from Exercism's Rust track (MIT): of 106 practice exercises, 69 are kept because their reference solution is verified by the real checker and the unmodified starter stub is not; R′ is used for evaluation only (addendum A14, committed before any model generated for it). The frozen Laya, isotonic map included, was run on the answers of Qwen3.5-2B, 4B and 9B. Accuracy is 6/69, 28/69 and 37/69. The executed gate (first test only) verifies 21, 40 and 45 answers with precision 0.286, 0.700 and 0.822, and its confident-wrong rate is 0.217, 0.174 and 0.116 against 0.913, 0.594 and 0.464 when everything is answered. On R′ the 9B H1 difference (Laya minus raw log-prob, AURC) is −0.070 (95% CI −0.180 to +0.041; Holm $p = 0.20$), not supported on any tier; with 69 tasks this is underpowered and is not evidence of no effect. The H3 difference is exactly 0 on every tier, because Laya's top-k tasks at the gate's coverage are the gate-verified tasks themselves. Exploratory: Laya is below the hand-feature logistic regression (AUROC difference −0.039 [−0.117, +0.023] on the 9B, −0.067 [−0.144, −0.001] on the 4B, −0.153 [−0.258, −0.070] on the 2B). The router with the old $\tau = 0.5$ has the accuracy of always-9B (0.536) and a confident-wrong rate of 0.116 against 0.464, but a cost ratio of 0.987 (CI 0.946 to 1.024), i.e. no saving, against 0.79 on E′. We also report a harness property found while building R′: the Rust oracle compiled without `-edition` (edition 2015). Re-checking every stored Rust answer under edition 2021 (A15.1) changed at most one hidden-test verdict per tier, always fail to pass (E 4B 1/507, E 9B 1/507, R′ 4B 1/69); published numbers stay labelled edition 2015 and the oracle default is now 2021 (D58). Reasons for the non-transfer are hypotheses only. **Improving the gate (Section 7.8).** Descriptively, on the stored Rust answers of E (505 tasks), showing the gate $k = 1, 2, 3$ visible assertions instead of one lowers the confident-wrong rate from 0.067 to 0.024 (4B) and from 0.061 to 0.014 (9B) while gate coverage falls from 0.594 to 0.550 and from 0.648 to 0.600. This is a property of the stored answers, not a hypothesis test; it was not run on Python or on R′, and the frozen Laya was not changed.

Revision 3.4.0 added *exploratory, unregistered* baselines on the full E set (1,536 tasks: Python 529, Rust 507, math 500). Revision 3.5.0 adds three further exploratory analyses

on the same E set, all post hoc: (A10) the registered program-of-thought math gate, which 3.4.0 reported as not implemented; (A11) cross-fitted calibrated abstention; (A12) a Qwen3.5-9B tier on a four-chip TPU slice with gate cascades. Models: Qwen2.5-Coder-1.5B-Instruct, Qwen3.5-2B, 4B and 9B. All used greedy decoding with one sample, in bf16 under vLLM on TPU v5e, and all outputs were scored locally in a sandbox. E is burned for the registered H1/H3 (Section 8); 3.6.0 therefore uses a fresh E′. The 3.4.0 analysis plan was committed after generation and as the 2B/4B scoring started, before any scored outcome was inspected but after smaller descriptive runs on 80 tasks had shown approximate pass rates. The A10, A11 and A12 addenda were each written before their results were computed, but after the E baselines had been seen. **This revision supersedes the 3.4.0 statement that the math gate has coverage 0**: that was an implementation gap in our checker, now closed (Section 6.6).

Pooled pass rates on the hidden checks were 0.423 (2B), 0.606 (4B), 0.653 (9B) and 0.482 (Coder). Most of the findings are negative or null: (i) Raw exp(mean log-prob) confidence is badly calibrated. On the 4B its mean is 0.927–0.938 per domain, while only 0.529–0.675 of answers pass. (ii) A gate cascade (2B, then an executed gate, then the 4B if the gate does not verify) is "GwenLaya without Laya" and is not a registered arm. With the program-of-thought math gate and its generation cost charged, it gains no accuracy over always using the 4B: the pooled difference is −0.001 (CI −0.013 to 0.012). Pooled, it costs about twice the accelerator time (cost ratio 2.021, CI 1.933–2.111, with the first compilation chunk; 2.023, CI 1.912–2.134, without it), because on math it pays for programs and usually for both tiers. On Python alone it breaks even with warm-up (0.997) and is about 10% cheaper without it (0.900), at a 2.6-point accuracy cost. Cost is configuration dependent and these ratios are not robust. (iii) The program-of-thought math gate (A10) verifies 0.454 of the 4B's math answers (CI 0.410–0.498) with precision 0.943 (CI 0.911–0.971), just below the 0.95 flag point of the registration, and the CI includes it. Its confident-wrong rate over all math tasks is 0.026. **Corrected in 3.9.0:** revisions 3.5.0 to 3.8.0 set this against 0.388 "when everything is answered", but 0.388 counts the 160 tasks without a boxed answer (160 of the 4B's 194 wrong rows; mostly truncated) as confident-wrong. Answering whenever a boxed answer exists gives 0.068 at coverage 0.680 with the registered scorer, and 0.016 under the exploratory re-score of D62, where the gate's own rate is 0 of 500 (Section 6.6). (iv) A tier gap does change the cascade result (A12). Using a fixed reading rule written before the 9B results (a cascade pays off only if it is cheaper than always-9B in both the with- and without-warm-up readings and its accuracy CI does not exclude zero on the harmful side), 4B→9B pays off for Python and Rust but not for math (programs are charged for each tier consulted) and not pooled (cost ratio 1.381 with warm-up and 1.724 without, although pooled accuracy is higher by 0.037). 2B→9B on Python fails on accuracy (−0.042, CI −0.068 to −0.015). The 9B's effective engine concurrency was not measured, so its cost is less certain. (v) A cross-fitted logistic score (A11) that includes the gate verdict ranks and calibrates better than log-prob alone (4B pooled AUROC 0.931 against 0.708). Because it uses the gate as a feature, that is a calibration gain, not independent evidence. Its ECE bootstrap intervals sometimes exclude their own point estimates, a known bias of percentile bootstrap for ECE.

The most favourable observation remains the 4B gate. Answering only when the executed gate verifies gave fewer confident-wrong answers than a log-prob threshold at the same coverage: −0.076 for Python, −0.112 for Rust and, with the new math gate, −0.084 for math. The baseline answers the k most confident tasks, with k equal to the gate's answered count in the same domain. Several caveats apply. The gate runs tests or programs that the log-prob does not see. k is matched on E, not taken from a threshold frozen on a separate split. The benchmarks are public and were probably seen in pretraining. There is a single greedy sample. This comparison is not H1 (3.5.0 text: "nothing here supports H1 or H3 or says anything about Laya, no Laya head was trained"; superseded for H1 and H3 by Section 7, where the head exists).

A serial re-check of all 1,536 non-passing code rows gave 3 flips. Two of them (one task, for the 2B and the 4B) changed the headline. The third is a nondeterministic candidate and stays FAILED. A re-run of all 1,572 passing rows changed none.

This revision also keeps the corrections of 3.4.0 (Section 2) and 3.5.0 (Section 2.7). The recorded GCP spend is $22.0055, a list-price estimate from 31 ledger lines as read when this revision was built (2026-10-11 15:54:04). The 4 lines after the 27th ($2.7254) are 3.10.0's: the F generations, the frozen Laya's prediction on F and the A22 training run (Table 44); the two

A18 Lean generation lines ($1.5220) were counted in 3.9.0. Revision 3.9.0 had 27 lines and $19.2801; 25 lines and $17.7581 up to 3.8.0 (corrected in 3.9.0: the 3.8.0 abstract still said $17.24 and 24 lines); 3 lines and $1.28 were added by revision 3.7.0 and 10 lines and $11.05 by 3.6.0 ($4.91 in 11 lines up to 3.5.0, of which $2.05 was the 9B smoke test and full run). It says nothing about the system.

1 Retraction and the v3 gate failure, stated first

Retraction. Revision 3.1.0 of the predecessor report [2] claimed that the low-tier optimizer improved pass rates by at least +8.0 percentage points. That claim rested on a hand-written results file with invented numbers (A0 33 %, A3 100 %). The file was fabricated, it has been deleted, and it is not evidence (`results/PREREGISTRATION.md`). The claim is retracted. Nothing in this paper depends on it.

The measured v3 gate failed. The pre-registered v3 gate was A3 − A0 ≥ +8.0 points pass@1 on MBPP-sanitized with `qwen2.5-coder:1.5b`. Both measured runs **FAIL** that gate:

- $n = 60$, CPU according to the file name and README (the file itself records hardware "unspecified"): A3−A0 = +6.67 points, 95% CI [1.67, 13.33], 4 gained / 0 lost, exact McNemar $p = 0.125$; verdict FAIL (`results/results_n60_cpu.json`).
- $n = 257$, L4 GPU: A0 64.2 %, A3 71.98 %, A3−A0 = +7.78 points, 95% CI [4.67, 11.28], 20 gained / 0 lost; verdict FAIL against +8.0. This file's `git_commit` field is empty (`results/results_n257_l4_gpu.json`).

At $n = 60$ the difference is not significant by the exact McNemar test ($p = 0.125$, 4/0 discordant pairs). The percentile-bootstrap interval there is not informative. When every discordant pair favours A3, a resample can reach $\Delta \leq 0$ only by drawing no discordant item at all, so the lower bound is positive almost by construction. Only the $n = 257$ run supports a nonzero effect (20/0 discordant; the file rounds the exact McNemar p to 0.0, i.e. $p < 0.001$). Both runs missed the registered +8.0 threshold, and we report both as FAIL.

The same file shows the v3 gate weakness that motivates v4. Of the answers the gate called VERIFIED (coverage 80.93 %), only 88.94 % passed the hidden tests. By arithmetic on those two file values (not a measurement), about 8.95 % of all problems were confident-wrong under v3.

2 Corrections since v3.3.1

An independent audit checked 99 claims of the v3.3.1 paper against raw rows, generations, the ledger, logs and live external records (`docs/CLAIM_AUDIT.md`; summary in Appendix A). It found 4 WRONG and 3 OVERSTATED claims. Each one, and how it is fixed here:

- **A32 (wrong hash).** v3.3.1 quoted the sha256 of `papers/numbers_v4.json` (v3.3.0) as `e5488f6c...`. That value is the file as committed in `b5be32a`, before the spend repricing. The file at tag `v3.3.0` hashes to `fccb1695...5995`. I recomputed this with `git show v3.3.0:papers/numbers_v4.json | sha256sum`. Fixed in Section 12.
- **A33 (wrong hash).** The quoted v3.3.1 hash `952bfc77...` belonged to an uncommitted scratch regeneration. The file at tag `v3.3.1` and on HEAD hashes to `5f3c2a6e...a1c0` (recomputed). Fixed.
- **A28 (stale spend).** v3.3.1 reported 5 ledger lines totalling $1.54. That was true when v3.3.1 was released, but later TPU runs added lines. Section 11 now gives the full ledger; for revision 3.4.0 it held 8 lines, $2.5289 in total, and for revision 3.5.0 it holds 11 lines, $4.9136.

- **C12 (bibliography).** The `gwayav3` title now matches the Zenodo record: "GWAYA v3.1: Fail-Closed Gate & Real Toolchain Verification for LLM Code Generation".
- **B15 (H13 "by construction", overstated).** v3.3.1 said the ledger refuses any stage that would exceed its cap. The ledger file enforces nothing. Corrected wording (Section 8): the GPU stage launcher is designed to refuse a stage that would push the ledger over its cap, so H13 was expected to hold largely by construction (red-team prior 0.75). The TPU runs of v3.3.1 were launched by separate scripts with a run-time limit but no ledger check, so the low total reflects low usage. In either case it is not evidence about the GwenLaya system.
- **B12 (CPU-vs-TPU check, overstated).** v3.3.1 called TPU run #3 a consistency check "across hardware and quantisation". In fact the engine, the code system prompt and the prefill/stop also changed. The run #3 reading now uses the audit's wording (Section 10). The new same-protocol comparison A7 (Section 6.5) replaces it as the CPU-vs-TPU comparison.
- **A README claim (C17, overstated)** was fixed in the README in commit `39a5a0a`. It is not part of this paper.

Audit notes (VERIFIED_WITH_NOTE or NOT_VERIFIABLE) that are addressed in the text:

- Rust assert counting (B3): the harness counts `assert!`, `assert_eq!` and `assert_ne!` calls.
- ECE binning (A5): ECE is computed with 15 equal-mass bins.
- Run #2 (D12, B8): the run #2 *scores* are not used; only its generation count is cited. The reason is the run #2 gate self-test (`python_ok False`), not D29, which describes the re-scoring machine.
- TPU-host Python failure in run #3 (B9): the cause is not recorded, and the gate self-test on the same host passed.
- Registration timing (D11): the hypotheses were registered before any GwenLaya evaluation data were generated (registration first committed in `0b4f104`, 2026-10-07 23:33). This was after the TPU smoke tests, which produced no study data.
- "72 minutes" for L2 (A30) is the sum of sequential per-call times, not a start-to-end timestamp.
- The run #3 timers (A24) are self-reported by the run.
- LoRA (B14): the absence of a TPU LoRA path is limited to the stacks inspected. vLLM-TPU LoRA was tried only on Qwen2.5.
- Laya (C14): we did not verify that the `convaiinnovations/laya` model is an encoder, and the text now calls it a model.
- MultiPL-E (C2): volume, number, pages and DOI were added to the bibliography.
- Missing scripts (B4, B10): the scripts behind the v3.3.1 TPU and Rust-validation numbers are now committed (`scripts/validation/rust_reference.py`, `scripts/validation/rescore_rust.py`, `scripts/tpu/descriptive_run3/`, commit `39a5a0a`). I did not re-run them for this revision.

2.1 Corrections since 3.9.0 (revision 3.10.0)

- **"No Lean result is reported"; "A18 generation is in progress"; "the $k = 8$ sampling is being re-run"** (3.9.0 abstract, Sections 7.17 and 11, limitations, H11 row).

Superseded: the greedy generations of A18 were scored (Section 7.11). The $k = 8$ sampling was not re-run: the stored sample files are empty, the ledger has no further Lean generation line, and any-of-8 proof rates are not reported (D66, D75). The 3.9.0 file `lean\_sanity\_summary\_390.json` still says "generation has not been run"; it is a 3.9.0 record and was not edited.

- **"A Lean miniF2F run (A18) is planned"**: it ran, with one tier more than planned (0.8B, D66), a timeout re-scoring (D67) and exact-versus-definitional reporting (D68). The A18 result files were first committed in this revision, copied from the data disk (D75).
- **"Spend includes the A18 generation, in progress"**: the A18 lines ($1.5220) were the last Lean generation; the four lines added since are F and A22 (Table 44).
- **"E′ has only 169 Python tasks and mostly easy math"** (limitations, next step 4) is partly addressed: F adds 407 Python and 501 math tasks, with harder strata and a competition stratum that is too long for the 1,024-token limit to be informative.
- **A15.3 was not in 3.9.0.** The plan (`f4af1e4`, 2026-10-10 07:35:49), code (`e556339`, 07:38:01) and results (`b2920c8`, 08:20:49) were committed on a separate branch (merge `3b9e9df`) around the 3.9.0 paper commit (`2800549`, 08:10); the audit of 3.9.0 listed them as out of scope and the paper did not mention them. They are in Section 7.12.
- No registered number and no number of 3.9.0 changed. (The independent audit of 3.9.0 gave no conclusion-changing item; its W1 wording fix was applied in `c6d5a8e`.)

2.2 Corrections since 3.8.0 (revision 3.9.0)

- **Math headline, 0.026 against 0.388** (abstract (iii) and Section 6.6, 3.5.0 to 3.8.0). The baseline 0.388 ("everything answered") counts every math task without a final `\boxed{}` as confident-wrong; for the 4B these are 160 of its 194 wrong rows, mostly generations truncated at 1,024 tokens, which a deployed system would not present as answers. The fair comparison is with answering whenever a boxed answer exists: 0.068 (coverage 0.680) with the registered scorer, against the gate's 0.026 at coverage 0.454. Under the exploratory re-score (next item) it is 0.016 against 0 of 500 (230 verified). Both baselines are now given wherever the headline appears (`rescore_exploratory.json`, keys `cwr_all` and `answer_if_boxed_confident_wrong`).
- **Math scorer (D62; commits `f837c9f`, `2305d90`, `5e14bd8`, merged in `becb57f`).** The registered scorer missed notational equivalences (brace-less `\frac14`, units on the gold only, a leading `x =`, base subscripts, spacing inside tuples, mixed numbers). A post-hoc re-score of the stored E math answers flips 22, 26 and 25 labels wrong-to-right (none right-to-wrong; 2B, 4B, 9B), so math accuracy becomes 0.558, 0.664 and 0.692 instead of 0.514, 0.612 and 0.642, and the math gate's VERIFIED-but-wrong count becomes 2, 0 and 0 instead of 7, 13 and 10 (Table 10). The scorer change was designed after inspecting the stored outputs, so these numbers are **exploratory**; the registered numbers are unchanged and remain the primary record. The re-score was **not** propagated into the matched-coverage comparison A4, the cascades A5 and A12, the abstention models A11, or the Laya training and E′ labels, all of which use the registered math labels.
- **"Python answers were generated against prompts that showed no tests"** (plan addendum A17) is false (D63). All 373 MBPP prompts of E print the gate's assert verbatim ("Your code should pass this test: ..."), and 71 of the 156 HumanEval prompts carry doctest examples (`prompt_test_visibility_390.json`). The 3.7.0 limit that the stored Rust answers "were produced from prompts that showed one test" (Section 7.8) was inaccurate for MBPP-Rust and only partly true for HumanEval-Rust: the gate assertion itself is in no Rust prompt, but in 87 of the 155 HumanEval-Rust prompts (about 17% of

E Rust) the gate's first call and its expected value are shown as a doc-comment example (152 of the 155 carry doc-comment examples); none of the 352 MBPP-Rust prompts shows a test.

- **"Python was not covered"** by the depth dial (3.7.0) is superseded: A17 covers it (Section 7.15).
- **Stale spend.** The 3.8.0 abstract, hypothesis table and H13 paragraph still gave $17.24 and 24 lines, although the 3.8.0 ledger had 25 lines and $17.7581 (Section 11 was right). Fixed; the ledger has since grown by the A18 Lean generation (Section 11).
- **"Lean 4 did not run"** (limitations) is refined: the Lean gate was repaired and hardened, A18 is pre-registered, and its generation is in progress; no Lean result is reported (Section 7.17).
- No registered number changed.

2.3 Corrections since 3.7.1 (revision 3.8.0)

- **"No ablation was run" and "the text adds little"** (3.6.0 to 3.7.1: abstract of 3.6.0, Sections 7.2 and 7.6, limitations, next step 3) are superseded in part. A text-only ablation (A16) now exists (Section 7.18). It shows the text alone is predictive on E′ (AUROC 0.839 to 0.870) and not on R′. It does *not* remove the text from the full Laya, so what the text adds to the gate and log-prob inputs is still not measured.
- **Spend.** 24 lines and $17.2421 become 25 lines and $17.7581 (one line, $0.5160, the ablation run).
- **Found by the independent audit of 3.8.0 (`docs/CLAIM_AUDIT_380.md`); no conclusion changed.** (1) The convention for the R′ answers with a missing log-prob in the text-minus-log-prob differences was not stated; it is now (Section 7.18), with the result under the other convention. (2) The 3.6.0 reproducibility entry for `scripts/train_laya.py`, and the older entries of the plan, `DEVIATIONS.md` and the ledger, are now labelled as of their revision (living files). (3) The order of events now discloses that the ablation VM was created about 4 s before the training-code commit; the plan commit preceded everything that was run. (4) The 3.6.0 sentence that the text "adds little" is rewritten in place. (5) The wording that nearly all of Laya's value on R′ "comes from" the gate and log-prob inputs is now a hypothesis.
- No number of 3.7.1 changed.

2.4 Corrections since 3.7.0 (revision 3.7.1)

An independent audit of 3.7.0 (`docs/CLAIM_AUDIT_370.md`) found five problems; no result changed.

- **Score labels.** The values quoted for Laya's score inside and outside the gate-verified set were the raw head outputs, called P(correct). They are now given both raw and calibrated (through the frozen isotonic map), labelled, and `rfresh_gate_laya_overlap.json` was regenerated; the conclusion that Laya's top-k is the gate-verified set holds on both.
- **Hash entries.** Three 3.6.0 entries (plan, `DEVIATIONS.md`, 21-line ledger) are now marked as of 3.6.0; those files are living and have changed.
- **A15 timing.** The edition run had started (log created 18:56:48) before A15 was committed (18:57:22); no result had been seen. Disclosed in Section 7.8.

- **Flaky item in the depth dial.** `rs/mbpp_130_max_occurrences` is non-deterministic for the 4B at $k = 2, 3$; both outcomes are now given, and the 4B dial is accurate to about one task.
- **Headings.** `DEVIATIONS.md` said D53–D57 and one paper sentence said D53–D57 although D58 exists; the deviation table listed D58 before D57.

2.5 Corrections since 3.6.0 (revision 3.7.0)

- **"Rust has no fresh evaluation" and "nothing about Rust is claimed from E′"** (3.6.0 abstract, Section 7, limitations) is superseded: a fresh Rust set R′ now exists (Section 7.7). The sentence that nothing is claimed about Rust from E′ stays true; what R′ shows about Rust is that H1 and H3 are *not* supported there.
- **Any reading that Laya generalises across domains.** The 3.6.0 abstract, hypothesis table and router paragraph stated H1 as "supported" and the router cost ratio 0.79 without saying that E′ is math and Python only. That reading is withdrawn: H1 is supported on E′ (math, Python) only and is not supported on the fresh Rust set R′; the router saved cost on E′ and did not on R′ (cost ratio 0.987). The 3.6.0 limitation "Rust has no fresh evaluation" no longer describes the evidence.
- **"Rust could not be assessed"** (3.6.0 next step 4, "build a fresh Rust evaluation set") is done in a limited form (69 tasks, filtered by a reference check, public repository). Harder fresh math and Python sets are still open.
- **Rust harness property, measured.** The Rust oracle ran `rustc` without `-edition`, that is, edition 2015 (D55), in every Rust result of E in 3.4.0 to 3.6.0 and in R′. A15.1 re-checked all stored Rust answers under edition 2021 (Section 7.8): at most one confirmed hidden-test flip per tier, always fail to pass, below the 2 percent threshold fixed in advance, so no number was re-analysed and all published Rust numbers stay labelled edition 2015. The statements in the first drafts of this revision that the oracle default stays 2015 and that the edition sensitivity was not run are superseded: the default is now 2021 (D58).
- **Spend and ledger counts.** The 21 lines and $15.9608 of 3.6.0 are now the first 21 of 24 lines; the total is $17.2421 (Section 11). H13 text updated.

2.6 Corrections since 3.5.0 (revision 3.6.0)

- **"No pre-registered test has been completed"** (3.5.0 abstract) is superseded for H1 and H3: a deviated test on E′ ran (Section 7.2); H1 is supported, H3 is not. Nothing was tested in the registered form, and the other hypotheses were not tested.
- **"Both primaries need the Laya calibrator and a split C, and neither exists yet"**, "(no Laya head was trained)", "no Laya training has been done" and "Train the Laya head and build split C" (3.5.0 abstract, limitations and next steps) are superseded: a Laya was trained on TPU and C exists (Section 7).
- **H1 and H3 "not run"** in the 3.5.0 hypothesis table are superseded (Table 55). H4, H5 and H6 stay "not run": the exploratory Laya analyses of 3.6.0 are on E′ and with other metrics, so they are not those tests.
- **"The 27B was not attempted (budget)"** (3.5.0 A12 and D44) is superseded: the 27B was attempted in 3.6.0 and did not finish loading in 33 minutes on a four-chip slice (D51).
- **"The training steps (Laya head, SFT) therefore stay on the GPU/CPU plan"** (3.5.0, Section 9) is superseded for the Laya head, which was trained on TPU with torch_xla and peft. The statement that no TPU path was found for LoRA of the Qwen3.5 generator

stands, with the added measurement that a torch_xla and peft probe on Qwen3.5-0.8B finished no step in 25 minutes.

- **"A11 is not the Laya head"** remains true; A11 and the Laya of 3.6.0 are different scores and should not be merged.
- **Analysis-script bug (found by the independent audit, fixed).** `scripts/analyze_laya.py` computed the ECE and Brier of the raw log-prob baseline from the negative mean log-prob instead of exp(mean log-prob). Only those two baseline entries were wrong (a first draft of this revision quoted ECE 0.76 for the 9B); they are now 0.336 / 0.341 (2B), 0.275 / 0.288 (4B) and 0.243 / 0.260 (9B) for ECE / Brier. AUROC, AURC, all inference results and the router results are unchanged.
- **Spend and ledger counts.** The 11 lines and $4.9136 of 3.5.0 are now the first 11 of 21 lines; the total is $15.9608 (Section 11). H13 text updated.
- **Limitations "E is spent for registered H1/H3 ... a registered test needs fresh generations on a new set E′"** is acted on, with the deviations listed; E itself is still burned.

2.7 Corrections since 3.4.0 (revision 3.5.0)

- **The math-gate gap statement is superseded.** Revision 3.4.0 said that the math gate was not implemented and had coverage 0 (abstract, Sections 6.4 and 13, D37, the hypothesis table and the next steps). The registered program-of-thought gate is now implemented (`gwaya/domains/math_gate.py`, addendum A10) and its results are in Section 6.6. The statement was true of the 3.4.0 run. It is no longer true of this revision. D37 stays in the record, with D43 added.
- **3.4.0 numbers that depended on the empty math gate are superseded, not deleted.** These are the math and pooled rows of Tables 5 and 6, the 757 cascade abstentions, the pooled selective figures (coverage 0.507, confident-wrong rate 0.103), the cascade-to-4B cost ratios 1.357 and 1.243 (pooled) and 1.514 (math), the pooled accuracy 0.604 and the pooled McNemar counts 40 and 44. Where they still appear in Section 6.4 they are labelled as 3.4.0 values. The current values are in Tables 8 and 9. The Python and Rust figures of those tables did not change. The pooled A4 difference (-0.063 in 3.4.0, diluted by 500 empty math tasks) is replaced by -0.090.
- **Spend.** 3.4.0 reported 8 ledger lines and $2.5289. The ledger now has 11 lines and its recomputed sum is $4.9136 (Section 11).
- **Cost of gated math arms.** The 3.4.0 cascade did not charge anything for a math gate because none ran. From A10 on, every gated arm is charged for the program that its gate executes, for each tier it consults.

2.8 Corrections after a re-audit of revision 3.4.0

A second independent audit checked 144 claims of the 3.4.0 draft (`docs/CLAIM_AUDIT_340.md`). All E-set numbers reproduced, but it found 4 WRONG and 8 OVERSTATED rows (5 distinct issues) in explanations, definitions and timing. All are fixed in this text:

- **Pooled A4 (E47).** The old pooled -0.042 used one log-prob cut-off across all domains, so the baseline answered 280 math tasks that the gate never answers. It is replaced by a pooled difference with coverage matched within each domain (-0.063, Section 6.4).
- **"Threshold tuned on E favours the baseline" (AB11, E45).** This was unsupported and is removed. The baseline answers the k most confident tasks, with k equal to the gate's

answered count in the same domain. No outcome labels are used to choose k, and ties are broken pessimistically. We claim no direction of bias.

- **Commit attribution of the scoring runs (E25).** `results.json` records the HEAD at the *end* of a run. The scoring runs started before the commits recorded there (Section 6.3).
- **"Test H1/H3 as registered on these generations" (NS1).** This is impossible: the registration freezes the calibrator and thresholds before E is generated (Section 8).
- **Plan timing (AB4, E3).** The 2B/4B scoring started 60 s *before* the plan commit.
- **Engine concurrency (E14).** The engine logs report KV capacity for at most 54 (2B) and 11 (4B) concurrent requests, not the configured 96 and 32. The fatal 4B error at 96 is now quoted correctly.
- **Math gate (E42; superseded in 3.5.0 by Section 6.6).** The zero math coverage was an implementation gap in our checker, not a property of math tasks.
- **Smaller items.**
  - A throughput typo: 4,865, not 4,866, tok/s.
  - Cascade abstentions are now reported.
  - The cluster bootstrap is effectively item-level, and this is now stated.
  - The Python cascade interval is called borderline.
  - Truncated answers are described as having "no complete `\boxed{}`".
  - Truncation analysis is in the plan, not post hoc.
  - The hash files were split by root.
  - A warm-up-excluded cost (S2) is now reported next to the planned cost.

2.9 Corrections made in v3.3.1 (history)

Revision v3.3.0 contained a wrong Rust result caused by our own harness. v3.3.1 corrected it and recorded four deviations (D27–D30). All were found and fixed after the night generations existed. No registered test had run, so none was written with a test result in view.

- **D27, Rust harness (v3.3.0 Rust result withdrawn).** Inside the bubblewrap sandbox, `rustc`'s linker `cc` resolves through `/etc/alternatives`, which the sandbox did not mount. As a result, *every* Rust program failed to link, and the v3.3.0 Rust result (0/26) was a harness artefact.
  - **Forgery.** The harness also wrote its pass nonce into `main.rs`, so a candidate could forge a pass. We reproduced this and closed it: the nonce now goes on stdin, compile and run use separate sandboxes, the run sandbox has no `/proc`, and `unsafe`, `link_section` and `asm` are rejected.
  - **Assert counting.** The harness counts `assert!`, `assert_eq!` and `assert_ne!` calls.
  - **Validation.** Hand-written reference solutions pass for 25 of the 26 night Rust tasks (25/25), and 9 deliberately wrong solutions all fail (9/9) (`results/gwenlaya_v4/rust_harness_validation.json`).
  - **Re-score.** The unchanged night generations now give Rust 6/26.
- **D28, math prompt path and scoring.** The night math generations went through the code path, so they remain invalid. For future runs the code was fixed in three ways: a prose math prompt that ends in `\boxed{}`, non-thinking mode for Qwen3.x, and boxed-only scoring as the default. Under boxed-only, all 26 night math generations are UNDECIDED.

- **D29, Python harness environment.** A `uv`-managed interpreter could not start inside bubblewrap on the re-scoring machine. This was fixed by recreating the interpreter symlinks. The Python result is unchanged (11/28).
- **D30, TPU run #3 (descriptive).** A third TPU run generated all 80 night tasks for three models. Python scoring on the TPU host failed for every item, for a reason that was not recorded. All TPU generations were therefore re-scored on our machine.

3 What GwenLaya is and what was registered

The full registration is `docs/GWENLAYA_PREREGISTRATION.md` (revision 2, design freeze; sha256 of each revision in `docs/PREREG_FREEZE.txt`). In brief:

- **System (GL).** The design has the following steps:

  1. Generate with a small Qwen3.5 tier.
  2. Run a *gate-visible* executed check that, by design, never sees the scoring tests. In the night task files this was violated for HumanEval+, whose gate payloads contained the whole hidden suite. No gate ran, and the files were rebuilt (D25, Section 5.1).
  3. Score the candidate with a calibrated confidence head (a LoRA head on the `convaiinnovations/laya` model).
  4. Answer VERIFIED or LIKELY_CORRECT(p), abstain, or escalate to a larger tier.

  The calibration label is the outcome on the *hidden* checks, so VERIFIED is never treated as ground truth.
- **Eval pool E.** HumanEval+ and MBPP+ [11] (Python, 542 items), MultiPL-E Rust [3] (510), MATH-500 [7, 10] (500), and miniF2F Lean 4 (244, the registered size; outside the pooled analysis).
- **Arms.** B1 smallest tier alone; B2 largest tier alone; B3 log-probability selective prediction [4] with its threshold set on a separate split C; B4 self-consistency; B5 largest tier with the v3-style gate only; B6 Laya zero-shot; LR; and a shuffled-label negative control SH.
- **Primary family** (Holm [8], $m = 2$; a primary that does not run enters with $p = 1$, so the other is tested at 0.025).
  - H1: CWR(GL)−CWR(B3) at matched coverage, by exact one-sided McNemar [12] plus a cluster-stratified paired bootstrap; MEI 3.0 points.
  - H3: GPU-seconds per correct answered item, GL vs B5; MEI 25 % reduction, with answered accuracy non-inferior by 2.0 points.
- **Secondary:** calibration (H4 ECE [6], H5 Laya vs LR AUROC, H6 split-conformal selective error [1]), quantization (H8), training (H9), and systems (H13 budget, H14 endpoint latency, H15 preemption). **Exploratory:** H2, H7, H10–H12.

The hypothesis wording and MEIs are unchanged. VERIFIED keeps its registered meaning: all gate-visible checks passed. The outcome on the hidden checks is written PASS_HIDDEN / FAIL_HIDDEN / UNDECIDED.

4 The night run and its deviations

The night of 2026-10-07/08 was meant to run the primaries cheaply. Two constraints shaped it:

- the total GCP budget ($50 hard, $45 working ceiling, $40 planned);

- the project's only GPU quota unit (`GPUS_ALL_REGIONS` limit 1), which a VM outside this project held all night. We did not touch that VM.

All night generation therefore ran on a shared 8-core CPU, and the scope was cut. Every deviation is in `docs/DEVIATIONS.md`. The night deviations are summarised in Table 1, and those of revisions 3.4.0 and 3.5.0 in Tables 15 and 16 (Section 6). When each group was written:

- D1–D20: before generation.
- D21: during L2.
- D22–D23: after the L2 generations existed.
- D24–D26: after the external review of the first draft.
- D27–D30: after release v3.3.0.
- D31–D41: during and after the full-E TPU baselines of revision 3.4.0.
- D48–D52: revision 3.6.0, committed before E′ generation (Table 49).
- D43–D47: revision 3.5.0, all after the E baselines had been seen (Table 16). The D42 note concerns the burned E set (Section 8).

No registered test has run, so none of these was written with a registered test result in view. Some of D31–D41 and all of D43–D47 were written after exploratory results had been seen, and they say so.

Benchmarks reduced. To stay within budget and wall clock, the night ran no benchmark beyond the three primary domains, on 80 of their 1,541 included items, with two tiers instead of four. Lean 4, GSM8K, AIME25, LiveCodeBench, the MBPP continuity anchor, the 0.8B tier, self-consistency and the second seed were dropped (D7–D10).

Power was known to be insufficient before generation. The freeze addendum computed the power of H1's exact McNemar test at $n = 80$ before any item was generated: 0.052 at the 3-point MEI and 0.261 at a 6-point effect ($\alpha = 0.025$). These values depend on the assumed discordance rates.

What actually ran at night.

- L0 (environment, throughput) and L1 (manifest, freeze addendum, decontamination).
- L2: 2B generation on all 80 E-night items, about 72 minutes as the sum of sequential call times.
- Scoring of those 80 generations.

L3 (4B generation) was held because the L2 review found a blocking defect. G1 (GPU) and E1 (endpoint) never ran, because the GPU slot never freed.

5 Night results (CPU, 80 items, descriptive)

The scored arm `base` is Qwen3.5-2B q4_K_M on llama.cpp [5], greedy, thinking off, `max_new_tokens` 1024, with no gate, no calibrator and no escalation. It is not one of the registered arms. Confidence is the raw exp(mean token log-prob), uncalibrated and not cross-fitted. Intervals are 95% cluster-stratified bootstrap percentile intervals (10,000 resamples, seed 0), and ECE uses 15 equal-mass bins. Every number below is read from `papers/numbers_v4.json`, written by `scripts/analyze_study.py` from the re-scored rows `results/gwenlaya_v4/night_L2fix/rows.jsonl`. Its tables are reproduced unedited in Appendix B.

Table 1: Deviations from the registration for the night run (from `docs/DEVIATIONS.md`). Reason codes: B = budget, G = no GPU slot, T = CPU wall clock, I = implementation or defect.

#	Registered	Night run	Why
D1	E = 1,552 items on L4	E-night: seeded prefix, 80 items (python 28, rust 26, math 26)	G, T
D2	tiers 2B/4B/9B/27B	2B and 4B only (B2/B3/B5 on 4B)	G, T
D3	H3 cost in L4 GPU-seconds	CPU-seconds on this machine	G
D4	separate calibration set C	5-fold cross-fitting on E-night by source cluster	T; C was not label-matched
D5	Laya LoRA r=16 heads	LoRA only if CPU timing fits, else frozen-embedding probe	T
D6	llama.cpp server	llama.cpp b11476 CPU; Ollama not used (no logprobs)	I
D7	Lean 4 (E, C, H11)	dropped tonight	T, B
D8	SFT, DPO, S4b–S5	deferred; H9 deferred, H10–H12 dropped	G, B
D9	B4 self-consistency, second seed	dropped	B, T
D10	GSM8K, AIME25, LiveCodeBench, MBPP anchor, 0.8B	dropped	B, T
D11	H8 on full E	300-item 9B subsample only if GPU; no TOST	G
D12	endpoint, 4–6 cold samples	4 samples, only after G1	G, B
D14–D15	`-t 8`; 256-token tok/s probe	`-t 4`; 64 tokens	T (foreign CPU load)
D16	–	fix: multi-letter words in math checker return undecided	I
D17	reference passes its own checks	Rust: typecheck only (no reference solution)	I
D18	decontaminate T, C	all 170 MBPP-sanitized calib items removed; no T/C	I
D19	n from measured throughput	n from measured tok/s and an *assumed* 196.6 tokens/item	T
D20	prompt text not fixed	gate-visible tests and prompts frozen in the task file	I
D21	–	logprobs and per-call CPU-seconds captured	I
D22	`random.Random(0)` bootstrap; primary Holm $m = 2$	numpy `default_rng(0)`, 10,000 resamples; Holm over the primaries that ran, reverted in D26	I
D23	–	scored the 80 L2 generations; single-arm descriptives added	I
D24	math: boxed answer only	scorer fell back to the last number; registered boxed-only rescoring reported too	I (review)
D25	gate never sees the scoring checks	HumanEval+ gate payloads held the full hidden suite; rebuilt, 5 primary items excluded; no gate ran	I (review)
D26	Holm $m = 2$	restored; per-domain descriptives, Python headline	I (post-L2)
D27	Rust: compile and run all hidden asserts	sandbox could not link any Rust program; fixed, nonce forgery closed; validated 25/25 correct, 9/9 wrong	I (post-v3.3.0)
D28	math: boxed answer, math prompt	night math went through the code prompt path (still invalid); prompt and boxed-only default fixed	I (post-v3.3.0)
D29	Python checks in bwrap	`uv`-managed interpreter could not start in the sandbox; fixed; Python unchanged	I (post-v3.3.0)
D30	–	unregistered descriptive TPU run #3 (3 models, 80 tasks); all rows re-scored locally	I, G (post-v3.3.0)

Outcome labels. The per-row outcome here is the result of the *hidden* checks (`rows.jsonl` field `score`: VERIFIED, FAILED or UNVERIFIED, i.e. PASS_HIDDEN, FAIL_HIDDEN, UNDECIDED). It is not the registered VERIFIED. The `gate` field is null in all 80 night rows.

Why Python is the headline. Python ($n = 28$) was chosen as the headline in D26, before the Rust harness was fixed, and we keep that choice. Rust ($n = 26$) is a second descriptive domain. The math and pooled numbers are sensitivity rows only, because the night math generations are invalid.

Table 2: Night CPU arm `base`, raw confidence, descriptive (from `papers/numbers_v4.json`). Pass = PASS_HIDDEN. 95% cluster bootstrap CI in brackets. ECE: 15 equal-mass bins. Cost = server CPU-seconds per passing item (D3, D21). TBD = undefined (no passing item).

Subset	n	Pass rate	Mean conf.	ECE	AUROC	Cost / pass
Python (headline)	28	0.393 [0.214, 0.571]	0.896 [0.875, 0.917]	0.519 [0.392, 0.689]	0.717 [0.483, 0.917]	330 [174, 726]
Rust (descriptive, D27)	26	0.231 [0.077, 0.385]	0.894 [0.882, 0.906]	0.675 [0.549, 0.813]	0.875 [0.709, 0.990]	494 [261, 1552]
Sensitivity only, not valid estimates						
Math (invalid), boxed-only	26	0.000	0.832 [0.807, 0.857]	0.832 [0.807, 0.857]	TBD	TBD
Pooled (boxed-only math)	80	0.212 [0.125, 0.300]	0.875 [0.863, 0.886]	0.662 [0.585, 0.741]	0.806 [0.675, 0.917]	970 [629, 1648]

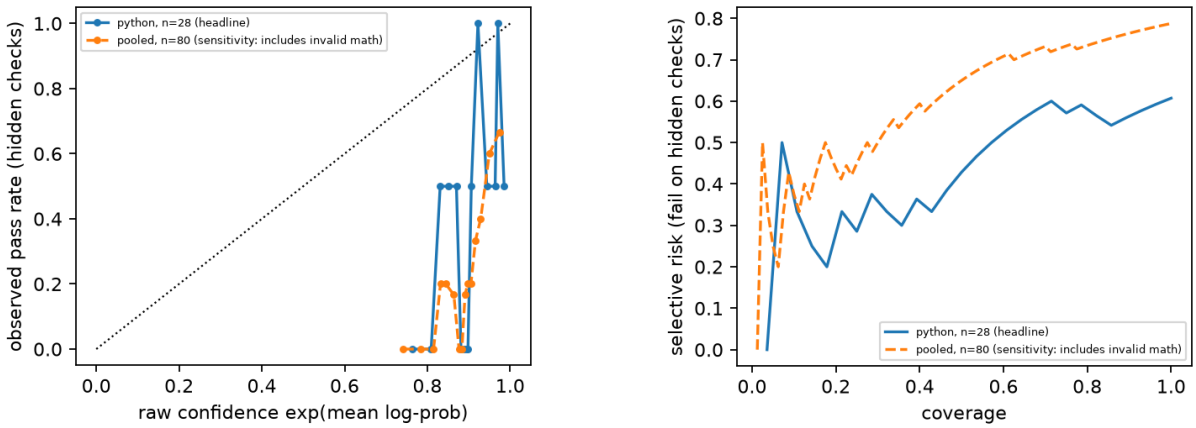


Figure 1: Night CPU arm `base`, raw confidence (v3.3.1 re-scored rows). Solid: Python, $n = 28$ (headline). Dashed: all 80 items. This is a sensitivity curve only, because it contains the invalid math items. Left: reliability diagram (equal-mass bins). Right: risk–coverage curve, where risk is the share failing the hidden checks. Descriptive only; not cross-fitted.

Python. 11 of 28 items passed the hidden checks and 17 failed (pass rate 0.393, CI 0.214–0.571). The mean raw confidence was 0.896, so the score is badly calibrated (ECE 0.519; Brier 0.476, CI 0.334–0.617). The AUROC interval (0.483–0.917) includes 0.5. The confident-wrong rate, counting every answered item that did not pass, was 0.607 (CI 0.429–0.786). The cost per passing item was 330 server CPU-seconds.

Rust. 6 of 26 items passed (0.231, CI 0.077–0.385). Mean raw confidence was 0.894 (ECE 0.675). The AUROC point estimate was 0.875 (CI 0.709–0.990). There were 6 passing items, one seed and an uncalibrated score, so we do not read this AUROC as evidence of good ranking. The harness was validated after these generations existed, on hand-written solutions, not on an independent reference set.

Math (not interpretable). Under the registered boxed-only rule, all 26 night math items are UNDECIDED. None of the generations contains `\boxed`, because they were produced through the code prompt path.

5.1 Defects found in the night data

1. **Math prompt path (blocking).** All 26 math completions were produced with the Python code system prompt and a ```` ```python ```` prefill, because the prompt builder fell back to the `python` tag. The fix is in the code (D28). The 26 night items have not been regenerated on CPU. The math generations of Section 6 use the fixed prompt.
2. **Unregistered math answer extraction (D24).** The v3.3.0 scorer fell back to the last number in the text. Since v3.3.1 the registered boxed-only rule is the default.
3. **Rust harness (D27).** No Rust program could link in the sandbox, and the pass nonce could be forged. Both are fixed and validated (25/25, 9/9).
4. **Python harness environment (D29).** Fixed; the Python result is unchanged.
5. **Gate not recorded at night.** The `gate` field is null in every night row.
6. **Hidden tests in the HumanEval+ gate payload (D25).** 11/11 E-night and 161/161 primary HumanEval+ gate payloads contained the hidden `inputs`/`results` lists. No reported number changes, because the L2 run executed no gate. The task files were rebuilt with the prompts and hidden checks unchanged. E-night has 80 items; primary E has 1,536 items after 5 fail-closed exclusions. All E-set runs in Section 6 use the rebuilt `.d25` primary file.

What "passed" means in this paper. An item is PASS_HIDDEN only if it passed the named hidden checks:

- Python: the full EvalPlus test suite, run in a bubblewrap sandbox.
- Rust: compilation with `rustc`, then all `assert!`/`assert_eq!`/`assert_ne!` calls of the MultiPL-E test, run in a separate sandbox.
- Math: sympy/numeric equivalence between the gold answer and the last `\boxed{}` answer. If there is no `\boxed`, the item is UNDECIDED and is not counted as correct.

It means nothing more.

6 Exploratory full-E baselines on one TPU v5e chip (not registered arms)

This section is the main addition of revisions 3.4.0 and 3.5.0. Sections 6.1 to 6.5 report the 3.4.0 analyses (with the 3.5.0 supersessions marked); Sections 6.6, 6.7 and 6.8 are new in 3.5.0. None of it is a pre-registered test. H1 compares GL with B3 and H3 compares GL with B5. GL needs the Laya router and calibrator, and B3 needs a threshold set on split C. None of these existed for the E analyses below; Section 7 (revision 3.6.0) trains them and runs the comparison on a fresh set.

6.1 Analysis plan and what is post hoc

The analyses A1–A9 are defined in `docs/ANALYSIS_PLAN_E_TPU.md`, committed in `8ee9da3` at 2026-10-08 22:22:55. The original text hashes to `1f2fa2c1...0d46` (recomputed from `git show 8ee9da3:docs/ANALYSIS_PLAN_E_TPU.md`).

Timing, stated plainly.

- All generation was complete before the plan was committed.
- The 2B/4B scoring run started at 22:21:55, *60 seconds before* the plan commit, and finished at 22:36:38 (`results.json` `wall_clock`). Its log prints only progress counts, not outcomes.
- The scored rows of the Coder anchor were complete at 22:22:45, *10 seconds before* the plan commit. The Coder enters only A1, A2 and A9.
- To the best of our records, no scored E outcome was inspected before the commit. However, the descriptive 80-task runs (night CPU arm, TPU run #3) had already shown approximate pass rates for these models.
- The analysis script was committed in `5e10ad2` at 22:25.

Post-hoc amendments. Four dated amendments were made after the first analysis run and are post hoc (a fifth, after the re-audit of revision 3.4.0, is listed in Section 2.8):

1. A7 excludes math. The CPU night math came from the defective prompt, and the pooled comparison had been seen before this decision.
2. The A8 timeout audit became a correction procedure, the serial overlay.
3. The cascade cost result is reported as found, with no tuning of the gate or the tier pair afterwards.
4. A stability audit of passing rows and the fail-closed sensitivity S1 were added. This was decided before the passing-row audit finished.

Everything not listed in the plan is labelled post hoc where it appears. In particular, the warm-up-excluded cost S2 and the per-domain-matched pooled A4 were added after the re-audit. The truncation analysis, including the pass rate of truncated items, is in the plan (A9).

6.2 Generation

Setup.

- **Hardware and engine.** One on-demand TPU v5e chip (v5litepod-1, us-west4-a) and vLLM-TPU 0.31.0 [9], with the engine version taken from the logs.
- **Decoding.** bf16, greedy (temperature 0), `max_new_tokens` 1024, thinking off, with the log-probability of each sampled token recorded.
- **Prompts.** The prompts are the study's raw Qwen protocol prompts. They are built by `OllamaGenerator.build_raw_prompt` with the stop sequences of `OllamaGenerator.stop_sequences`, which is the same code path as the llama.cpp CPU arm.
- **Code prompt unchanged.** The diff of `gwaya/generators.py` between the L2 commit `64441eb` and HEAD leaves the code-domain system prompt, the ‘‘‘`lang` prefill and the ‘‘‘ stop unchanged. It changes only the math path (D28).
- **Driver scripts.** Generation used `scripts/tpu/gen_batch.py`, run by `deploy/tpu/run_e_gen.sh` on the VM, which sets `SKIP_JAX_PRECOMPILE=1`.
- **Tasks.** `tasks_E_primary.d25.jsonl`: 1,536 tasks (Python 529, Rust 507, math 500) in 1,079 source-problem clusters.

Models and concurrency. Qwen2.5-Coder-1.5B-Instruct (continuity anchor), Qwen3.5-2B and Qwen3.5-4B. vLLM loaded each model at revision `main`, so the exact weight commit is not pinned (a limitation).

Requested and effective concurrency. The configured `max_num_seqs` was 96 for the Coder and the 2B. For the hybrid Qwen3.5 models the engine logs show the KV cache being re-split. The 2B log says "re-split at 54 concurrent requests", so its effective concurrency was at most 54.

The 4B at 96 failed to start with `ValueError: Cannot fit both KV pools under gpu_memory_utilization=0.92: the mamba pool needs 193 blocks (2 per request x 96 requests) ...` (`gen_qwen3.5-4b-bf16_s96.log`). It then ran with `max_num_seqs` 32, and its log says "re-split at 11 concurrent requests". The line "Mamba and attention pools together exceed the HBM budget" is an INFO message that also appears in the successful runs.

vLLM calls the recurrent-state pool "mamba", although Qwen3.5 is a hybrid of full and linear attention layers. The 4B therefore ran at much lower effective concurrency than the 2B (at most 11 vs at most 54), which makes the 4B dearer per token (D33).

A first pass was set aside. A first E-generation VM (`gwenlaya-tpu-e-gen-210955`) produced 1,536 Coder and 1,536 2B generations. Both have `mean_logprob` null in every row, and the 4B failed at engine start. The maintainer reports that this pass requested `logprobs=0`, for which vLLM-TPU returns no log-probabilities. I did not see that pass's code. The pass was not used, and everything was regenerated on a second VM (`gwenlaya-tpu-e-gen2-214445`) with `logprobs=1`. The committed `gen_batch.py` now stops after the first chunk if no log-probabilities come back (D40). All three raw files of the second pass have a log-probability in every row (`mean_logprob` non-null for $3 \times 1{,}536$ rows). Their sha256 values match `results/gwenlaya_v4/e_tpu/SHA256SUMS_dataroot.txt` both on the data disk and in the data lake (verified).

Throughput (from the data-lake `gen_batch` logs, second pass). Each model's tasks ran in 8 chunks of 192, with every chunk mixing all three domains. Chunk 1 was much slower in every case: 62 s for the Coder, 144 s for the 2B and 192 s for the 4B. Chunks 2–8 took 9–10 s, 21–23 s and 50–55 s, a throughput of 4,507–4,865, 2,864–3,120 and 1,085–1,128 tok/s. The slow first chunk matches lazy compilation; we did not instrument it. These are batched-throughput figures, not single-stream latency.

Totals. Summed over the 1,536 generations (`data/gens_summary.jsonl`):

- Completion tokens: 353,862 (Coder), 509,089 (2B) and 464,253 (4B).
- Attributed chip-seconds: 129.4, 296.7 and 561.0. These equal the sums of the chunk wall times, *including* the slow first chunk.

6.3 Scoring and verification

Scoring. Nothing was scored on the TPU host.

- `scripts/import_remote_gens.py` wrote the raw generations into the study's hash-chained `gens.jsonl`, with the same keys and seeds.
- The unchanged `scripts/run_study.py -mode score` then scored them locally in the bubblewrap sandbox, with `GWAYA_ALLOW_UNISOLATED` unset.
- Chip-seconds are attributed by token share within each chunk.

The 2B/4B run replayed six arms over the same cached answers (`base`, `always_smallest`, `always_largest`, `gate_only`, `raw_confidence` and `gwenlaya`, the last being the static gate ladder). It used 5 parallel check workers (`-check-workers 5`, recorded in `results.json`). The Coder rows were scored with one check process; their `results.json` records no check-worker

option. Both scoring runs ran from working trees with uncommitted changes. `results.json` records the HEAD at the *end* of each run (`66e2313` for the Coder, `5e10ad2` for the 2B/4B, both `dirty: true`). That is not the code that started the run:

- The Coder scoring began at 22:05:17, before `66e2313` existed (22:08:27). Its arguments have no check-worker key, which matches the earlier command line.
- The 2B/4B scoring began at 22:21:55, before `5e10ad2` (22:25:43).

The code that ran was therefore `39a5a0a` and `66e2313` respectively, plus uncommitted changes that I cannot identify. I re-ran `tests/test_import_remote_gens.py` and `tests/test_analyze_e_tpu.py` for this revision, and 22 tests passed.

Serial re-check of every non-passing code row (A8). Parallel checks can time out under CPU contention. Every non-passing Python/Rust `base` row was therefore re-checked one at a time: 644 (2B), 412 (4B) and 480 (Coder), 1,536 in all (`audit/recheck_*.json`). Three outcomes flipped from FAILED to VERIFIED:

- `py/MBPP/271`, for the 2B and for the 4B. This is a contention timeout.
- `rs/mbpp_130_max_occurrences`, for the 4B. Plan amendment 4 identifies this as a *non-deterministic* candidate: it iterates a Rust `HashMap` and takes the first maximum, so with tied counts the result depends on the per-run hash order. According to the amendment it passes about half the time. I did not re-measure that frequency.

How the corrections were applied. The correction is an *overlay* (`overlay/rows_overlay_tiers.jsonl`, 16 rows). It holds a serial re-score of all six arms for the two flipped tasks. The original hash-chained rows are untouched, and the overlay takes precedence in the analysis.

- `py/MBPP/271` becomes VERIFIED in all 8 affected rows (`serial_recheck_corrections`).
- The `rs/mbpp_130` rows re-scored as FAILED and stay FAILED in the headline.

A bug in the first overlay was found and fixed. Re-importing the corrected tasks alone re-attributed the chunk time over far fewer tokens, which inflated those tasks' chip-seconds. The importer now keeps the original attributed seconds, and `test_subset_reimport_keeps_original_attributed_seconds` guards this.

Timeouts and passing-row stability.

- **Timeouts when run alone.** Five checks per model still time out when run alone (5 for the 2B, 5 for the 4B, 5 for the Coder, listed in the audit files). They remain FAILED.
- **Re-run of every passing row.** All 1,572 passing Python/Rust rows were re-run once: 392 (2B), 624 (4B) and 556 (Coder). None changed outcome (`audit/recheck_pass_*.json`). One re-run finds a 50/50 unstable candidate only half the time, so the correct reading is "no unstable passing row was observed", not "none exist".

Sensitivity S1 (fail-closed). S1 counts every item that produced two different outcomes in any run as not correct: 1 item for the 2B and 2 for the 4B. Pooled accuracy is then 0.4225 vs 0.4232 headline (2B), 0.6055 vs 0.6061 (4B) and 0.6029 vs 0.6035 (cascade). The largest difference in any domain is 0.0019 (4B Python, 0.6730 vs 0.6749). The headline and S1 agree well inside the confidence intervals.

Hash files. The hashes are kept in two files, so that each can be checked from its own root:

- `results/gwenlaya_v4/e_tpu/SHA256SUMS_repo.txt`, checked from the repository root;

Table 3: Pass rate on the hidden checks, full E set, greedy, one sample (95% cluster bootstrap CI). TPU v5e bf16.

Model	Python (529)	Rust (507)	Math (500)	Pooled (1536)
qwen3.5-2b-bf16	0.452 [0.408, 0.493]	0.304 [0.264, 0.345]	0.514 [0.470, 0.558]	0.423 [0.398, 0.447]
qwen3.5-4b-bf16	0.675 [0.635, 0.715]	0.529 [0.485, 0.574]	0.612 [0.568, 0.654]	0.606 [0.582, 0.631]
qwen2.5-coder-1.5b-bf16	0.647 [0.605, 0.686]	0.422 [0.381, 0.465]	0.368 [0.326, 0.410]	0.482 [0.457, 0.507]

Table 4: Raw confidence $\exp(\overline{\log p})$ as a correctness score (ECE: 15 equal-mass bins).

Model	Domain	mean conf.	ECE	AUROC	AURC
qwen3.5-2b-bf16	python	0.898 [0.893, 0.903]	0.446 [0.405, 0.488]	0.649 [0.601, 0.694]	0.455 [0.398, 0.515]
qwen3.5-2b-bf16	rust	0.902 [0.898, 0.906]	0.598 [0.559, 0.636]	0.772 [0.728, 0.813]	0.512 [0.453, 0.573]
qwen3.5-2b-bf16	math	0.902 [0.898, 0.905]	0.388 [0.345, 0.430]	0.741 [0.697, 0.784]	0.332 [0.276, 0.389]
qwen3.5-4b-bf16	python	0.927 [0.923, 0.931]	0.252 [0.215, 0.291]	0.681 [0.631, 0.728]	0.209 [0.169, 0.254]
qwen3.5-4b-bf16	rust	0.930 [0.927, 0.933]	0.402 [0.359, 0.443]	0.775 [0.733, 0.815]	0.279 [0.230, 0.331]
qwen3.5-4b-bf16	math	0.938 [0.935, 0.940]	0.326 [0.284, 0.369]	0.697 [0.646, 0.746]	0.285 [0.229, 0.341]
qwen2.5-coder-1.5b-bf16	python	0.915 [0.911, 0.919]	0.268 [0.230, 0.309]	0.722 [0.674, 0.766]	0.215 [0.173, 0.261]
qwen2.5-coder-1.5b-bf16	rust	0.934 [0.931, 0.937]	0.512 [0.470, 0.554]	0.773 [0.731, 0.812]	0.366 [0.313, 0.421]
qwen2.5-coder-1.5b-bf16	math	0.932 [0.930, 0.935]	0.564 [0.523, 0.606]	0.615 [0.565, 0.665]	0.571 [0.509, 0.630]

- `SHA256SUMS_dataroot.txt` (the full generations, rows and raw files), checked from the data root.

I ran `sha256sum -c` on each from its root, and every entry matches.

6.4 Results

Tables 3–7 are `papers/tables_e_tpu.tex` exactly as written by `scripts/analyze_e_tpu.py` for revision 3.4.0 (input, not retyped). **In 3.5.0 their math and pooled gate and cascade entries (Tables 5 and 6) are superseded by Section 6.6**, because they were computed with an empty math gate. Every number in the text is read from `results/gwenlaya_v4/e_tpu/numbers_e_tpu.json` (key in parentheses where useful). All CIs are 95% source-problem cluster bootstrap percentile intervals (10,000 resamples, numpy `default_rng(0)`). Within each domain every cluster holds exactly one item (checked on the task file), so the bootstrap is in effect item-level, stratified by domain. Python and Rust problems from the same source are resampled independently. According to the re-audit's own check, resampling them together would widen pooled CIs by at most 0.004. "Correct" means PASS_HIDDEN. FAILED and UNVERIFIED both count as not correct.

A1, pass rates (Table 3). On all 1,536 tasks the 4B passes 0.606 of the hidden checks, the 2B 0.423 and the Coder 0.482. On Python the Coder (0.647) is close to the 4B (0.675). These are single greedy samples on public benchmarks the models have probably seen. They are not clean capability estimates.

A9, truncation. Math answers are often cut at 1,024 tokens. The share of math generations with `finish_reason = length` is 0.400 (2B), 0.320 (4B) and 0.152 (Coder) (`A9.trunc.*.math`). The code shares are at most 0.053 (2B Python).

From the rows, no truncated generation passed in any model or domain. Five truncated math answers contain the string `\boxed` (1, 1 and 3 for the Coder, 2B and 4B; read from the raw files), and four of these are unclosed. The truncated math answers therefore have no complete `\boxed{}` and are UNVERIFIED, except one 2B item that was FAILED. The math pass rates are therefore lower bounds under this token budget, and the math comparison between models partly measures verbosity.

Table 5: Gate-only on qwen3.5-4b-bf16 (answer iff the executed gate verifies) versus answering everything, and versus a log-probability baseline that answers the same number of tasks (the k most confident, k = the gate's answered count in that domain; no labels are used to choose k).

Domain	coverage	precision	CWR gate	CWR answer-all	Δ vs matched log-prob
python	0.813 [0.779, 0.847]	0.830 [0.795, 0.864]	0.138 [0.110, 0.168]	0.325 [0.285, 0.365]	-0.076 [-0.098, -0.057]
rust	0.596 [0.552, 0.639]	0.887 [0.851, 0.922]	0.067 [0.045, 0.091]	0.471 [0.426, 0.515]	-0.112 [-0.134, -0.093]
math	0.000 [0.000, 0.000]	TBD	0.000 [0.000, 0.000]	0.388 [0.346, 0.432]	0.000 [0.000, 0.000]
all (per-domain matched)	–	–	0.070 [0.058, 0.082]	–	-0.063 [-0.074, -0.054]

Table 6: Exploratory gate cascade (qwen3.5-2b-bf16 → gate → qwen3.5-4b-bf16) versus always-smallest and always-largest. Cost = accelerator chip-seconds per task (throughput at each tier's effective generation concurrency, which differed between tiers; not latency).

Domain	acc. small	acc. large	acc. cascade	cost ratio casc./large	escalated
python	0.452 [0.408, 0.493]	0.675 [0.635, 0.715]	0.648 [0.607, 0.688]	0.997 [0.925, 1.071]	0.408 [0.365, 0.450]
rust	0.304 [0.264, 0.345]	0.529 [0.485, 0.574]	0.548 [0.505, 0.594]	1.221 [1.153, 1.291]	0.633 [0.592, 0.675]
math	0.514 [0.470, 0.558]	0.612 [0.568, 0.654]	0.612 [0.568, 0.654]	1.514 [1.489, 1.538]	1.000 [1.000, 1.000]
pooled	0.423 [0.398, 0.447]	0.606 [0.582, 0.631]	0.604 [0.579, 0.628]	1.357 [1.329, 1.385]	0.675 [0.655, 0.695]

A2, raw confidence (Table 4). exp(mean log-prob) sits near 0.9 in every cell, from 0.898 (2B Python) to 0.938 (4B math), whatever the accuracy. On the 4B the mean confidence is 0.927 / 0.930 / 0.938 (Python / Rust / math), against pass rates of 0.675 / 0.529 / 0.612. ECE ranges from 0.252 (4B Python) to 0.598 (2B Rust). The score is not usable as a probability without calibration.

Its ranking is weak to moderate. AUROC lies between 0.615 (Coder math) and 0.775 (4B Rust), and every CI excludes 0.5. This does not test H4 or H5, which concern the Laya head (the Laya of Section 7 is a different object).

A3, gate-only on the 4B (Table 5). The executed gate runs the gate-visible test of each task. On Python it answers 0.813 of the tasks, and 0.830 of those pass the hidden checks. On Rust the figures are 0.596 and 0.887. Over all tasks, the confident-wrong rate falls from 0.325 to 0.138 (Python) and from 0.471 to 0.067 (Rust), compared with answering everything. That reduction comes from abstaining on 19 % and 40 % of the tasks.

On math the 3.4.0 gate answered nothing (coverage 0, precision undefined). That was an *implementation gap* in our checker, not a property of math tasks. The checker executed only the task's `gate_payload`, which is empty for all 500 math rows, so every math gate verdict was UNVERIFIED. The registered math gate is a sandboxed program-of-thought re-execution that reproduces the boxed answer (registration Section 1). It needs no task payload. **It is implemented in revision 3.5.0 and its results replace the empty row (Section 6.6).**

Both code-domain precisions are below the 0.95 point that the registration uses to flag a gate weakness (H7). This is a descriptive analogue on unregistered hardware and tiers, not an H7 result.

A4, gate vs a log-prob threshold at matched coverage (exploratory; most favourable result). At the gate's own coverage, "answer the most confident tasks" gives a confident-wrong rate of 0.214 on Python and 0.180 on Rust. The gate gives 0.138 and 0.067. The differences are −0.076 (CI −0.098 to −0.057) and −0.112 (CI −0.134 to −0.093) (`A4.delta_cwr_gate_minus_b3.qwen3.5-4b-bf16.*`). Figure 2 shows the same comparison as risk–coverage curves.

Several caveats limit this result:

1. The gate executes a visible test, which is information the log-prob does not have. Part of the gap is simply "running a test helps". (Added in 3.9.0, D63: for the 373 MBPP tasks

Table 7: Same raw protocol, night Python and Rust tasks (math excluded: the CPU math used a defective prompt): Qwen3.5-2B q4_K_M llama.cpp CPU versus bf16 vLLM TPU. Hardware, quantisation and engine all differ.

Domain	n	CPU pass	TPU pass	agree	McNemar p
python	28	11	10	27	1.000
rust	26	6	8	22	0.625
pooled	54	17	18	49	1.000

the gate's assert is printed in the prompt, so the model's log-prob is conditioned on that test; the gate executes it, the log-prob does not.)

2. The baseline answers the k most confident tasks, with k equal to the gate's answered count in the same domain (recomputed in each bootstrap resample). No outcome labels are used to choose k, and ties are broken pessimistically. k is matched on E rather than taken from a threshold frozen on a separate split. We do not claim a direction for any resulting bias.
3. The benchmarks are public and probably in pretraining.
4. There is a single greedy sample.
5. This is **not** H1. H1 compares GL (gate plus Laya) with B3, whose threshold is set on a separate split C, and requires the coverage difference to lie within ±3 points.

In 3.4.0 the gate answered no math task, so the matched baseline answered $k = 0$ tasks, the math difference was 0 by construction, and the pooled difference with coverage matched within each domain (−0.063, CI −0.074 to −0.054) was the Python and Rust differences diluted by 500 empty math tasks. Both are superseded by the 3.5.0 values with the math gate (math −0.084, pooled −0.090; Table 8).

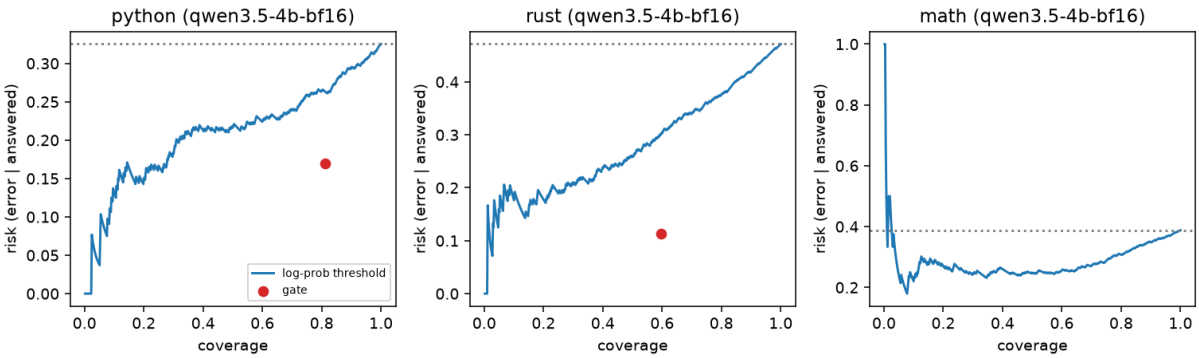


Figure 2: Qwen3.5-4B (bf16, TPU), full E set, exploratory. Blue: risk–coverage curve of a raw log-prob threshold. Here risk is the error rate *among answered tasks*, while the tables report CWR over *all* tasks. Red dot: the executed gate, whose risk is one minus its precision. Dotted line: error rate when every task is answered. The curve is traced on E by ranking tasks by confidence; no labels choose the cut-off. This is the 3.4.0 figure: on math our gate then verified nothing, so there is no red dot. Figure 4 replaces it with the math gate. The left end of each curve rests on very few tasks.

A5, the gate cascade (exploratory, not GL; Table 6, Figure 3). *All pooled and math figures in this paragraph and the next bullet lists are 3.4.0 values, computed with an empty math gate and no program cost. They are superseded by Table 9 and Section 6.6. The Python and Rust figures are unchanged in 3.5.0.* The cascade works as follows. The 2B answers. If the executed gate verifies, that answer is final; otherwise the task goes to the 4B and the 4B's answer is final.

This is GwenLaya without Laya. It is compared with always-smallest (B1) and always-largest (B2), not with the registered B5. Holm is taken over these two contrasts.

In 3.4.0 the cascade arm *abstained* when the 4B's gate did not verify. This happened on 757 of 1,536 tasks: all 500 math tasks, 77 Python and 180 Rust. The "cascade accuracy" below is the hidden pass rate of the final candidate, whether or not the arm answered. Its selective metrics are:

- pooled coverage 0.507 (CI 0.490–0.525) with confident-wrong rate 0.103 (CI 0.089–0.117);
- Python 0.854 and 0.206;
- Rust 0.645 and 0.097;
- math coverage 0 (`A5.cascade_selective_*`; superseded: 3.5.0 pooled 0.669, CWR 0.113, math 0.496 and 0.030).
- **No accuracy gain.** Pooled accuracy is 0.604 for the cascade and 0.606 for the 4B, a difference of −0.003 (CI −0.014 to 0.009). The paired exact McNemar test gives 40 tasks correct only under the cascade and 44 only under the 4B, $p = 0.744$ (Holm 0.744). On Python the cascade is slightly worse: −0.026 (CI −0.053 to −0.002). The upper bound is within Monte-Carlo noise of 0 (the re-audit obtained 0.000), so this is borderline. Gate-verified 2B answers that fail the hidden checks are never escalated. On Rust the difference is +0.020 (CI −0.004 to 0.043). Against the 2B alone, the cascade gains 0.180 pooled (295 vs 18 discordant, Holm $p = 2.0 \times 10^{-65}$). That gain comes from the 4B.
- **Not cheaper than always-largest, pooled.** The pooled ratio of cascade to 4B chip-seconds is 1.357 (CI 1.329–1.385) with the first, compilation-bearing chunk counted, as planned. By domain it is 0.997 (CI 0.925–1.071) for Python, 1.221 for Rust and 1.514 for math.

  Excluding the first chunk of every model gives the post-hoc sensitivity S2 (`S2.cost_ratio_cascade_over_large_nowarm.*`): pooled 1.243 (CI 1.224–1.260), Python 0.900 (CI 0.834–0.965), Rust 1.090 and math 1.398. Pooled, the cascade is more expensive either way. For Python it breaks even with warm-up and is about 10% cheaper without it, at a 2.6-point accuracy cost. The cascade escalates 0.675 of tasks pooled: 0.408 on Python, 0.633 on Rust and every math task. It therefore pays for the 2B and then for the 4B on most tasks. The cost per correct answer is 0.821 chip-seconds for the cascade (CI 0.759–0.889) and 0.603 for the 4B (CI 0.563–0.646).
- **Ceiling (A6).** An oracle that picks a correct tier whenever one exists would reach 0.644 pooled (CI 0.620–0.668), against 0.606 for the 4B. By domain it reaches 0.707 / 0.574 / 0.648 (Python / Rust / math), against 0.675 / 0.529 / 0.612. Any router between *these two* tiers can gain at most these differences in accuracy on these tasks. The oracle is a bound only and never a comparator.

Cost caveats. The chip-seconds measure throughput at each model's engine concurrency, not latency. They are not comparable to the registered L4 GPU-seconds. Three points matter here:

1. The 4B ran at an effective concurrency of at most 11, against at most 54 for the 2B (configured 32 and 96). This makes the 4B dearer per token, so the measured cascade cost penalty is *understated* relative to an equal-concurrency setup.
2. The slow first chunk is a larger share of the 2B's total (144 of 297 s) than of the 4B's (192 of 561 s). S2 above removes it.
3. The CPU time of the gate itself is not counted.

Cost is therefore configuration dependent. The ratios above are not robust estimates of any deployment cost, and we draw no cost conclusion beyond "not cheaper pooled, in this configuration". The accuracy result does not depend on cost.

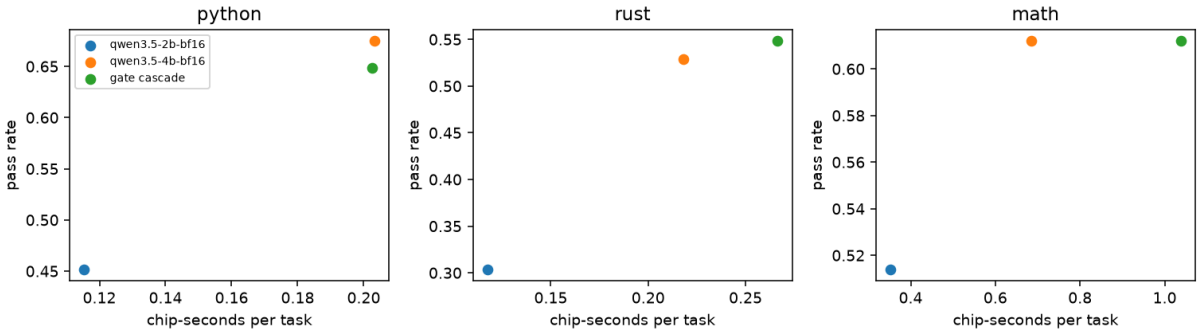


Figure 3: Exploratory, not a registered arm. Pass rate against accelerator chip-seconds per task (TPU v5e, throughput at the engine's concurrency: configured 96 for the 2B and 32 for the 4B, effective at most 54 and 11 per the engine logs; first chunk included; not latency; not comparable to GPU-seconds) for always-2B, always-4B and the gate cascade (2B → executed gate → 4B). The axes do not start at zero. On math the cascade escalates every task, so it costs the 2B plus the 4B for the 4B's accuracy.

6.5 A7: same-protocol CPU vs TPU on the night tasks

This comparison uses the 54 night Python and Rust tasks, which are contained in the E set with identical prompts (D25). It compares the night CPU arm (Qwen3.5-2B q4_K_M, llama.cpp, re-scored v3.3.1 rows) with the TPU 2B (bf16, vLLM). Both use the same raw prompts, prefill and stop sequences (Table 7; `A7_night_cpu_vs_tpu`).

- **Python.** 11 vs 10 passes; 27 of 28 items agree.
- **Rust.** 6 vs 8 passes; 22 of 26 agree; McNemar $p = 0.625$.
- **Pooled.** 17 vs 18 passes; 49 of 54 agree; $p = 1.0$.

Hardware, quantisation and engine all differ, while the prompt text, prefill and stop do not. This is a consistency check with small n, not an isolation of any one factor, and not an equivalence test.

Math is excluded (plan amendment 1, post hoc) because the CPU night math used the defective code prompt (D28). Including it had given a pooled p driven entirely by that defect.

6.6 A10: the program-of-thought math gate (revision 3.5.0; post hoc, exploratory)

What changed. The math-gap statement of 3.4.0 is superseded (Section 2.7). The registered design (registration Section 1) is now implemented in `gwaya/domains/math_gate.py` and specified in addendum A10 of `docs/ANALYSIS_PLAN_E_TPU.md`, which was written before any program was generated and after the E baselines had been seen. The tier that answered writes a short Python program that computes the answer and prints it. The program runs in the same bubblewrap sandbox as the code checks (no network, no host files, time and memory limits). The gate VERIFIES iff the program's last output line is equivalent to the answer's final `\boxed{}` value, with the same equivalence as the scorer. The gold answer is never an input. A missing boxed answer, a missing program, a crash, a timeout, unparseable output or a mismatch all give UNVERIFIED (a wrong program does not refute the answer). One greedy program per tier and math task, at most 1,024 new tokens, same TPU, engine and settings as the answers.

Cost. The program is a second accelerator generation. Every gated arm (gate-only and the cascade) is charged the program's attributed chip-seconds for each tier it consults; ungated arms are not. The program run and this charge are why the math cascade is dearer than in 3.4.0.

Gate-only (Table 8, Figure 4). Numbers are in the table (`A3.*`, `A4.*` of `numbers_e_tpu_mathgate.json`). On the 4B the gate verifies 0.454 of the math answers and 0.943 of those pass the hidden check, so the confident-wrong rate over all math tasks is 0.026. **Corrected in 3.9.0:** the 3.5.0 text set this against 0.388 "when every answer is given", but that baseline counts the 160 tasks with no boxed answer (160 of the 194 wrong rows; mostly truncated) as confident-wrong. Answering whenever a boxed answer exists gives 0.068 at coverage 0.680 (registered scorer; Table 10), so against that baseline the gate removes 0.042 of confident-wrong answers over all tasks at a cost of 0.226 in coverage. At the same coverage, "answer the most confident" gives a higher rate; the difference is −0.084. Pooled over all 1,536 tasks with coverage matched within each domain, the gate's rate is 0.078 against 0.169 for the baseline (−0.090, CI −0.104 to −0.079; `A4.*.domain_matched`). The 2B's math gate verifies fewer answers (0.288) at precision 0.951. Python and Rust are unchanged from 3.4.0.

Reading and caveats.

1. The program is written by the same tier that wrote the answer. A match is agreement between two outputs of one model, not independent ground truth. Precision 0.943 means that a share of verified math answers still fail the hidden check; its point estimate is below the registration's 0.95 flag point and its CI (0.911–0.971) includes it. This is a descriptive analogue, not an H7 result.
2. Coverage 0.454 means that 0.546 of the 4B's math answers stay UNVERIFIED for any of the reasons above. We did not break the cause down.
3. As in A4, the gate runs a program that the log-prob does not see, k is matched on E, the benchmarks are public, and there is a single greedy sample. This is not H1.

Exploratory re-score of the math labels (revision 3.9.0; D62; Table 10). After this section was written, a review of the stored math answers found that the registered scorer (`gwaya/domains/math_check.py` at `78cf88b`) marks some equivalent answers wrong: brace-less `\frac14` and `\sqrt2`, units or a leading `x =` on the gold answer only, base subscripts, spacing inside tuples, intervals and matrices, and mixed numbers. The scorer was changed to normalize these and to stay fail-closed on anything undecidable (two drafts were rejected by independent reviews for fail-open paths; commits `f837c9f`, `2305d90`, `5e14bd8`). `scripts/rescore_math_stored.py` scored every stored (tier, task) with both scorers (the registered one reproduces all 1,500 stored labels) and re-ran each program once. Because the rules were designed after seeing these outputs, **the re-score is exploratory and post hoc; the registered values in this section and in Table 8 remain the primary record**. What it shows: 22, 26 and 25 wrong-to-right flips (2B, 4B, 9B), none right-to-wrong, all notation equivalences; math accuracy 0.558, 0.664 and 0.692 instead of 0.514, 0.612 and 0.642. The math gate's VERIFIED-but-wrong count falls from 7, 13 and 10 to 2, 0 and 0 of 500: on the 4B and 9B every remaining verified answer is correct (rule-of-three 95% upper bounds 0.006 over all tasks and 0.013 among verified), and the 2B keeps two genuinely wrong answers that its program agreed with. So most of the 4B gate's apparent misses below the 0.95 precision flag were scorer false negatives, under a scorer change we made after looking. Against the answer-if-boxed baseline (0.016 and 0.002 under the re-score), the 4B and 9B gates remove only small absolute amounts of confident-wrong answers on E, at a coverage cost of about a third of the boxed answers; E's math is easy for these tiers, so a zero here is not evidence that the gate has no confident-wrong answers in general. The re-score was not propagated into A4 (the math difference −0.084 and the pooled −0.090), A5, A11, A12 or the Laya labels.

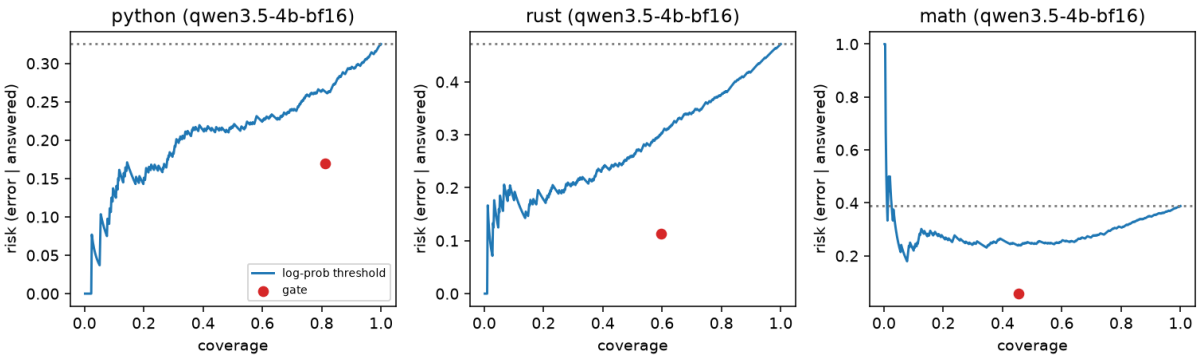


Figure 4: Qwen3.5-4B (bf16, TPU), full E set, exploratory; revision 3.5.0, with the program-of-thought math gate. Blue: risk–coverage curve of a raw log-prob threshold (risk = error rate *among answered tasks*). Red dot: the executed gate, whose risk is one minus its precision. Dotted line: error rate when every task is answered. Math now has a gate point. The left end of each curve rests on very few tasks.

Cascade with the math gate (Table 9, Figure 5). The cascade is the 3.4.0 one (2B, executed gate, then the 4B; the arm abstains if the 4B's gate does not verify), now with the math gate and the program charge. Python and Rust are as in 3.4.0.

- **No accuracy gain over always-4B.** Pooled accuracy is 0.605 for the cascade and 0.606 for the 4B (−0.001, CI −0.013 to 0.012). Exact McNemar gives 45 tasks correct only under the cascade and 46 only under the 4B, $p = 1.0$. Against the 2B alone the cascade gains 0.182 (293 against 13 discordant tasks, Holm $p = 8.2 \times 10^{-70}$; `paired_tests`); that gain comes from the 4B. On math the cascade gains +0.006 (CI −0.004 to 0.016) over the 4B.
- **Dearer than always-4B, with and without warm-up.** Pooled cost ratio 2.021 (CI 1.933–2.111) as planned and 2.023 (CI 1.912–2.134) without the first chunk (S2). On math 2.601 (CI 2.471–2.729) and 2.670 (CI 2.501–2.835): the cascade costs 1.784 chip-seconds per math task against 0.686 for the 4B, by construction of the charge: it generates a 2B answer and a 2B program for every task and escalates 0.712 of them to the 4B, which is then also charged for its own program. The cost per correct answer is 1.219 chip-seconds for the cascade pooled (CI 1.111–1.338) against 0.603 for the 4B (CI 0.563–0.646).
- **Selective view.** With the 4B gate as the last word the cascade answers 0.669 of all tasks pooled (CI 0.646–0.692) with confident-wrong rate 0.113 (CI 0.098–0.128); math 0.496 and 0.030; Python 0.854 and 0.206; Rust 0.645 and 0.097 (`A5.cascade_selective_*`). The "cascade accuracy" above counts the final candidate whether or not it was answered.
- **Ceiling (A6) unchanged.** The oracle over the two tiers reaches 0.644 pooled (CI 0.620–0.668) against 0.606 for the 4B.

The cost caveats of 3.4.0 (different effective concurrency, warm-up, gate CPU time not counted) apply, and the program charge adds one more: it assumes that every tier consulted on a math task also generates a program for the gate.

6.7 A11: cross-fitted calibrated abstention (revision 3.5.0; post hoc, exploratory)

Addendum A11 (written before any model was fitted) replaces a calibrator fitted on the non-existent split C with a fully pre-specified cross-fitted procedure run once on E. It is *not* the registered Laya calibrator and not H4, H5 or H6. Features come from stored outputs only (gate verdict, mean, minimum and lowest-10% token log-probability, log token count, truncation flag,

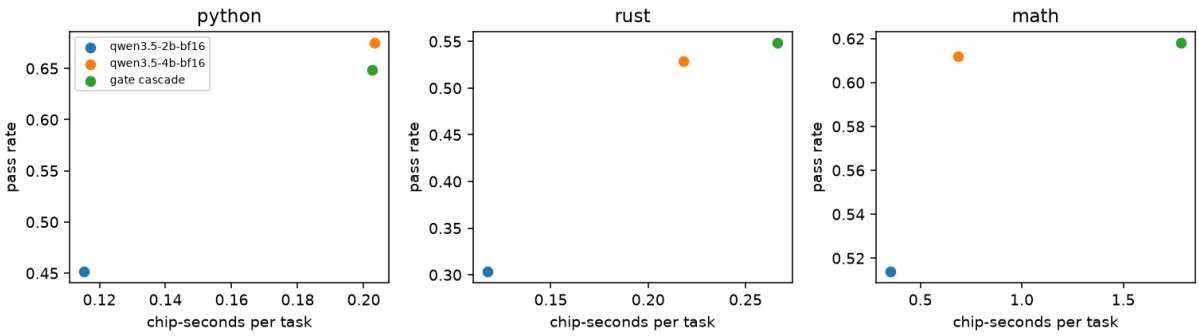


Figure 5: Exploratory, not a registered arm; revision 3.5.0 (with the math gate and the program charge). Pass rate against accelerator chip-seconds per task for always-2B, always-4B and the gate cascade (first chunk included; not latency; not comparable to GPU-seconds; concurrency differs between tiers). The axes do not start at zero and differ between panels.

Table 8: A10, gate-only with the program-of-thought math gate (answer iff the executed gate verifies). 4B: all domains; 2B: math only. CWR = confident-wrong rate over all tasks. Δ = gate CWR minus the per-domain coverage-matched log-prob baseline (A4). 95% cluster bootstrap CIs.

Tier	Domain	coverage	precision	CWR gate	CWR answer-all	Δ vs matched log-prob
4B	Math	0.454 [0.410, 0.498]	0.943 [0.911, 0.971]	0.026 [0.012, 0.040]	0.388 [0.346, 0.432]	−0.084 [−0.110, −0.062]
4B	Pooled	0.624 [0.602, 0.648]	0.875 [0.854, 0.895]	0.078 [0.065, 0.092]	0.394 [0.369, 0.418]	−0.090 [−0.104, −0.079]
4B	Python	0.813 [0.779, 0.847]	0.830 [0.795, 0.864]	0.138 [0.110, 0.168]	0.325 [0.285, 0.365]	−0.076 [−0.098, −0.057]
4B	Rust	0.596 [0.552, 0.639]	0.887 [0.851, 0.922]	0.067 [0.045, 0.091]	0.471 [0.426, 0.515]	−0.112 [−0.134, −0.093]
2B	Math	0.288 [0.250, 0.328]	0.951 [0.913, 0.985]	0.014 [0.004, 0.026]	–	–

domain); the gold answer and the hidden checks are never inputs. Folds are 5, by source problem (deterministic hash). M0 is the gate verdict alone, M1 uses the mean log-prob only, M2 all non-gate features and M3 all features, the last three as logistic regressions evaluated only on out-of-fold predictions (`scripts/analyze_abstention.py`; `abstention_a11.json`). The 4B is the headline tier and the 2B secondary. A11 was not run for the 9B.

Discrimination (Table 11). On the 4B, pooled AUROC is 0.851 (M0), 0.708 (M1) and 0.931 (M3); pooled AURC (lower is better) is 0.256 for M1 and 0.135 for M3. M3 beats M1 on AUROC by 0.223 (CI 0.197–0.249) and M0 by 0.079 (CI 0.064–0.095) (`disc.d_auroc.*.pooled`); per domain the M3−M0 AUROC differences are 0.094 (Python), 0.029 (Rust) and 0.119 (math), all with CIs above zero. On the 2B pooled AUROC is 0.958 for M3 against 0.703 for M1. **M3 uses the gate verdict as a feature, so its advantage over the log-prob score is mostly calibration of the gate information and not independent evidence that a learned score is better than the gate.** Raw confidence was far worse calibrated (ECE 0.252–0.402 per domain on the 4B, Table 4) than the out-of-fold M3 (0.025–0.045).

Operating points (Table 12). With target selective risk $\alpha = 0.05$ the log-prob-only M1 never met the inner-fold rule (coverage 0 in every domain and pooled), while M3 answers 0.275 of tasks pooled (CI 0.254–0.298) at realised risk 0.052 (CI 0.031–0.074). At $\alpha = 0.10$ M3 answers 0.585 pooled at realised risk 0.101 (CI 0.082–0.122); M1 answers 0.033 at risk 0.220 (CI 0.109–0.339). The gate alone answers 0.624 at risk 0.125 (CI 0.105–0.146). The realised risk of M3 at $\alpha = 0.05$ is 0.043 (Python), 0.037 (Rust) and 0.066 (math), with CIs that cover 0.05 in each case. We claim no pass/fail on "meeting the target"; the CIs are wide and the plan says so in advance.

Caveats.

Table 9: A10, gate cascade 2B → gate → 4B with the program-of-thought math gate, versus always-4B. Cost ratio = cascade chip-seconds / always-4B chip-seconds; programs are charged to the cascade for each tier consulted. "no warm-up" drops the first chunk of every model (S2, post hoc). Selective: coverage and CWR of the cascade's final answer when it abstains if the 4B gate does not verify.

Domain	acc. 4B	acc. cascade	Δ acc. (casc.−4B)	cost ratio	cost ratio, no warm-up	escalated
Python	0.675	0.648	−0.026 [−0.053, −0.002]	0.997 [0.925, 1.071]	0.900 [0.834, 0.965]	0.408
Rust	0.529	0.548	+0.020 [−0.004, +0.043]	1.221 [1.153, 1.291]	1.090 [1.036, 1.140]	0.633
Math	0.612	0.618	+0.006 [−0.004, +0.016]	2.601 [2.471, 2.729]	2.670 [2.501, 2.835]	0.712
Pooled	0.606	0.605	−0.001 [−0.013, +0.012]	2.021 [1.933, 2.111]	2.023 [1.912, 2.134]	0.581

Table 10: D62, exploratory post-hoc re-score of the stored E math answers (500 tasks per tier; `rescore_exploratory.json`). Each cell is registered scorer → re-score. The registered values remain the primary record. "Gate V-wrong" = math gate VERIFIED but wrong on the hidden answer (count, and rate over all 500). "Answer all" counts every task with no final `\boxed {}` (mostly generations cut at 1,024 tokens) as confident-wrong; "answer if boxed" answers only when a boxed answer exists (coverage = boxed share) and counts the rest as abstentions. No model output changed; only the scorer did.

Tier	accuracy	flips w→r / r→w	gate verified	gate V-wrong	gate CWR	no boxed	answer-all CWR	boxed: cov., CWR
2B	0.514 → 0.558	22 / 0	144 → 149	7 → 2	0.014 → 0.004	199	0.486 → 0.442	0.602, 0.088 → 0.04
4B	0.612 → 0.664	26 / 0	227 → 230	13 → 0	0.026 → 0.000	160	0.388 → 0.336	0.680, 0.068 → 0.01
9B	0.642 → 0.692	25 / 0	234 → 236	10 → 0	0.020 → 0.000	153	0.358 → 0.308	0.694, 0.052 → 0.00

1. CIs resample source-problem clusters of *fixed* out-of-fold predictions. They do not include variability of the model fit.
2. ECE percentile-bootstrap CIs sometimes exclude their own point estimate. For example, for M2 on 4B Python the point estimate is 0.056 and the CI 0.057–0.111 (`disc.ece.M2.python`); the same happens for M2 on Rust (0.045, CI 0.047–0.101). This is the known upward bias of ECE under resampling, so the CIs of ECE are not read as symmetric intervals around the estimate.
3. It is one run on the burned E set, public benchmarks, one greedy sample, and not the registered GL.
4. (Added in 3.9.0.) The outcome label of every math row is the registered scorer's; the exploratory D62 re-score turns 26 of the 4B's and 22 of the 2B's math labels from wrong to right (Section 6.6). A11 was not re-fitted with those labels, so its math and pooled numbers are on the registered labels only.

6.8 A12: a Qwen3.5-9B tier and wider-gap cascades (revision 3.5.0; post hoc, exploratory)

Setup. Addendum A12 was written before the 9B run completed and with no 9B result seen. It restores the registered ladder's 9B tier (3.5.0: the 27B was not attempted; superseded, Section 7.5). Qwen3.5-9B, bf16, vLLM-TPU, one `v5litepod-4` slice with tensor parallelism 4, greedy, same raw protocol, prompts, stop sequences and log-probabilities as the 2B/4B, answers for all 1,536 E tasks and, for math, a program per task (A10). A 6-task-per-domain smoke test (18 tasks) was not analysed. Chip-seconds are the slice wall time split by token share times the 4 chips in use (D45), so they are in the same unit as the single-chip 2B/4B but the 9B effective concurrency was not measured. The ledger records 587 s (smoke) and 954 s (full) for the slice, $0.7827 and $1.272, about $2.05 in total (list price, Section 11). Cascades are replayed offline from the cached generations with the same ladder rule as the Study's `gwenlaya` arm (first tier

Table 11: A11 (4B), 5-fold cross-fitted out-of-fold scores. M0: gate verdict alone; M1: mean log-prob only; M3: logistic regression on all features (includes the gate verdict, so M3−M1 is a calibration gain, not independent evidence). Point [95% cluster bootstrap CI of fixed out-of-fold predictions]. ECE: 15 equal-mass bins; AURC: lower is better.

Domain	AUROC M0	AUROC M1	AUROC M3	AURC M1	AURC M3	ECE M3
Python	0.788 [0.750, 0.824]	0.681 [0.630, 0.728]	0.881 [0.846, 0.913]	0.211 [0.170, 0.255]	0.113 [0.087, 0.142]	0.045 [0.035, 0.077]
Rust	0.929 [0.906, 0.950]	0.772 [0.730, 0.812]	0.958 [0.939, 0.974]	0.280 [0.232, 0.332]	0.162 [0.131, 0.196]	0.025 [0.020, 0.052]
Math	0.816 [0.785, 0.846]	0.693 [0.642, 0.742]	0.936 [0.907, 0.960]	0.290 [0.233, 0.347]	0.138 [0.101, 0.179]	0.033 [0.025, 0.059]
Pooled	0.851 [0.833, 0.869]	0.708 [0.680, 0.734]	0.931 [0.916, 0.945]	0.256 [0.228, 0.285]	0.135 [0.116, 0.156]	0.021 [0.017, 0.039]

Table 12: A11 (4B), target-risk operating points: realised coverage and realised selective risk P(wrong | answered), pooled over the outer folds; threshold chosen inside each training portion for $\alpha \in \{0.05, 0.10\}$. M1 never met $\alpha = 0.05$ (coverage 0 in every domain, risk undefined). Gate: no threshold. Point [95% CI].

	M3, $\alpha = 0.05$		M3, $\alpha = 0.10$		gate	
Domain	coverage	risk	coverage	risk	coverage	risk
Python	0.174 [0.144, 0.208]	0.043 [0.010, 0.090]	0.622 [0.580, 0.664]	0.112 [0.080, 0.147]	0.813 [0.779, 0.847]	0.170 [0.136, 0.205]
Rust	0.264 [0.227, 0.304]	0.037 [0.008, 0.072]	0.533 [0.489, 0.576]	0.100 [0.066, 0.137]	0.596 [0.552, 0.639]	0.113 [0.078, 0.149]
Math	0.394 [0.352, 0.436]	0.066 [0.033, 0.103]	0.598 [0.554, 0.640]	0.090 [0.059, 0.124]	0.454 [0.410, 0.498]	0.057 [0.029, 0.089]
Pooled	0.275 [0.254, 0.298]	0.052 [0.031, 0.074]	0.585 [0.561, 0.609]	0.101 [0.082, 0.122]	0.624 [0.602, 0.648]	0.125 [0.105, 0.146]

whose executed gate verifies, else the last tier, unanswered); the replay reproduces the Study's own `gwenlaya` rows exactly (1,536 tasks, 0 mismatches in correctness, answered, escalated and cost; `meta.validation`), so no cascade was re-scored.

The 9B alone (Table 13). Pooled pass rate 0.653 (CI 0.630–0.676); by domain 0.726 (Python), 0.588 (Rust), 0.642 (math), against 0.675, 0.529 and 0.612 for the 4B. Math generations cut at the token limit: 0.306 (pooled share over all tasks 0.100; none in Python or Rust). Raw confidence remains badly calibrated (ECE 0.207, 0.344, 0.304 by domain). The executed gate on the 9B gives coverage 0.666 pooled, precision 0.885 and confident-wrong rate 0.077, and at matched coverage the gate's rate is below the log-prob baseline by 0.053 (Python), 0.126 (Rust) and 0.082 (math) (all CIs below zero; `A4.delta_cwr_gate_minus_b3.qwen3.5-9b-bf16.*`). An oracle that picks the cheapest correct tier among 2B, 4B and 9B reaches 0.726 pooled (CI 0.704–0.748), against 0.653 for the 9B: a ceiling for any router over these three tiers on these tasks, not a comparator.

Reading rule, fixed before the results. A cascade *pays off* only if its cost ratio against always-9B is below 1 in both the with-warm-up and the without-warm-up readings and the CI of its accuracy difference does not exclude zero on the harmful side. Anything else is reported as not paying off. This is a descriptive rule, not a test, and there is no multiplicity correction across the twelve domain-by-cascade cells.

Results (Table 14).

- **4B→9B pays off for Python and Rust, not for math and not pooled.** Python: cost ratio 0.601 (CI 0.553–0.656) with warm-up and 0.749 (CI 0.700–0.800) without; accuracy difference +0.013 (CI −0.013 to 0.040); escalated share 0.187. Rust: 0.879 (CI 0.807–0.952) and 0.947 (CI 0.895–0.998); +0.081 (CI 0.053–0.110); escalated 0.404. Math: 1.806 and 2.306, with the program charged for each tier consulted, so it does not pay off even though accuracy is higher by +0.018 (CI 0.008–0.030). Pooled: 1.381 (CI 1.306–1.458) and 1.724 (CI 1.636–1.812), with accuracy higher by +0.037 (CI 0.023–0.051; 86 against 29 discordant tasks, Holm over the three cascades $p = 2.9 \times 10^{-7}$). The pooled cascade is more accurate than always-9B (0.690 against 0.653) but not cheaper.
- **2B→9B pays off for Rust only.** Rust: 0.909 and 0.892, accuracy difference +0.018 (CI

Table 13: A12, Qwen3.5-9B (bf16, v5litepod-4, TP=4) alone. Pass rate, share of generations cut at 1,024 tokens, executed-gate coverage, precision and CWR, and the oracle router over the three tiers (accuracy, and its chip-seconds per task, picking the cheapest correct tier; a ceiling, not a comparator). 95% cluster bootstrap CIs.

Domain	pass rate	truncated	gate coverage	gate precision	gate CWR	oracle acc.	oracle cost
Python	0.726 [0.688, 0.764]	0.000	0.870 [0.841, 0.898]	0.833 [0.798, 0.866]	0.146 [0.115, 0.176]	0.784 [0.749, 0.819]	0.242 [0.213, 0.274]
Rust	0.588 [0.544, 0.631]	0.000	0.649 [0.607, 0.690]	0.906 [0.873, 0.936]	0.061 [0.041, 0.083]	0.708 [0.669, 0.748]	0.329 [0.286, 0.375]
Math	0.642 [0.598, 0.684]	0.306	0.468 [0.424, 0.510]	0.957 [0.930, 0.982]	0.020 [0.008, 0.032]	0.682 [0.640, 0.722]	0.933 [0.807, 1.075]
Pooled	0.653 [0.630, 0.676]	0.100	0.666 [0.644, 0.688]	0.885 [0.865, 0.903]	0.077 [0.064, 0.090]	0.726 [0.704, 0.748]	0.496 [0.450, 0.545]

Table 14: A12, gate cascades versus always-9B, with the reading rule fixed in addendum A12. "Pays off" needs a cost ratio below 1 in both the with-warm-up and no-warm-up readings (point estimates) and a CI of the accuracy difference that does not lie wholly below zero. 9B effective concurrency was not measured, so its cost is less certain. Point [95% CI].

Cascade	Domain	Δ acc. (casc.−9B)	cost ratio	cost ratio, no warm-up	pays off?
2B→9B	Python	−0.042 [−0.068, −0.015]	0.668 [0.598, 0.740]	0.681 [0.624, 0.737]	no
	Rust	+0.018 [−0.006, +0.041]	0.909 [0.841, 0.975]	0.892 [0.843, 0.938]	yes
	Math	+0.000 [−0.008, +0.008]	1.711 [1.620, 1.802]	2.017 [1.905, 2.129]	no
	Pooled	−0.008 [−0.021, +0.004]	1.344 [1.285, 1.403]	1.526 [1.452, 1.602]	no
4B→9B	Python	+0.013 [−0.013, +0.040]	0.601 [0.553, 0.656]	0.749 [0.700, 0.800]	yes
	Rust	+0.081 [+0.053, +0.110]	0.879 [0.807, 0.952]	0.947 [0.895, 0.998]	yes
	Math	+0.018 [+0.008, +0.030]	1.806 [1.680, 1.938]	2.306 [2.173, 2.439]	no
	Pooled	+0.037 [+0.023, +0.051]	1.381 [1.306, 1.458]	1.724 [1.636, 1.812]	no
2B→4B→9B	Python	−0.038 [−0.068, −0.008]	0.563 [0.505, 0.629]	0.648 [0.581, 0.715]	no
	Rust	+0.077 [+0.043, +0.110]	0.916 [0.827, 1.005]	0.952 [0.882, 1.019]	yes
	Math	+0.016 [+0.004, +0.030]	2.110 [1.960, 2.268]	2.639 [2.463, 2.815]	no
	Pooled	+0.018 [+0.002, +0.033]	1.565 [1.470, 1.661]	1.905 [1.787, 2.024]	no

−0.006 to 0.041). Python is cheaper (0.668 and 0.681) but fails the accuracy condition: −0.042 (CI −0.068 to −0.015). Math: 1.711 and 2.017. Pooled: 1.344 and 1.526, accuracy −0.008 (CI −0.021 to 0.004; 40 against 53 discordant tasks, Holm across cascades $p = 0.213$).

- **2B→4B→9B.** Python fails on accuracy (−0.038, CI −0.068 to −0.008) although it is cheaper (0.563 and 0.648). Rust meets the rule by point estimates (0.916 and 0.952, accuracy +0.077) but the CIs of both ratios include 1 (0.827–1.005 and 0.882–1.019), so we do not read this as a robust saving. Math 2.110 and 2.639. Pooled 1.565 and 1.905, accuracy +0.018 (CI 0.002–0.033; 91 against 64 discordant tasks, Holm across cascades $p = 0.073$).

In short, a wider tier gap makes the gate cascade cheaper than always-largest for the code domains (4B→9B for Python and Rust), but math, which carries the program cost of every tier consulted, and the pooled view still cost more than always-9B.

Caveats. The 9B effective concurrency was not measured, so its per-task cost is less certain (we do not know in which direction the cascade-to-9B ratios would move). The 4-chip unit is a slice wall-time split by token share. First-chunk warm-up is treated by the with/without readings only. The programs' CPU time and the code gates' CPU time are not counted. Per-domain rule outcomes have no multiplicity correction. All math accuracies here (9B 0.642, 4B 0.612, cascade accuracies and their differences) use the registered math labels; under the exploratory D62 re-score the 9B's math accuracy is 0.692 and the 4B's 0.664 (Section 6.6), and the cascades were not recomputed with the re-scored labels. E is burned for H1/H3, the benchmarks are public, there is a single greedy sample, and nothing here involves Laya.

Table 15: Deviations D31–D41 of revision 3.4.0 (full text in `docs/DEVIATIONS.md`). "Post hoc" = decided after exploratory results were seen.

#	Registered	Done here	When
D31	L4 GPU, llama.cpp GGUF q4 (D6: llama.cpp)	one TPU v5e chip, vLLM-TPU 0.31.0, bf16; weights at HF revision `main` (not pinned)	before generation
D32	tiers 2B/4B/9B/27B	2B/4B only ("largest" = 4B), plus Qwen2.5-Coder-1.5B as a continuity anchor	before generation
D33	one engine configuration	configured `max_num_seqs` 96 (Coder, 2B) and 32 (4B; 96 failed with "Cannot fit both KV pools"); effective KV capacity at most 54 (2B) and 11 (4B) concurrent requests	during generation
D34	H3 cost in L4 GPU-seconds	accelerator chip-seconds, chunk wall time split by token share, including first-chunk warm-up; throughput, not latency	plan
D35	B3 threshold set on split C	A4 baseline answers the k most confident tasks, k = gate's count in the same domain (no labels used; pessimistic ties); matched on E, not frozen on C; not B3	plan
D36	GL = gate + Laya; H3 vs B5	cascade = gate ladder without Laya, compared with B1/B2; not GL, not H3	plan
D37	math gate = program-of-thought re-execution	not implemented in 3.4.0: our gate executed only `gate_payload`, empty for 500/500 math rows; math gate coverage 0. **Superseded by D43 in 3.5.0**	plan (stated)
D38	–	A7 restricted to Python/Rust (CPU night math defective)	post hoc (amend. 1)
D39	–	serial overlay for contention flips; passing-row rerun; S1 fail-closed sensitivity; nondeterministic item stays FAILED	post hoc (amend. 2, 4)
D40	logprobs captured (D21)	first TPU pass returned no log-probabilities; set aside and fully regenerated with `logprobs=1`	during generation
D41	sequential checks	2B/4B scored with 5 parallel check workers (Coder: one process); contention timeouts corrected by D39	scoring

Table 16: Deviations D43–D47 of revision 3.5.0 (full text in `docs/DEVIATIONS.md`). All were decided after the E baselines had been seen.

#	Registered / before	Done here	When
D43	math gate = program-of-thought re-execution (D37: not implemented)	implemented and run on E math for the 2B and 4B after the zero math coverage was seen (A10); programs charged to every gated arm per tier consulted; math rows of the overlay replaced by the replay	post hoc
D44	tiers 2B/4B/9B/27B (D32: 2B/4B only)	Qwen3.5-9B run on all 1,536 E tasks plus math programs (A12); 27B not attempted; reading rule for "pays off" fixed before results	post hoc; A12 written before the 9B run completed
D45	cost in L4 GPU-seconds (D34: chip-seconds)	9B on a v5litepod-4 slice, TP=4; chip-seconds = slice wall time split by token share × 4; 9B effective concurrency not measured	post hoc
D46	calibrator fitted on split C (no Laya, no C)	A11 cross-fitted logistic abstention on E, run once; M3 uses the gate as a feature; run for 4B and 2B only	post hoc
D47	–	ledger now 11 lines, $4.9136 (list price); three lines added since 3.4.0	recorded

6.9 Deviations recorded for this section

7 Laya trained on tier labels and tested on a fresh set E′ (revision 3.6.0)

This section is the main addition of revision 3.6.0. Its protocol is addenda A13 and A13b of `docs/ANALYSIS_PLAN_E_TPU.md` and deviations D48 to D52 of `docs/DEVIATIONS.md`. A13 was committed (`20219bc`) before any of the new data were generated; A13b (`65ce98f`) was written after the training job was launched and before E′ was generated. All numbers below are read from `results/gwenlaya_v4/laya/laya_eprime.json`, `router_nowarm.json`, `data/*` and `controls_C.json` by `scripts/make_tables_360.py`, which writes the tables of this section. Section 7.6 states the limits.

7.1 What was built

Fresh pool P. Pool P is disjoint from E. It holds 1,500 problems sampled from the MATH train split (7,496 available) and 600 from GSM8K train (7,473 available), and 556 MBPP tasks whose id and prompt are not in MBPP+ (`pool_manifest.json`: 974 raw MBPP tasks, 378 dropped because their task id is in MBPP+, 2 because their prompt hash is; 38 more dropped because their cluster is in E). That gives 2,656 tasks, 2,100 math and 556 Python (Table 17). The seed is 20261009. Source dataset revisions are recorded in the manifest. Four MATH problems with no boxed answer were dropped. Rust has no fresh pool here (MultiPL-E Rust is all in E), so **nothing about Rust is evaluated on E′**; the fresh Rust set R′ of revision 3.7.0 is a separate evaluation-only set (Section 7.7).

Splits. By cluster: sha256(cluster) mod 10 in 0–5 train, 6 calibration split C, 7–9 evaluation set E′ (D48; the reserved 20-bucket rule would have left about 10% of a small pool for training). The training set is P train plus *all* of E (D49), i.e. 3,081 tasks and 9,243 (task, tier) calibrator rows. C has 283 tasks (849 rows) and is used only to fit an isotonic map. E′ has 828 tasks (659 math, 169 Python; 2,484 rows). Tier labels (correct against the hidden check; gate verdict; for math, a program-of-thought run) come from Qwen3.5-2B, 4B and 9B, run with the raw protocol of Section 6. The generation jobs of P and E′ appear in the ledger as `v5litepod-4` lines (names containing `-9b-`) and `v5litepod-1` lines (names containing `-small-`); I infer from the names which tiers ran on which.

Table 17: Data of the Laya study (A13b). Pool P: MATH train, GSM8K train and MBPP tasks not in MBPP+; split by cluster hash mod 10 (D48). Training adds all 1,536 E tasks (D49). Counts from `pool_manifest.json` and `build_counts_*.json`.

Set	math tasks	Python tasks	tasks	calibrator rows (task × tier)
P train	1223	322	1545	–
Train = E + P train	–	–	3081	9243
C (calibration, P)	218	65	283	849
E′ (evaluation, P)	659	169	828	2484

Which model "Laya" is here. The backbone is `answerdotai/ModernBERT-large` with a LoRA adapter of rank 16 on `Wqkv`, `Wo` and `Wi`, maximum length 512 (`head_config.json`, sha256 in Section 12); the HF revision of the backbone is not recorded in that file. There are two heads. The *router* head predicts, from the prompt alone ($n_{\text{features}} = 0$), the probability that each of the three tiers is correct. The *calibrator* head predicts P(correct) for one (prompt, candidate answer, tier) row and also receives 16 numeric features built from the gate verdict and the signals

Table 18: E′ (828 tasks), per tier, one greedy sample. Score = P(correct) of the answer. Laya = calibrator head after the isotonic map fitted on C; log-prob = exp(mean token log-prob); gate = 1 if the executed gate verified, else 0; hand LR = logistic regression on gate, mean and min log-prob, domain and tier, fitted on the same training rows. AURC: lower is better. Acc. = fraction of answers correct; gate cov. = fraction the gate verifies.

Tier	Score	AUROC	AURC	ECE15	Brier	acc.	gate cov.
2B	Laya	0.923	0.159	0.050	0.115	0.562	0.405
	log-prob	0.753	0.295	0.336	0.341		
	gate	0.794	0.354	0.222	0.222		
	hand LR	0.918	0.158	0.051	0.119		
4B	Laya	0.951	0.103	0.028	0.077	0.656	0.583
	log-prob	0.738	0.195	0.275	0.288		
	gate	0.862	0.256	0.147	0.147		
	hand LR	0.935	0.095	0.049	0.093		
9B	Laya	0.945	0.087	0.033	0.080	0.696	0.612
	log-prob	0.736	0.165	0.243	0.260		
	gate	0.866	0.220	0.146	0.146		
	hand LR	0.933	0.078	0.041	0.090		

Table 19: E′: the two registered primaries as tested with deviations D48 (split), D49 (training on E) and D50 (shuffled-label control withdrawn). Paired cluster bootstrap, 10,000 resamples, seed 0; one-sided p in Laya's favour, Holm over {H1, H3}. H1: ΔAURC, Laya minus raw log-prob. H3: Δ selective risk, Laya minus gate, at the gate's own coverage (Laya keeps the same number of answers). Primary tier: 9B.

Tier	H1 ΔAURC [CI]	p_{Holm}, supp.	H3 Δrisk [CI]	p_{Holm}, supp.
9B (primary)	−0.078 [−0.106, −0.053]	0.0002 / yes	−0.012 [−0.023, +0.011]	0.211 / no
4B	−0.092 [−0.123, −0.063]	0.0002 / yes	−0.010 [−0.024, +0.016]	0.335 / no
2B	−0.136 [−0.167, −0.102]	0.0002 / yes	+0.018 [−0.026, +0.039]	0.716 / no

Table 20: E′, exploratory: Laya minus the logistic regression on hand-built features (gate, mean and min log-prob, domain, tier). Paired cluster bootstrap, 95% CI, no multiplicity correction.

Tier	ΔAUROC [CI]	ΔAURC [CI]
9B	+0.013 [+0.001, +0.026]	+0.008 [−0.007, +0.025]
4B	+0.015 [+0.003, +0.029]	+0.008 [−0.009, +0.026]
2B	+0.004 [−0.011, +0.020]	+0.001 [−0.009, +0.011]

Table 21: Controls on split C (AUROC). The registered criterion was 0.4 to 0.6 for the shuffled-label (SH) heads. "Untrained twin" = same architecture and seed, no training. Recomputed from `preds\_C\_*.jsonl` and the labelled C rows by `scripts/make\_tables\_360.py` (cached in `controls\_C.json`). Strata = domain × tier (6).

	calibrator		router (per tier: 2B / 4B / 9B)		
Head	pooled	range over strata	2B	4B	9B
trained Laya	0.95	0.91–0.99	0.78	0.81	0.82
SH (shuffled labels)	0.69	0.55–0.69	0.57	0.64	0.39
untrained twin	0.48	0.24–0.51	0.65	0.48	0.58

Table 22: E′, exploratory router policy (tau = 0.5, chosen on C as the cheapest grid value whose C accuracy is at least that of the static ladder; on C the static ladder had accuracy 0.700). Accuracy and confident-wrong rate (CWR) are over all 828 tasks. For the ladder and the router, accuracy counts the correctness of the final candidate even where no gate verified, and CWR = answered (gate-verified) and wrong; the always-9B policy answers everything, so its CWR is one minus its accuracy. Cost = attributed chip-seconds per task, math programs charged for every tier consulted. Cost ratios: with the first compile chunk of each run, and with that chunk (192 tasks) set to zero (`router\_nowarm.json`).

Policy	accuracy [CI]	CWR [CI]	cost per task [CI]
Router + gate cascade	0.705 [0.674, 0.737]	0.040 [0.027, 0.053]	1.57 [1.47, 1.68]
Static ladder 2B→4B→9B	0.704 [0.673, 0.734]	0.066 [0.051, 0.083]	2.06 [1.92, 2.21]
Always 9B (answer all)	0.696 [0.664, 0.727]	0.304 [0.273, 0.336]	1.99 [1.90, 2.07]
Differences and ratios (router)			
Accuracy, router − always-9B	+0.010 [+0.001, +0.018]		
Accuracy, router − ladder	+0.001 [−0.008, +0.012]		
Cost ratio vs always-9B, with warm-up	0.788 [0.749, 0.829]		
Cost ratio vs always-9B, first chunk zeroed	0.895 [0.866, 0.924]		
Cost ratio vs ladder, with warm-up	0.760 [0.735, 0.787]		
Cost ratio vs ladder, first chunk zeroed	0.712 [0.688, 0.739]		

Table 23: Choice of tau on C (`laya\_eprime.json`, `router.tau\_selection\_on\_C`). Static ladder accuracy on C: 0.700. Chosen: 0.5.

tau	accuracy on C	cost per task on C
0.5	0.703	1.041
0.6	0.693	1.030
0.7	0.693	1.033
0.8	0.693	1.049
0.9	0.696	1.057

`tests_passed_frac`, `selftest_agreement`, `mean_logprob`, `min_logprob` and `repair_rounds` (I did not audit which of these are non-constant in the data). It therefore sees the gate and the log-prob. Training used torch_xla 2.8 and peft on one TPU v5e chip: 3 epochs, learning rate 2×10^{-4}, batch 16, seed 1234, bf16 autocast (A13b). The calibrator run took 1,566 steps (1,170.0 s wall) and the router run 522 steps (448.8 s), with torch 2.8.0+cu128, transformers 5.19.0 and peft 0.21.2 recorded in the run manifests. On the 10% task-grouped validation part of the training data (not C, not E′) the calibrator had AUROC 0.927 (n = 903) and the router 0.828 / 0.807 / 0.807 for the 2B / 4B / 9B (n = 301). The isotonic map has 38 knots and was fitted on C (n = 849). **This is not a model named "Laya" that was pretrained elsewhere: the data-lake "laya" weights were not used.** No generator adapter exists (Section 7.5).

Freeze order. The sha256 of the adapters, heads, head configurations, C predictions and isotonic map are in `docs/LAYA_FREEZE.txt`, committed in `d217f1e` at 2026-10-09 16:39:32 +0200. The two E′ generation VMs of the ledger carry start times 16:39:47 (`tpu_ep-small-163947`) and 16:41:58 (`tpu_ep-9b-164158`) in their names, after that commit; their ledger lines are Table 50. The results commit is `8af8776` (17:37:55).

7.2 Confirmatory family on E′: H1 and H3, with deviations stated

The family was fixed in A13b: Holm over {H1, H3} on the primary tier (Qwen3.5-9B), math and Python pooled; per tier as secondary. **These are tests with three recorded deviations**

that every reading must carry: D48 (the split above, not the registered 20-bucket rule), D49 (E, which was already burned for these tests, is in the training set; E′ shares no cluster, task id or prompt hash with the training data) and D50 (the registered shuffled-label control failed and was withdrawn, Section 7.3). The metrics are those of A13b (AURC for H1, selective risk at the gate's own coverage for H3), not the registered CWR-at-matched-coverage metric and McNemar test of the registration; this is a further difference from the registered form.

- **H1 is supported on E′ (math and Python)** (Table 19): on the 9B the AURC of the calibrated Laya P(correct) is lower than that of the raw mean log-prob by 0.078 (95% CI −0.106 to −0.053; Holm-adjusted $p = 0.0002$, which is the floor of a 10,000-resample bootstrap). The same holds for the 4B (−0.092) and the 2B (−0.136).
- **H3 is not supported on any tier**: the selective risk of Laya at the gate's own coverage differs from the gate's by −0.012 (CI −0.023 to +0.011) on the 9B, −0.010 (CI −0.024 to +0.016) on the 4B and +0.018 (CI −0.026 to +0.039) on the 2B. All CIs include zero. This does not show that Laya and the gate are equal, only that E′ cannot separate them.

The raw log-prob is a weak comparator: its ECE, taking exp(mean log-prob) as the probability, is 0.243 on the 9B (0.275 on the 4B, 0.336 on the 2B), against 0.033, 0.028 and 0.050 for the calibrated Laya (Table 18). The exponentiated mean log-prob is a confidence that is not calibrated by construction, so the ECE comparison says little about ranking; H1 is on AURC, which is rank-based.

Exploratory: Laya against a logistic regression on hand-built features. A logistic regression on the gate verdict, mean and minimum log-prob, domain and tier, fitted on the same training rows, is about as good as Laya (Tables 18, 20). Laya's AUROC is higher by 0.013 on the 9B (CI 0.001 to 0.026) and by 0.015 on the 4B (CI 0.003 to 0.029), and not distinguishable on the 2B (0.004, CI −0.011 to 0.020). These gains are below the +0.02 minimum effect that the registration states for H5 (H5 is not tested here; its LR and its data differ). On AURC, Laya's point estimates are *higher* (worse) than the LR's on all three tiers (+0.008 on the 9B) and every CI includes zero. My reading: most of Laya's advantage over the raw log-prob and over the gate alone comes from combining the gate verdict with the log-prob signals, which the calibrator receives and which the LR also uses; as for the text of the prompt and answer: a text-only calibrator (3.8.0, Section 7.18) is predictive on E′ (AUROC 0.84 to 0.87) and at chance on R′, and these data do not show what the text adds on top of the gate and log-prob inputs (no ablation removed the text from the full model). This reading is consistent with the numbers but is not a tested decomposition.

7.3 The shuffled-label control failed, and what was done about it

A13b registered a control: heads trained identically on shuffled labels (SH) must give AUROC between 0.4 and 0.6 on C, otherwise "the pipeline is declared broken and E′ is not generated". They did not (Table 21). The SH calibrator scored 0.693 pooled (0.553 to 0.692 within the six domain × tier strata) and the SH router 0.572 / 0.637 / 0.389 for the 2B / 4B / 9B. We then scored the untrained, same-seed twin of each head on C: the calibrator twin scored 0.480 pooled but 0.239 to 0.514 within strata, and the router twin 0.653 / 0.482 / 0.575. The Spearman correlation between the SH and twin calibrator scores is 0.136, so the SH value is not an initialisation effect. The trained heads score 0.952 (calibrator) and 0.780 / 0.808 / 0.821 (router).

The criterion was withdrawn (D50) and replaced by a direct leakage check; applied as written, it would have stopped the pipeline and E′ would not have been generated. E′ was generated after D50 was committed. The replacement check found no overlap between E′ and the training data by cluster, task id or normalised prompt hash (0 / 0 / 0), and between C and the training data 0 / 0 / 1 (one prompt-hash match, in C only; D49). The reading we offer for the failed control is that a head fed the strongly predictive gate and log-prob features, or a

router scored pooled over domains with different base rates, ranks by those inputs in an arbitrary direction when it has learned nothing from the labels, so the criterion cannot be met reliably by a correct pipeline. **This reading is not proven.** We did not test it (for example, by repeating the SH run with several seeds or by dropping the numeric features), and the SH result remains an unexplained failure of a registered check. A reader who does not accept the withdrawal should treat the E′ results as obtained without a passed leakage control.

7.4 Exploratory router policy

The router policy starts each task at the cheapest tier whose router P(correct) is at least τ (else at the 9B), then climbs the tiers and answers at the first tier whose gate verifies (A13b). τ was chosen on C from $\{0.5, \ldots, 0.9\}$ as the cheapest value whose C accuracy is at least that of the static ladder (Table 23); only $\tau = 0.5$ met the target on C, and 0.5 is the lowest value in the grid, so lower values were not tried. On E′ (Table 22):

- accuracy is 0.705 against 0.696 for always-9B (difference +0.010, CI 0.001 to 0.018) and 0.704 for the static ladder (+0.001, CI −0.008 to 0.012);
- the confident-wrong rate is 4.0% (CI 2.7–5.3%) against 30.4% for always-9B. That comparison is not like for like: always-9B answers every task, while the router answers only where a gate verifies. The static ladder's rate is 6.6%;
- cost per task is 0.79 of always-9B's with the first compile chunk of each run included (CI 0.75–0.83) and 0.89 (CI 0.866–0.924) with each run's first 192-task chunk set to zero (`router_nowarm.json`). Against the static ladder the ratios are 0.76 and 0.71. Cost is list-price chip-seconds, with the 9B counted at four chips and its concurrency not measured (D45).

This is exploratory: E′ is mostly easy math (GSM8K and easy MATH), so accuracies are higher than on E and the router may find it easy to start low; the cost advantage need not hold on harder tasks.

7.5 Negative and measured results

- **27B.** Qwen3.5-27B did not finish loading in 33 minutes on a `v5litepod-4` (no token generated; the VM line is `tpu_b27-tp4-122313` in the ledger, 41 minutes, $3.284). An 8-chip slice is blocked by the per-zone v5e quota of 4 chips; no quota request was made. The tier set is therefore {2B, 4B, 9B} (A13b, D51). This supersedes the 3.5.0 sentences that the 27B "was not attempted".
- **Generator LoRA.** The probe (Qwen3.5-0.8B, torch_xla and peft) loaded the model, fell back to the pure-PyTorch linear-attention path and finished no training step in 25 minutes. No generator adapter is trained or claimed.
- **A first Laya run was lost (D52).** It trained the calibrator but could not save it (safetensors cannot read XLA-device tensors), and the router found no examples because tier names were not passed. Both were fixed and the run repeated, with the save moved before any prediction. The first run cost about $0.9 (ledger item `tpu_laya-train-132634`, $0.94).
- **Driver bug.** The TPU driver reported quota errors as "no capacity" until it was fixed; the current text of `deploy/tpu/drive.sh` says "capacity or per-zone quota". I did not inspect the earlier driver output, so I cannot say how many launches it affected.

7.6 Limits specific to these results

E′ has only 169 Python tasks, so Python-only statements are low-powered, and its math is mostly GSM8K and easy MATH. E′ contains no Rust, and **nothing about Rust is claimed from E′**

(Rust is tested separately on R′, Section 7.7, where H1 and H3 are not supported). Each tier has one greedy sample. Costs are list-price chip-second estimates, and the 9B figures rest on an unmeasured engine concurrency. E is in the training set (D49). The calibrator receives the gate and the log-prob as inputs, which may be part of why it beats each of them alone (a reading; the 3.8.0 text-only ablation, Section 7.18, does not remove the text from the full model, so it leaves this open). The control failure (Section 7.3) is unresolved. The H1 and H3 metrics are the A13b ones, not the registered CWR/McNemar form. The public benchmarks were probably seen in pretraining. No generator adapter exists.

7.7 Revision 3.7.0: the frozen Laya on a fresh Rust set R′

This subsection is the main addition of revision 3.7.0. The protocol is addendum A14 of `docs/ANALYSIS_PLAN_E_TPU.md`, committed (`0f47b15` 18:25:26, task-file hash `ecdd44e` 18:25:36) before any model generated an answer for R′ and before any Laya score on it was seen; the generation VMs carry start times 18:25:48 in their names (`tpu_rf-small-182548`, `tpu_rf-9b-182548`), and the results commit is `790e224` (18:53:27). Deviations D53 to D58 (Table 51) apply. Numbers are read from `laya_rfresh.json`, `rust_fresh_summary.json`, `rust_fresh_report.json`, `build_counts_Rfresh.json` and `rfresh_gate_laya_overlap.json` by `scripts/make_tables_370.py`.

The set. R′ comes from Exercism's Rust track (MIT, commit `cc099ae0...`) by `scripts/data/build_rust_pool.py`. Each practice exercise becomes a task in the harness's assertion format (instructions and starter stub as the prompt; hidden tests = the test file's helper items plus every `##[test]` body; the gate sees the first test only). An exercise is kept only if its own reference solution is VERIFIED by the real checker with latency at most 4 s of the 10 s budget and the unmodified starter stub is not (D54). Of 106 exercises 69 are kept and 37 dropped (Table 24). The 69 tasks are used for evaluation only: none was used for training, calibration or choosing τ. They produce 207 (task, tier) rows and 69 router rows.

Table 24: Fresh Rust set R′ (A14), from `rust_fresh_report.json` and `build_counts_Rfresh.json`: 69 of 106 Exercism practice exercises kept; 37 dropped. Rows for Laya: 207 (task × tier) and 69 router rows. Drop reasons as written in the report.

Drop reason	exercises
reference_not_verified	16
external_crate_in_tests	12
has_dependencies	4
tests_files≠1	2
no_test_functions	1
reference_too_slow_for_a_stable_timeout	1
should_panic	1
Total dropped	37
Kept (used)	69

Models and what was frozen. Qwen3.5-2B, 4B and 9B, with the same raw protocol as E and E′ (greedy, one sample, log-probs, 1,024 new tokens); no math programs. Laya's adapters and heads, and the isotonic map, are those of `docs/LAYA_FREEZE.txt`; the map recomputed for this run (`isotonic_C_recomputed_rust.json`) is identical to `docs/laya_isotonic_C.json` in its fields `x`, `y` and `n`. Nothing was retrained or re-calibrated. The cluster is the exercise, so the bootstrap is item-level.

Table 25: R′ (69 Rust tasks), per tier, one greedy sample (`rust_fresh_summary.json`). Pass = hidden tests pass. Gate = first test only. Precision = share of gate-verified answers that pass. Confident-wrong (CW) = gate-verified and failing, over all 69 tasks; answer-all CW = one minus accuracy. Trunc. = share of generations that stopped at the 1,024-token limit.

Tier	pass	acc.	gate verified	gate precision	CW gate	CW answer-all	trunc.
2B	6/69	0.087	21	0.286	0.217	0.913	0.087
4B	28/69	0.406	40	0.700	0.174	0.594	0.043
9B	37/69	0.536	45	0.822	0.116	0.464	0.014

Table 26: R′: the frozen Laya against the comparators (`laya\_rfresh.json`). Paired cluster bootstrap (cluster = exercise), 10,000 resamples, seed 0; one-sided p in Laya's favour, Holm over {H1, H3}. H1: ΔAURC, Laya minus raw log-prob. H3: Δ selective risk, Laya minus gate, at the gate's coverage. Last column, exploratory and without multiplicity correction: ΔAUROC, Laya minus the hand-feature logistic regression. AUROC columns are the scores themselves. $n = 69$, so intervals are wide.

Tier	AUROC Laya / log-prob / gate / LR	H1 ΔAURC [CI]	p_{Holm}, supp.	H3 Δrisk [CI]	p_{Holm}, supp.
9B (primary)	0.89 / 0.73 / 0.88 / 0.93	−0.070 [−0.180, +0.041]	0.201 / no	+0.000 [+0.000, +0.000]	1.000 / no
4B	0.82 / 0.66 / 0.85 / 0.89	−0.063 [−0.210, +0.082]	0.407 / no	+0.000 [+0.000, +0.000]	1.000 / no
2B	0.81 / 0.78 / 0.88 / 0.97	+0.054 [−0.049, +0.137]	1.000 / no	+0.000 [+0.000, +0.000]	1.000 / no

Table 27: R′, exploratory router policy with tau = 0.5 (chosen on C in 3.6.0, not re-chosen). Same definitions as Table 22; accuracy and confident-wrong rate (CWR) over 69 tasks. Cost = attributed chip-seconds per task. In `laya\_rfresh.json` these entries sit under the key `router.E\_prime`, a name inherited from the analysis script.

Policy	accuracy [CI]	CWR [CI]	cost per task [CI]
Router + gate cascade	0.536 [0.420, 0.652]	0.116 [0.043, 0.188]	8.03 [6.87, 9.35]
Static ladder 2B→4B→9B	0.420 [0.304, 0.536]	0.377 [0.261, 0.493]	8.14 [6.17, 10.53]
Always 9B (answer all)	0.536 [0.420, 0.652]	0.464 [0.348, 0.580]	8.13 [7.08, 9.37]
Accuracy, router − always-9B	+0.000 [+0.000, +0.000]		
Accuracy, router − ladder	+0.116 [+0.014, +0.232]		
Cost ratio vs always-9B	0.987 [0.946, 1.024]		
Cost ratio vs ladder	0.986 [0.865, 1.169]		

Table 28: A15.1, stored Rust answers re-checked under edition 2021 (`rust_edition_sensitivity.json`); differences are confirmed by re-running both editions on the item. No pass-to-fail flip anywhere. Flip items: rs/exercism/allergies; rs/mbpp_619_move_num; rs/mbpp_90_len_log.

Set	Tier	tasks	pass 2015 / 2021	gate verified 2015 / 2021	hidden fail→pass	hidden pass→fail
R′	2B	69	6 / 6	21 / 21	0	0
R′	4B	69	28 / 29	40 / 41	1	0
R′	9B	69	37 / 37	45 / 45	0	0
E	2B	507	154 / 154	186 / 186	0	0
E	4B	507	268 / 269	302 / 303	1	0
E	9B	507	298 / 299	329 / 330	1	0

Table 29: A15.2, visible-test depth dial on the stored Rust answers of E (`rust_gate_depth.json`; edition 2015; 505 of 507 tasks have at least 3 assertions; the splitter rebuilt the stored k=1 gate payload with 0 mismatches over 507 tasks). The gate runs the first k assertions. Coverage = share of tasks verified; precision = share of verified that pass; CW = verified and failing over all tasks. 95% cluster-bootstrap CIs (10,000, seed 0). Descriptive; not a hypothesis test. In the 4B rows $k = 2$ and $k = 3$ one item (`rs/mbpp\_130\_max\_occurrences`) is non-deterministic under the real checker; see the text.

Tier (accuracy)	k	coverage [CI]	precision [CI]	CW [CI]
4B (0.527)	1	0.594 [0.550, 0.638]	0.887 [0.849, 0.921]	0.067 [0.048, 0.091]
	2	0.570 [0.527, 0.614]	0.924 [0.891, 0.952]	0.044 [0.028, 0.063]
	3	0.550 [0.505, 0.594]	0.957 [0.932, 0.979]	0.024 [0.012, 0.038]
9B (0.586)	1	0.648 [0.606, 0.687]	0.905 [0.872, 0.936]	0.061 [0.042, 0.083]
	2	0.618 [0.576, 0.659]	0.949 [0.923, 0.972]	0.032 [0.018, 0.048]
	3	0.600 [0.558, 0.642]	0.977 [0.959, 0.993]	0.014 [0.004, 0.024]

Descriptive results (Table 25). Pass rates are 6/69 (2B), 28/69 (4B) and 37/69 (9B). The gate verifies 21, 40 and 45 answers, with precision 0.286, 0.700 and 0.822; verified-and-wrong answers are 0.217, 0.174 and 0.116 of all tasks, against 0.913, 0.594 and 0.464 when every answer is given. For the 2B the gate verifies 21 answers and 6 tasks pass; precision 0.286 is 6/21, so all six passes are among the verified ones and the other 15 verified answers fail (confident-wrong $0.217 = 15/69$), so on Rust a gate that executes only the first test is a weak check at that tier. The shares of generations that stopped at the token limit are 0.087, 0.043 and 0.014.

H1 and H3 on R′ (Table 26). The family and its definitions are those of A13b, Rust only, primary tier 9B. **H1 is not supported on any tier.** On the 9B the AURC difference (Laya minus raw log-prob) is −0.070 (95% CI −0.180 to +0.041; Holm $p = 0.20$); on the 4B −0.063 (CI −0.210 to +0.082; Holm $p = 0.41$); on the 2B +0.054 (CI −0.049 to +0.137; Holm $p = 1$). The 9B and 4B point estimates are in Laya's favour, but all intervals include zero. With 69 tasks and one cluster per task the test is underpowered; **this is not evidence that there is no effect**, only that R′ cannot show one. By contrast, on E′ the 9B difference was −0.078 with a CI that excluded zero. **H3 is not supported on any tier, and the difference is exactly** 0 **(CI** $[0, 0]$**) on all three.** The reason is in the numbers: with k equal to the gate's verified count (21, 40, 45), the k answers that Laya scores highest are exactly the gate-verified answers on every tier (`rfresh_gate_laya_overlap.json`). On the calibrated P(correct) (raw head output through the frozen isotonic map, the score that H1 and H3 use) the lowest value inside the gate-verified set is 0.429 (2B), 0.689 (4B) and 0.689 (9B) and the highest outside it is 0 on all three tiers, so there are no ties across the boundary. On the raw head output, before the map, the lowest inside is 0.621, 0.737 and 0.752 and the highest outside is 0.0018, 0.0020 and 0.0020. (An earlier version of this sentence called the raw values P(correct); that was a mislabel.) Laya therefore ranks the gate-verified answers first, and at the gate's coverage its selective risk is the gate's; on R′ it adds nothing to the gate's selection at that coverage. We do not extend this to a statement about what Laya has learned.

Exploratory: Laya against the hand-feature logistic regression. The regression (gate, mean and min log-prob, domain, tier; trained on the same rows as Laya) has the higher AUROC on every tier: Laya minus regression is −0.039 [−0.117, +0.023] on the 9B, −0.067 [−0.144, −0.001] on the 4B and −0.153 [−0.258, −0.070] on the 2B (95% CIs, no multiplicity correction). The 9B interval includes zero; the 4B and 2B intervals do not. On E′ the sign was the opposite (Laya higher by +0.013 on the 9B).

Exploratory: router policy (Table 27). $\tau = 0.5$ was chosen on C in 3.6.0 and is not re-chosen (D56). Router accuracy on R′ is 0.536, equal to always-9B (difference exactly 0), and the confident-wrong rate is 0.116 against 0.464 for always-9B (the policies answer different sets of tasks; this comparison is not like for like, as in 3.6.0). The cost ratio against always-9B is 0.987 (CI 0.946 to 1.024), **so no saving**, against 0.79 on E′; against the static ladder it is 0.986 (CI 0.865 to 1.169). The static ladder has accuracy 0.420 and confident-wrong rate 0.377, and the router's accuracy is higher than the ladder's by 0.116 (CI 0.014 to 0.232).

Contrast with E′ and what is not known. On E′ (math, Python) H1 was supported, Laya was close to the hand-feature regression and the router saved cost; on R′ (Rust) H1 is not supported, Laya is below the regression and the router saves nothing. Laya's advantage did not transfer to this fresh Rust set. We did not test why. Plausible reasons, all hypotheses: (i) the Exercism prompts are much longer and differently styled than Laya's Rust training prompts, and Laya's 512-token input will often truncate the candidate (stated in A14 in advance; the share of truncated inputs was not measured); (ii) Rust in the training set came only from MultiPL-E, a different task style; (iii) $n = 69$ with wide intervals, so part of the difference from E′ may be noise. None of these was tested.

Harness finding (D55). While building R′ we found that the Rust oracle invokes `rustc` without `-edition`, so it compiled as edition 2015. Exercises whose reference needs the 2021 prelude (`FromIterator`) or let-chains therefore failed the reference check and were dropped from R′ (in the report, they are among the `reference\_not\_verified` drops; I did not separate them by cause). The effect on model answers was measured afterwards in A15.1 and is small (Section 7.8).

Limits. 69 tasks from a public repository that may be in the models' pre-training data; a single greedy sample; tasks selected by a reference and stub check using the same harness; list-price costs and the unmeasured 9B concurrency (D45); edition 2015 (above); Laya was not trained on this task style. The field names `meta.n_Eprime_rows` and `router.E_prime` in `laya_rfresh.json` are inherited from the analysis script; they hold the R′ data (D57).

7.8 Improving the gate: edition and test depth (revision 3.7.0)

Both analyses are addendum A15 of `docs/ANALYSIS_PLAN_E_TPU.md`, committed (`f0f514b`, 18:57:22) before any result was seen. The edition run had however already started when A15 was committed (its log on the data disk was created at 18:56:48), and the A15 heading says it was written while the edition run was in progress; the depth script and its log were created after the commit; the edition results are in commit `8d3323f` (19:14:39) and the depth results in `4fe5405` (19:21:28). They are exploratory, CPU-only and reuse the stored generations (nothing was regenerated). They test no hypothesis. Numbers are read from `results/gwenlaya_v4/edition/rust_edition_sensitivity.json` and `rust_gate_depth.json` by `scripts/make_tables_370.py`.

A15.1: Rust edition. `scripts/rust_edition_sensitivity.py` re-checks every stored Rust answer (E: 507 tasks; R′: 69; tiers 2B, 4B, 9B) under edition 2021 for the hidden tests and the gate tests, and confirms each difference by running both editions again on that item, so that a load-dependent timeout is not counted as an edition effect (Table 28). Confirmed differences: E 4B one item (`rs/mbpp_90_len_log`), E 9B one item (`rs/mbpp_619_move_num`), R′ 4B one item (`rs/exercism/allergies`); E 2B, R′ 2B and R′ 9B none. Each is a hidden fail to pass and the same item moves the gate from failing to verified; there is no pass-to-fail flip anywhere. The largest share is 1 of 69 tasks (1.4%) and for E 1 of 507 (0.2%), below the 2 percent threshold of the decision rule fixed in advance, so no Rust result was re-analysed. All published numbers stay labelled

edition 2015, and the oracle default for future runs is 2021 (D58, `DEFAULT\_RUST\_EDITION` in `gwaya/oracles.py`). I did not re-run the re-check.

A15.2: the visible-test depth dial. Everywhere so far the gate sees one visible assertion. For the Rust tasks of E we re-run the gate on the first k assertions, $k = 1, 2, 3$, on the stored 4B and 9B answers, restricted to the 505 of 507 tasks that have at least three assertions, so that the task set is the same for every k (Table 29). The splitter was validated first: rebuilding the $k = 1$ payload reproduces the stored gate payload for all 507 tasks (0 mismatches). Edition 2015 is pinned. Deviation from A15 (D58): the plan said $k = 1, 2, 4$ on Python and Rust of E and R′; what ran is $k = 1, 2, 3$ on Rust of E only. What the numbers show: each added visible assertion lowers gate coverage and raises precision, and the confident-wrong rate over all tasks falls from 0.067 to 0.044 to 0.024 for the 4B and from 0.061 to 0.032 to 0.014 for the 9B, at a coverage loss of 0.044 (4B) and 0.048 (9B) from $k = 1$ to $k = 3$ (differences of the printed points). The first added assertion reduces the 4B rate by about a third and the 9B rate by about a half; I do not claim a fixed halving. The CIs of the confident-wrong rate at $k = 1$ and $k = 3$ do not overlap for either tier, but they are marginal bootstrap intervals, not a paired test. **One flaky item.** In the 4B rows, the item `rs/mbpp_130_max_occurrences` (a wrong answer that the gate verifies at $k = 1$) is non-deterministic under the real checker at $k = 2$ and $k = 3$: in four audit reruns the $k = 3$ gate verified 277 answers three times and 278 once, and the $k = 2$ gate verified 288 three times and 287 once (`docs/CLAIM_AUDIT_370.md`). The committed file, and so Table 29, shows the 278 outcome at $k = 3$ (coverage 0.550, precision 0.957, confident-wrong 0.024) and the 288 outcome at $k = 2$. With the more common 277 outcome the $k = 3$ figures are 0.549, 0.960 and 0.022; these come from the auditor's reruns, not from a committed result file, and I did not edit the JSON. The 9B rows and all $k = 1$ rows reproduce exactly. The 4B dial is therefore accurate to about one task (± 0.002 in coverage and confident-wrong rate, ± 0.003 in precision), which does not change the reading. **Reading.** This is a property of the stored answers, a dial, not a hypothesis test and not a threshold. It is consistent with, and may partly explain, the lower gate precision on R′ (0.700 and 0.822 for the 4B and 9B, with one visible test in the Exercism style) than on E here (0.887 and 0.905 at $k = 1$), but the two sets differ in task style and in the assertions per task, so this is a hypothesis, not tested. Limits: stored answers were produced from the stated prompts, so this is the gate's behaviour on those answers, not what a model would write with more tests visible (corrected in 3.9.0, D63: the 3.7.0 text said these prompts "showed one test"; the gate assertion itself is in no Rust prompt, but in 87 of the 155 HumanEval-Rust prompts (about 17% of E Rust) the gate's first call and its expected value are shown as a doc-comment example (152 of the 155 carry doc-comment examples); none of the 352 MBPP-Rust prompts shows a test); assertions beyond k are the same hidden tests, so $k =$ all is the hidden verdict. Python was not covered in 3.7.0 (it is now, A17, Section 7.15); **R′ was not covered**. **Not done:** exposing the number of visible assertions (`tests\_total`) as a signal to the calibrator. That changes Laya's input features, so it needs retraining and a new freeze; the frozen Laya of 3.6.0 was not changed, and none of the R′ results above uses it.

7.9 Revision 3.10.0: fresh sets F, descriptive replication (A21)

Order of events. The F task files were built and committed in `219178d` (2026-10-10 17:47:33), plan addendum A21 (`docs/ANALYSIS_PLAN_E_TPU.md`) in `d60bdc3` (17:48:53) and the one-command generation launcher in `fc01629` (17:49:58); the generation VMs were created at 18:51:04 (ledger items `tpu_f-9b-185104`, `tpu_f-small-185104`), so no model had generated for F when the plan was committed. Scoring ran locally and was committed in `e83778c` (22:56:38) and `0b21edc` (23:27:45); the frozen Laya was run on F on 2026-10-11 (ledger item `tpu_f-laya-152622`; results `0683fc5`, 15:47:28). The plan fixes: no threshold, feature, prompt, model choice or analysis changed after F results are seen. Deviations D69 to D73 apply (Table 54). Numbers are read from `manifest_F.json`, `numbers_F_pm.json` (written by `scripts/analyze_F.py`; shown by

`scripts/print_F_tables.py`) and `numbers_F_laya.json` by `scripts/make_tables_3100.py`.

What F is. Three tiers (Qwen3.5-2B, 4B, 9B; bf16, vLLM-TPU, greedy, one sample, logprobs, 1,024 new tokens, as E and E′) answered 407 Python tasks, 501 math tasks and 229 Lean 4 statements (Table 30); the math tasks also got a program-of-thought program for the A10 gate; the 27B tier was not run (D76). Python tasks come with a reference solution that the real checker verifies, a trivial stub that fails both the gate test and the hidden suite, and a hidden suite strictly larger than the gate's visible test (the first assertion or test method; hidden suites average 98 assertions for LeetCodeDataset, 3.4 for FullStackBench and 5.6 test methods for BigCodeBench). Candidates were screened with `gwaya.decontaminate` (exact, Python AST, 13-gram, Jaccard 0.8, math normal form, template and number multiset, test input/output, id, Lean statement) against E, pool P, R′, the A18 test statements and the Laya's router and calibrator rows; any hit removes the candidate, except that an n-gram hit is discounted when every shared gram is an alphabet or literal run (164 of 164 Python n-gram hits, 82 of 690 math hits and 3 of 3 Lean hits were discounted; `manifest_F.json`, `ngram\_filter`). The screening removed 202 of 4,500 MATH-test candidates, 12 of 183 competition problems and 34 FullStackBench items, and none of the Lean statements. **"Fresh" binds the Laya, not the generators:** LeetCode, BigCodeBench, MATH, GSM8K and miniF2F predate the Qwen3.5 release and the 2025 and 2026 competitions may also be in its data, so absolute accuracies are upper bounds. The Python prompts (the datasets' own statements and starters) are not the HumanEval and MBPP prompts of E; the gate's first test appears in the prompt for 89 of 407 tasks (22%), not for 304, and could not be decided for 14 (a whitespace- and quote-insensitive substring heuristic, stored per task as `gate_visible_in_prompt`).

Table 30: Fresh sets F (addendum A21; `manifest_F.json`, key `reports`, `counts`). Each count is a number of tasks. "Seeded cap" = the builder samples a fixed number from the survivors (MATH-test stratified by subject, GSM8K, FullStackBench, BigCodeBench; LeetCodeDataset and competition are not capped). GSM8K has no separate candidate count in the manifest; its first column repeats the post-sanity count. The last column is the share of tasks whose gate test appears in the prompt (yes / no / undecidable, `gate_visible_in_prompt`).

source	candidates	after sanity	after decontam.	final	gate test in prompt
python / LeetCodeDataset (test, 2024-08 to 2025-03)	228	205	205	205	44 / 161 / 0
python / FullStackBench (en)	359	146	112	112	27 / 79 / 6
python / BigCodeBench (v0.1.4, stdlib + numpy)	364	275	275	90	18 / 64 / 8
math / MATH test minus MATH-500	4,500	4,500	4,298	200	—
math / GSM8K test	1,319	1,319	1,319	130	—
math / competition (AIME 2025, 2026, HMMT Feb 2025, 2026, Nov 2025, BRUMO 2025)	183	183	171	171	—
lean4 / miniF2F-valid (Lean 4, Mathlib v4.7.0)	244	229	229	229	—
total (python 407, math 501, Lean 229)				1,137	

Table 31: F, accuracy against the hidden suite or gold answer (greedy, one sample, 1,024 new tokens), and the share of generations cut at the token limit (`numbers_F_pm.json`, key `per\_slice`; 95% cluster-bootstrap CIs, 10,000 resamples, seed 0, cluster = source problem). Descriptive; the models may have seen these public benchmarks.

		accuracy [CI]			cut at 1,024 tokens		
slice	n	2B	4B	9B	2B	4B	9B
Python + math	908	0.313 [0.283, 0.341]	0.427 [0.395, 0.459]	0.472 [0.441, 0.504]	0.420	0.318	0.337
python	407	0.189 [0.152, 0.229]	0.329 [0.283, 0.376]	0.396 [0.349, 0.445]	0.297	0.138	0.209
python/LeetCode	205	0.122 [0.079, 0.168]	0.176 [0.125, 0.228]	0.273 [0.212, 0.335]	0.507	0.273	0.400
python/FSB	112	0.250 [0.171, 0.333]	0.446 [0.356, 0.542]	0.554 [0.460, 0.648]	0.116	0.000	0.018
python/BCB	90	0.267 [0.176, 0.360]	0.533 [0.429, 0.639]	0.478 [0.373, 0.582]	0.044	0.000	0.011
Python, test in prompt	89	0.169 [0.096, 0.250]	0.337 [0.240, 0.438]	0.393 [0.293, 0.495]	0.382	0.124	0.258
Python, test not in prompt	304	0.197 [0.153, 0.243]	0.322 [0.271, 0.375]	0.398 [0.343, 0.455]	0.286	0.148	0.204
Python, undecidable	14	0.143 [0.000, 0.364]	0.429 [0.167, 0.700]	0.357 [0.100, 0.632]	0.000	0.000	0.000
math	501	0.413 [0.371, 0.457]	0.507 [0.463, 0.551]	0.535 [0.493, 0.579]	0.519	0.465	0.441
math/MATH-test	200	0.515 [0.444, 0.585]	0.655 [0.589, 0.720]	0.695 [0.630, 0.758]	0.425	0.315	0.270
math/GSM8K	130	0.785 [0.713, 0.852]	0.915 [0.864, 0.961]	0.954 [0.915, 0.985]	0.054	0.031	0.015
math/competition	171	0.012 [0.000, 0.030]	0.023 [0.005, 0.048]	0.029 [0.006, 0.057]	0.982	0.971	0.965

slice	n	2B	4B	9B	2B	4B	9B
math w/o competition	330	0.621 [0.568, 0.674]	0.758 [0.711, 0.803]	0.797 [0.753, 0.839]	0.279	0.203	0.170

Table 32: F, the executed gate (Python: first visible assertion or test method; math: program-of-thought gate) against the hidden result (`numbers_F_pm.json`). Coverage = share VERIFIED; precision = P(correct | VERIFIED); CWR = VERIFIED and wrong, over all tasks of the slice; "log-prob at same coverage" = answering the k most confident tasks by mean log-prob, k = the gate's verified count in the same slice and tier (A4 definition). On the Python slices the hidden suite extends the gate test, so every correct answer is VERIFIED (recall 1.000) and precision is accuracy over coverage. Intervals are marginal 95% cluster-bootstrap CIs, not a paired test.

slice	tier	coverage	precision [CI]	CWR [CI]	log-prob at same coverage, CWR [CI]
Python + math	2B	0.315	0.724 [0.674, 0.771]	0.087 [0.070, 0.105]	0.176 [0.159, 0.195]
	4B	0.453	0.854 [0.820, 0.886]	0.066 [0.051, 0.081]	0.149 [0.130, 0.166]
	9B	0.486	0.859 [0.827, 0.890]	0.068 [0.053, 0.084]	0.131 [0.113, 0.150]
python	2B	0.359	0.527 [0.448, 0.607]	0.170 [0.135, 0.206]	0.258 [0.226, 0.292]
	4B	0.464	0.709 [0.642, 0.772]	0.135 [0.103, 0.170]	0.216 [0.182, 0.243]
	9B	0.543	0.729 [0.670, 0.786]	0.147 [0.113, 0.182]	0.224 [0.192, 0.256]
python/LeetCode	2B	0.268	0.455 [0.323, 0.588]	0.146 [0.099, 0.196]	0.185 [0.149, 0.231]
	4B	0.298	0.590 [0.467, 0.716]	0.122 [0.079, 0.168]	0.156 [0.117, 0.197]
	9B	0.410	0.667 [0.564, 0.767]	0.137 [0.091, 0.185]	0.176 [0.133, 0.227]
python/FSB	2B	0.411	0.609 [0.463, 0.750]	0.161 [0.095, 0.234]	0.304 [0.237, 0.358]
	4B	0.536	0.833 [0.733, 0.924]	0.089 [0.039, 0.146]	0.205 [0.155, 0.267]
	9B	0.634	0.873 [0.790, 0.946]	0.080 [0.034, 0.134]	0.223 [0.172, 0.287]
python/BCB	2B	0.500	0.533 [0.385, 0.680]	0.233 [0.149, 0.323]	0.333 [0.263, 0.420]
	4B	0.756	0.706 [0.593, 0.812]	0.222 [0.139, 0.313]	0.311 [0.231, 0.384]
	9B	0.733	0.652 [0.530, 0.765]	0.256 [0.167, 0.351]	0.333 [0.253, 0.416]
Python, test in prompt	2B	0.236	0.714 [0.500, 0.900]	0.067 [0.022, 0.125]	0.202 [0.138, 0.263]
	4B	0.461	0.732 [0.588, 0.863]	0.124 [0.060, 0.195]	0.191 [0.118, 0.264]
	9B	0.528	0.745 [0.612, 0.864]	0.135 [0.070, 0.211]	0.202 [0.143, 0.282]
Python, test not in prompt	2B	0.382	0.517 [0.426, 0.607]	0.184 [0.142, 0.227]	0.273 [0.231, 0.307]
	4B	0.451	0.715 [0.639, 0.787]	0.128 [0.092, 0.166]	0.214 [0.179, 0.247]
	9B	0.533	0.747 [0.679, 0.814]	0.135 [0.097, 0.173]	0.214 [0.177, 0.248]
Python, undecidable	2B	0.643	0.222 [0.000, 0.545]	0.500 [0.231, 0.778]	0.500 [0.273, 0.778]
	4B	0.786	0.545 [0.222, 0.846]	0.357 [0.111, 0.636]	0.357 [0.143, 0.636]
	9B	0.857	0.417 [0.125, 0.714]	0.500 [0.231, 0.778]	0.571 [0.308, 0.800]
math	2B	0.279	0.929 [0.883, 0.968]	0.020 [0.008, 0.034]	0.080 [0.058, 0.098]
	4B	0.443	0.977 [0.956, 0.995]	0.010 [0.002, 0.020]	0.094 [0.072, 0.114]
	9B	0.439	0.991 [0.977, 1.000]	0.004 [0.000, 0.010]	0.078 [0.058, 0.098]
math/MATH-test	2B	0.300	0.983 [0.944, 1.000]	0.005 [0.000, 0.016]	0.060 [0.033, 0.092]
	4B	0.540	0.981 [0.952, 1.000]	0.010 [0.000, 0.026]	0.125 [0.080, 0.156]
	9B	0.545	0.991 [0.970, 1.000]	0.005 [0.000, 0.016]	0.085 [0.049, 0.115]
math/GSM8K	2B	0.615	0.887 [0.813, 0.951]	0.069 [0.030, 0.116]	0.100 [0.050, 0.155]
	4B	0.869	0.973 [0.939, 1.000]	0.023 [0.000, 0.053]	0.038 [0.008, 0.078]
	9B	0.831	0.991 [0.969, 1.000]	0.008 [0.000, 0.025]	0.015 [0.000, 0.039]
math/competition	2B	0.000	n/a	0.000 [0.000, 0.000]	0.000 [0.000, 0.000]
	4B	0.006	1.000 [1.000, 1.000]	0.000 [0.000, 0.000]	0.006 [0.000, 0.019]
	9B	0.018	1.000 [1.000, 1.000]	0.000 [0.000, 0.000]	0.012 [0.000, 0.030]
math w/o competition	2B	0.424	0.929 [0.883, 0.968]	0.030 [0.013, 0.050]	0.094 [0.066, 0.123]
	4B	0.670	0.977 [0.955, 0.995]	0.015 [0.003, 0.030]	0.097 [0.068, 0.123]
	9B	0.658	0.991 [0.976, 1.000]	0.006 [0.000, 0.015]	0.058 [0.037, 0.083]

Accuracy (Table 31). Pooled over Python and math (908 tasks) accuracy is 0.313 [0.283, 0.341], 0.427 [0.395, 0.459] and 0.472 [0.441, 0.504] for the 2B, 4B and 9B. By domain: Python 0.189, 0.329, 0.396; math 0.413, 0.507, 0.535. The strata differ much more than the tiers: GSM8K 0.785, 0.915, 0.954; MATH-test 0.515, 0.655, 0.695; **the competition stratum 0.012, 0.023, 0.029 with 98.2, 97.1 and 96.5% of its generations cut at 1,024 tokens, so it is a floor and says nothing about competition ability**. LeetCode, the hardest Python source, has

Table 33: F, the static gate cascade 2B → 4B → 9B (escalate when the gate does not verify) against always-9B (`numbers_F_pm.json`, key `cascade`). Cost is accelerator chip-seconds (9B × 4 chips) and includes the program-of-thought generation of every tier consulted on math. "Cost ratio" divides by always-9B without its program, "with program" divides by always-9B charged with its own program (the like-for-like reading on math). "warm" includes the first generation chunk, "no warm-up" drops the tasks of chunk 0 (a different, domain-unbalanced subset, D70: only the cost ratio is read from it). Accuracy counts the final candidate even where the gate did not verify.

slice	reading	accuracy cascade / 9B	Δ accuracy [CI]	cost ratio [CI]	cost ratio, with program [CI]	escalated / CWR
Python + math	warm	0.466 / 0.472	−0.007 [−0.026, +0.012]	1.737 [1.649, 1.832]	1.247 [1.196, 1.297]	0.685 / 0.131
Python + math	no warm-up	0.342 / 0.365	−0.023 [−0.047, +0.000]	2.187 [2.085, 2.286]	1.618 [1.564, 1.668]	0.739 / 0.128
python	warm	0.383 / 0.396	−0.012 [−0.052, +0.025]	0.947 [0.869, 1.028]	0.947 [0.869, 1.028]	0.641 / 0.260
python	no warm-up	0.242 / 0.270	−0.028 [−0.077, +0.022]	1.440 [1.356, 1.520]	1.440 [1.356, 1.520]	0.740 / 0.265
math	warm	0.533 / 0.535	−0.002 [−0.018, +0.014]	2.955 [2.804, 3.105]	1.478 [1.417, 1.536]	0.721 / 0.026
math	no warm-up	0.411 / 0.430	−0.019 [−0.040, +0.000]	2.664 [2.529, 2.796]	1.690 [1.625, 1.750]	0.738 / 0.032
math w/o competition	warm	0.794 / 0.797	−0.003 [−0.027, +0.021]	2.606 [2.341, 2.871]	1.157 [1.073, 1.236]	0.576 / 0.039
math w/o competition	no warm-up	0.884 / 0.928	−0.043 [−0.090, +0.000]	1.134 [0.963, 1.316]	0.844 [0.724, 0.968]	0.413 / 0.072
math/competition	warm	0.029 / 0.029	+0.000 [+0.000, +0.000]	3.383 [3.317, 3.451]	2.006 [1.981, 2.031]	1.000 / 0.000
math/competition	no warm-up	0.029 / 0.029	+0.000 [+0.000, +0.000]	3.383 [3.317, 3.451]	2.006 [1.981, 2.031]	1.000 / 0.000
Python, test not in prompt	warm	0.382 / 0.398	−0.016 [−0.059, +0.026]	0.939 [0.850, 1.033]	0.939 [0.850, 1.033]	0.618 / 0.260
Python, test not in prompt	no warm-up	0.245 / 0.288	−0.043 [−0.101, +0.013]	1.402 [1.299, 1.499]	1.402 [1.299, 1.499]	0.699 / 0.282
Python, test in prompt	warm	0.393 / 0.393	+0.000 [−0.082, +0.084]	1.074 [0.912, 1.228]	1.074 [0.912, 1.228]	0.764 / 0.213
Python, test in prompt	no warm-up	0.235 / 0.196	+0.039 [−0.052, +0.135]	1.568 [1.424, 1.689]	1.568 [1.424, 1.689]	0.882 / 0.196

50.7, 27.3 and 40.0% of generations cut at the limit, so its accuracies (0.122, 0.176, 0.273) are also partly an output-length artefact. The tier ordering 2B < 4B < 9B holds in every stratum except BigCodeBench, where the 4B (0.533) is above the 9B (0.478) with overlapping intervals, and the 14 tasks of undecidable visibility.

The executed gate against log-prob at the same coverage (Table 32). As on E, answering only when the gate verifies gives fewer confident-wrong answers than answering the same number of most-confident tasks: Python 0.170, 0.135 and 0.147 against 0.258, 0.216 and 0.224 (2B, 4B, 9B); math 0.020, 0.010 and 0.004 against 0.080, 0.094 and 0.078. The intervals are marginal and not paired, and the log-prob baseline is the A4 construction (k matched to the gate's count in the same slice), so this is a descriptive replication of A4 and A10, not a test. Three things the table also says. (i) The Python one-test gate is a weaker check on F than on E: precision is 0.527, 0.709 and 0.729 (E, 4B, A3: 0.830). BigCodeBench has the highest confident-wrong rate on every tier (0.233, 0.222, 0.256, with precision 0.533, 0.706, 0.652); LeetCode has the lowest precision for the 2B and 4B (0.455 and 0.590, against 0.533 and 0.706 on BigCodeBench) and BigCodeBench for the 9B (0.652 against 0.667). (ii) The confident-wrong rate is the same whether or not the gate test appears in the prompt on the 4B and 9B (0.124 against 0.128 and 0.135 against 0.135); the 2B is 0.067 [0.022, 0.125] against 0.184 [0.142, 0.227] on 89 and 304 tasks, marginal intervals that do not overlap. That is the opposite direction to the one a leak of the gate's test into the prompt would predict (visible tests should make wrong answers pass more easily), the two groups come from different sources and difficulty mixes, and the 4B and 9B show no difference; we read it as a difference between task groups, not as an effect of visibility, and no such effect is claimed. (iii) The math gate's confident-wrong rate is 0.004 [0.000, 0.010] on the 9B and its precision 0.991 [0.977, 1.000]; the F math tasks are mostly GSM8K and MATH, which are easy for these tiers, and the competition stratum is almost never verified (coverage 0.000, 0.006, 0.018), so this is not evidence that the gate has no confident-wrong answers in general.

The cascade (Table 33). The static ladder 2B → 4B → 9B with the gate again does not pay off against always-9B. Pooled, its accuracy is 0.466 against 0.472 (−0.007 [−0.026, +0.012]; McNemar over 908 tasks, 37 tasks only the cascade answers correctly and 43 only the 9B, two-sided $p = 0.58$) at 1.737 [1.649, 1.832] of the 9B's chip-seconds, or 1.247 [1.196, 1.297] when the 9B is charged with its own program; total chip-seconds are 2,349.8 (cascade), 1,352.5 (always-9B) and 1,884.4 (always-9B with program). On Python alone the cost ratio is 0.947 [0.869, 1.028] with the first generation chunk included and 1.440 without it. Applying the A12 reading rule as written (cheaper than always-9B in both readings and no harmful accuracy interval), no slice

of Table 33 qualifies; the nearest is math without the competition stratum, 0.844 [0.724, 0.968] with the program under the no-warm-up reading but 1.157 [1.073, 1.236] with it. The no-warm-up subset is not the A12 subset (D70). The competition stratum alone costs 3.383 (2.006 with program) of the 9B at equal accuracy (0.029), because every tier consulted generates a program that is never verified.

Limits. One greedy sample; a 1,024-token limit that truncates a third of the pooled generations (0.420, 0.318, 0.337) and almost all competition answers; hidden suites only as good as the datasets' own tests (checked against the references, not independently re-derived); a sandbox that lacks some third-party packages, so tasks whose reference needs them were dropped (149 references did not verify), which favours self-contained problems; the passing Python rows were not re-run for stability as they were for E′ (D69); CMIMC-2025 and SMT-2025 (93 problems) were left out for budget; the prompt-visibility labels are a heuristic. No hypothesis is tested in this subsection.

7.10 Revision 3.10.0: the frozen Laya on F (H1 and H3, deviated)

Procedure. The adapters, heads and isotonic map are those of `docs/LAYA_FREEZE.txt`; their sha256 values were checked against the freeze file locally and on the TPU VM before the prediction (D73), the row files against `SHA256SUMS_F_rows.txt`, and the row builder checked per row that no label field changes the rendered text (0 label changes, 0 prompt differences, 0 unexpected fields for both the 908 Python and math rows and the 229 Lean rows; `manifest_F_laya_rows.json`). 2,724 calibrator rows (908 tasks × 3 tiers) and 908 router rows were predicted on one v5e chip (444 s, $0.148). The analysis is that of A13b through `scripts/analyze_F_laya.py`: primary tier 9B, Python and math pooled (908 tasks), H1 = AURC(Laya) − AURC(raw mean log-prob), H3 = selective risk of Laya's top-k answers − the gate's, k = the gate's verified count, paired cluster bootstrap (10,000 resamples, seed 0, cluster = source problem, within domain), Holm over {H1, H3}; "supported" needs both an upper CI bound below zero and a Holm p below 0.05 (D71). H1 and H3 are the A13b metrics, not the registered cost-per-correct and matched-coverage definitions; all of D48 to D50 and the failed shuffled-label control (D50) carry over. Nothing was changed after F results were seen. **One inconsistency, found in the audit:** the Laya row files carry the gate verdict as scored before the serial re-check (D69), so three of the 2,724 (tier, task) rows have a stale verdict (2B `py/BigCodeBench/843` FAILED, 4B and 9B `ma/MATHtest/algebra/829` UNVERIFIED; the re-check made all three VERIFIED). The Laya's inputs, labels and the log-prob are unaffected, but the gate coverage and the top-k of H3 in Table 34 use the stale verdicts (9B 440 verified, coverage 0.485; the descriptive Table 32 has 441, 0.486). Recomputed with the re-checked verdicts the 9B and 4B H3 points are unchanged (−0.002, +0.007) and the 2B point is −0.004 instead of 0.000; no verdict changes and the Laya was not re-run.

Table 36: The frozen Laya on F by stratum (`numbers_F_laya.json`, key `secondary`); descriptive, no multiplicity correction, same estimators as Table 34. H3 is exactly 0 with a degenerate interval $[0, 0]$ in every Python slice; on R′ the same happened because Laya's top-k was the gate-verified set (Section 7.7); that was not checked for F and the cells are not interpreted. "n/a" = no interval could be formed (the competition stratum has too few correct answers).

slice, tier	n	accuracy	H1 ΔAURC [CI]	H3 Δrisk [CI]	ΔAUROC vs LR [CI]
python, 2B	407	0.189	−0.150 [−0.207, −0.094]	+0.000 [+0.000, +0.000]	−0.018 [−0.044, +0.008]
python, 4B	407	0.329	−0.056 [−0.113, +0.001]	+0.000 [+0.000, +0.000]	−0.040 [−0.068, −0.014]
python, 9B	407	0.396	−0.036 [−0.092, +0.019]	+0.000 [+0.000, +0.000]	−0.033 [−0.063, −0.004]
math, 2B	501	0.413	−0.104 [−0.137, −0.073]	+0.021 [−0.030, +0.072]	+0.035 [+0.012, +0.059]
math, 4B	501	0.507	−0.085 [−0.110, −0.061]	+0.009 [+0.000, +0.039]	+0.007 [−0.001, +0.017]
math, 9B	501	0.535	−0.088 [−0.116, −0.062]	+0.009 [−0.009, +0.023]	+0.011 [+0.002, +0.022]
python/LeetCode, 2B	205	0.122	−0.063 [−0.128, −0.000]	+0.000 [+0.000, +0.000]	−0.031 [−0.067, +0.003]
python/LeetCode, 4B	205	0.176	+0.028 [−0.039, +0.088]	+0.000 [+0.000, +0.000]	−0.057 [−0.092, −0.026]
python/LeetCode, 9B	205	0.273	+0.053 [−0.012, +0.111]	+0.000 [+0.000, +0.000]	−0.048 [−0.083, −0.017]
python/FSB, 2B	112	0.250	−0.245 [−0.340, −0.144]	+0.000 [+0.000, +0.000]	+0.002 [−0.044, +0.047]

slice, tier	n	accuracy	H1 [CI]	H3 [CI]	vs LR [CI]
python/FSB, 4B	112	0.446	−0.105 [−0.210, +0.001]	+0.000 [+0.000, +0.000]	−0.029 [−0.076, +0.012]
python/FSB, 9B	112	0.554	−0.112 [−0.209, −0.022]	+0.000 [+0.000, +0.000]	−0.013 [−0.056, +0.026]
python/BCB, 2B	90	0.267	−0.151 [−0.283, −0.019]	+0.000 [+0.000, +0.000]	+0.016 [−0.044, +0.078]
python/BCB, 4B	90	0.533	−0.110 [−0.202, −0.009]	+0.000 [+0.000, +0.000]	+0.013 [−0.067, +0.094]
python/BCB, 9B	90	0.478	−0.067 [−0.178, +0.057]	+0.000 [+0.000, +0.000]	−0.016 [−0.101, +0.067]
Python, test in prompt, 2B	89	0.169	−0.237 [−0.329, −0.129]	+0.000 [+0.000, +0.000]	−0.026 [−0.066, +0.008]
Python, test in prompt, 4B	89	0.337	+0.019 [−0.098, +0.135]	+0.000 [+0.000, +0.000]	−0.052 [−0.119, +0.013]
Python, test in prompt, 9B	89	0.393	−0.046 [−0.144, +0.060]	+0.000 [+0.000, +0.000]	−0.014 [−0.069, +0.038]
Python, test not in prompt, 2B	304	0.197	−0.127 [−0.193, −0.060]	+0.000 [+0.000, +0.000]	−0.016 [−0.048, +0.016]
Python, test not in prompt, 4B	304	0.322	−0.072 [−0.136, −0.009]	+0.000 [+0.000, +0.000]	−0.039 [−0.069, −0.012]
Python, test not in prompt, 9B	304	0.398	−0.031 [−0.095, +0.032]	+0.000 [+0.000, +0.000]	−0.046 [−0.080, −0.014]
math/MATH-test, 2B	200	0.515	−0.100 [−0.153, −0.053]	−0.017 [−0.053, +0.000]	+0.024 [−0.017, +0.067]
math/MATH-test, 4B	200	0.655	−0.101 [−0.144, −0.063]	−0.009 [−0.028, +0.009]	+0.019 [−0.001, +0.042]
math/MATH-test, 9B	200	0.695	−0.105 [−0.159, −0.059]	−0.009 [−0.030, +0.000]	+0.020 [−0.002, +0.045]
math/GSM8K, 2B	130	0.785	−0.072 [−0.148, −0.002]	+0.012 [−0.025, +0.065]	+0.049 [−0.030, +0.128]
math/GSM8K, 4B	130	0.915	+0.005 [−0.008, +0.023]	+0.009 [+0.000, +0.028]	−0.016 [−0.095, +0.072]
math/GSM8K, 9B	130	0.954	−0.013 [−0.041, +0.001]	+0.000 [+0.000, +0.000]	+0.024 [−0.030, +0.109]
math/competition, 2B	171	0.012	+0.016 [+0.000, +0.047]	n/a	−0.314 [−0.380, −0.247]
math/competition, 4B	171	0.023	−0.015 [−0.049, +0.018]	+0.000 [+0.000, +0.000]	+0.014 [−0.116, +0.148]
math/competition, 9B	171	0.029	−0.029 [−0.070, +0.005]	+0.000 [+0.000, +0.000]	+0.020 [−0.135, +0.200]

Result (Table 34). H1 is supported on F: on the 9B the AURC difference against the raw mean log-prob is −0.104 [−0.132, −0.078], one-sided bootstrap $p = 0.0001$, Holm $p = 0.0002$; on the 4B −0.132 [−0.162, −0.102] and on the 2B −0.230 [−0.261, −0.199] (no multiplicity claim off the primary tier). **H3 is not supported:** −0.002 [−0.022, +0.016] on the 9B (Holm $p = 0.43$), +0.007 [−0.010, +0.029] on the 4B and 0.000 [−0.045, +0.035] on the 2B; at the gate's coverage (0.485 on the 9B) Laya's selection has the gate's risk. The 9B's AUROC is 0.955 for Laya against 0.812 for the raw log-prob, 0.876 for the gate and 0.960 for the hand-feature regression; AURC 0.207, 0.311, 0.414 and 0.196. On E′ (3.6.0) the same two numbers were −0.078 [−0.106, −0.053] and −0.012 [−0.023, +0.011].

Exploratory: Laya against the hand-feature logistic regression. Pooled on the 9B, Laya minus the regression is −0.004 [−0.015, +0.006] in AUROC and +0.010 [+0.003, +0.019] in AURC (the regression has the lower AURC, with an interval excluding zero); on the 4B −0.011 [−0.021, −0.002] and +0.014 [+0.005, +0.023]; on the 2B +0.010 [−0.002, +0.023] and −0.003 [−0.017, +0.011]. On E′ Laya was above the regression by +0.013 [+0.001, +0.026] AUROC on the 9B; that advantage did not replicate. By domain (Table 36) the picture differs: on math Laya is at or above the regression (+0.011 [+0.002, +0.022] on the 9B, +0.035 [+0.012, +0.059] on the 2B), on Python it is below it on every tier (−0.018, −0.040 and −0.033; the 4B and 9B intervals exclude zero). H1 itself is carried by math on the 9B (−0.088 [−0.116, −0.062]); within Python it is −0.036 [−0.092, +0.019] on the 9B and −0.056 [−0.113, +0.001] on the 4B, with intervals that include zero. The pooled H1 may therefore partly reflect domain composition (the raw log-prob baseline has no domain input and the two domains have different base rates; a reading, not tested). Together with the F results for the hand-feature regression, this supports the reading of 3.6.0 and 3.8.0 that most of the gain over the raw log-prob and the gate comes from combining the numeric inputs; it does not settle what the text adds.

Exploratory: router policy (Table 35). With $\tau = 0.5$ (chosen on C, not re-chosen) the router's accuracy is 0.476 [0.443, 0.509] against 0.472 [0.441, 0.504] for always-9B (difference +0.003 [−0.009, +0.015]); its cost is 0.889 [0.864, 0.915] of always-9B's (charged with its program) and 0.713 [0.691, 0.737] of the static ladder's, and its confident-wrong rate is 0.079 against 0.131 for the ladder (not like for like with always-9B's 0.528, which counts every wrong answer as answered). On E′ the cost ratio was 0.79 with the first chunk included and 0.895 without it; the F value, 0.889, is the first-chunk-included reading. The F reading without the first chunk was not computed (no such analysis is in `numbers_F_laya.json`). So the saving on F is about 11% of the 9B's cost at equal accuracy, against about 21% on E′.

What this does and does not show. (i) The out-of-sample result for H1 (a deviated test, as on E′) now holds on 908 fresh Python and math tasks pooled, with harder strata than

Table 34: The frozen Laya v1 on F, Python and math pooled (908 tasks per tier; `numbers_F_laya.json`, key `per\_tier`). AUROC and AURC of correctness scores: Laya = calibrator P(correct) through the frozen isotonic map of C; log-prob = exp(mean log-prob); gate = binary verdict; LR = logistic regression on gate, mean and min log-prob, domain and tier, trained on the Laya's training rows (exploratory). H1 = AURC(Laya) − AURC(log-prob); H3 = selective risk of Laya's top-k − the gate's, k = verified count. Lower AURC and lower risk are better. Paired cluster bootstrap, 95% percentile CI. Holm p and the verdict apply to the 9B only (primary tier).

quantity	2B	4B	9B
accuracy	0.313	0.427	0.472
gate coverage (verdicts as in the Laya rows, before the serial re-check)	0.314	0.452	0.485
AUROC Laya	0.934	0.955	0.955
AUROC log-prob	0.672	0.791	0.812
AUROC gate	0.802	0.893	0.876
AUROC hand-feature LR	0.924	0.967	0.960
AURC Laya	0.384	0.246	0.207
AURC log-prob	0.614	0.378	0.311
AURC gate	0.615	0.434	0.414
AURC hand-feature LR	0.387	0.232	0.196
H1: ΔAURC Laya − log-prob [CI]	−0.230 [−0.261, −0.199]	−0.132 [−0.162, −0.102]	−0.104 [−0.132, −0.078]
H3: Δ selective risk Laya − gate [CI]	+0.000 [−0.045, +0.035]	+0.007 [−0.010, +0.029]	−0.002 [−0.022, +0.016]
exploratory: ΔAUROC Laya − LR [CI]	+0.010 [−0.002, +0.023]	−0.011 [−0.021, −0.002]	−0.004 [−0.015, +0.006]
exploratory: ΔAURC Laya − LR [CI]	−0.003 [−0.017, +0.011]	+0.014 [+0.005, +0.023]	+0.010 [+0.003, +0.019]
9B Holm p (H1, H3)	0.0002, 0.4284; H1 supported = true, H3 supported = false		

Table 35: F, exploratory router policy at the already chosen $\tau = 0.5$ (not re-chosen; `numbers_F_laya.json`, key `router\_policy`; 908 tasks, Python and math, first generation chunk included). Cost is chip-seconds per task, always-9B charged with its program. "Confident-wrong" = answered and wrong, over all tasks: for always-9B every answer counts as answered, so its value is 1−accuracy and the three policies are not answering the same tasks (not like for like).

	router ($\tau = 0.5$)	static ladder	always-9B
accuracy [CI]	0.476 [0.443, 0.509]	0.466 [0.433, 0.498]	0.472 [0.441, 0.504]
confident-wrong [CI]	0.079 [0.063, 0.096]	0.131 [0.110, 0.152]	0.528 [0.496, 0.559]
cost per task [CI]	1.845 [1.743, 1.949]	2.588 [2.433, 2.743]	2.075 [1.983, 2.167]
cost ratio router / ladder, router / always-9B [CI]	0.713 [0.691, 0.737]; 0.889 [0.864, 0.915]		
accuracy difference router − ladder, router − always-9B [CI]	+0.010 [−0.007, +0.026]; +0.003 [−0.009, +0.015]		

E′ (within Python alone the 9B interval includes zero, so the pooled result is carried by math). (ii) H3 is again not supported; the gate-vs-Laya comparison at the gate's coverage is a tie. (iii) The Laya's calibrated score is not distinguishable from the hand-feature regression pooled, and is worse on Python, so "Laya beats a logistic regression on the same features" is not supported on F. (iv) The router saves some cost on F at the accuracy of always-9B, less than on E′. (v) Nothing here concerns Rust (R′ stands: not supported) or Lean (the frozen Laya never saw Lean and no Laya score on Lean was computed; the Lean rows were built but not predicted, D73). (vi) The F tasks are public and may be in the generators' pre-training data, and the Laya never saw them; the replication is about the Laya, not about generator ability. (vii) One seed and one training run of the Laya: seed-to-seed variability of the Laya was never measured. Table 36 lists every stratum for completeness; no multiplicity correction applies to those cells.

7.11 Revision 3.10.0: Lean 4 results (A18 and A21)

Provenance. A18 (plan `5b53ce8`; amendment `f318f71`, which required scoring only with the hardened gate) generated greedy proofs for the miniF2F test statements that elaborate under the pinned Mathlib (v4.7.0): 223 of 244, 17 commented out in the dataset and 4 not elaborating. Generation ran on 2026-10-10 07:36 to 07:59 (ledger items `tpu_lean-9b-073256`, `tpu_lean-small-073256`), with one tier more than planned (the 0.8B, D66). The $k = 8$ sampling

failed on the engine (per-request seeds are rejected by vLLM-TPU, D66) and its stored sample files are empty, so **no any-of-8 result exists**. The greedy proofs were scored three times with the hardened gate (D67, D68): a first pass at the gate's default 120 s per Lean stage, in which 4, 10, 22 and 18 checks (0.8B, 2B, 4B, 9B) timed out under a machine load of 10 to 17; a second pass at 300 s that re-ran only the timeouts (timeouts 4, 10, 19, 16); a third that re-ran only the VERIFIED rows to record whether each matched the reference statement exactly or only definitionally (after a fix of the scorer's reading of the match type, `046ac08`). The third pass is the reported one; the earlier summaries are in `results/gwenlaya_v4/lean_a18/history/`. The A21 validation statements of F were scored once (greedy, `-timeout-s 300 -workers 3`, D72) on the same machine while the Python and math scoring ran, so their timeouts are reported rather than re-run. All numbers are read from `summary_a18.json` files; the scored rows are committed beside them.

Table 37: A18 on the miniF2F test split (223 statements that elaborate under the pinned Mathlib; greedy, one proof per task, 1,024 new tokens; `scored\_greedy\_v3/summary\_a18.json`, scored with the hardened gate at 300 s per Lean stage). "exact / defeq" = how VERIFIED proofs matched the reference statement; other columns count unverified rows by reason ("truncated" = generation hit the token limit and takes precedence over the Lean reason, in brackets how many of those also timed out; "stmt / import" = statement not preserved / unknown import; "forbidden" = rejected by a lexical rule, and in every such row the rule is `sorry` (`lean\_forbidden\_breakdown\_3100.json`); USD at the list price). The $k = 8$ sampling produced no rows. 0.8B is an extra tier (D66).

tier	proved	proof rate [CI]	exact / defeq	lean_error	forbidden	truncated (timed out)	timeout	stmt / import	USD
0.8B	22 of 223	0.099 [0.063, 0.139]	22 / 0	41	89	66 (2)	4	1 / 0	0.0534
2B	41 of 223	0.184 [0.135, 0.238]	41 / 0	64	77	31 (1)	10	0 / 0	0.0537
4B	59 of 223	0.265 [0.206, 0.323]	59 / 0	108	0	37 (4)	19	0 / 0	0.0795
9B	69 of 223	0.309 [0.251, 0.368]	69 / 0	95	1	38 (8)	16	3 / 1	0.2246

Greedy cascade 0.8B → 2B → 4B → 9B (escalate when not proved): 81 theorems proved at $0.3690 against 69 proved by always-9B at $0.2246 (cost ratio 1.643). Overlap of proved sets: 0.8B∩2B 16; 0.8B∩4B 18; 0.8B∩9B 19; 2B∩4B 35; 2B∩9B 35; 4B∩9B 50.

Table 38: A21 on miniF2F-valid (229 statements; same prompt, gate and settings; `eval\_sets\_F/lean\_scored/summary\_a18.json`). Columns as in Table 37.

tier	proved	proof rate [CI]	exact / defeq	lean_error	forbidden	truncated (timed out)	timeout	stmt / import	USD
2B	33 of 229	0.144 [0.100, 0.192]	33 / 0	90	75	24 (0)	4	3 / 0	0.0263
4B	65 of 229	0.284 [0.227, 0.345]	65 / 0	108	1	40 (3)	15	0 / 0	0.0441
9B	76 of 229	0.332 [0.271, 0.393]	76 / 0	100	2	37 (5)	11	2 / 1	0.0789

Greedy cascade 2B → 4B → 9B (escalate when not proved): 95 theorems proved at $0.1386 against 76 proved by always-9B at $0.0789 (cost ratio 1.758). Overlap of proved sets: 2B∩4B 25; 2B∩9B 27; 4B∩9B 50.

Results (Tables 37 and 38). On the 223 miniF2F test statements the greedy proof rate is 0.099 [0.063, 0.139] (0.8B), 0.184 [0.135, 0.238] (2B), 0.265 [0.206, 0.323] (4B) and 0.309 [0.251, 0.368] (9B); on the 229 validation statements of F 0.144 [0.100, 0.192] (2B), 0.284 [0.227, 0.345] (4B) and 0.332 [0.271, 0.393] (9B). **Every VERIFIED proof, in both sets and for every tier, matched the reference statement exactly (A18: 22, 41, 59, 69; F: 33, 65, 76); no proof was accepted only by definitional equality.** Proof rates rise with tier; 9B and 4B intervals overlap in both sets. The proofs of the smaller tiers are largely contained in the larger ones' (A18: 50 of the 4B's 59 theorems are among the 9B's 69; F: 50 of 65 and 76; overlaps in Tables 37 and 38), but not wholly: the greedy cascade escalating on every failure proves 81 theorems (A18) and 95 (F) against 69 and 76 for always-9B, at 1.643 and 1.758 of its list-price cost.

Why proofs were not verified. The `forbidden` column is the lexical rule of the hardened gate, and in every such row the construct is `sorry`: the 0.8B and 2B wrote an unfinished proof

in 89 and 77 of 223 (A18) and in 76 of 229 (F; 75 in the table, one of them is a cut generation) cases, the larger tiers in at most 2 rows (`lean_forbidden_breakdown_3100.json`; the rule fires, as designed, on text the model produced, so these are not false rejections of valid proofs). The largest bucket for the 4B and 9B is a Lean error (108 and 95 of 223); 37 and 38 generations were cut at the 1,024-token limit ("truncated"; all VERIFIED rows ended with a stop). Timeouts at 300 s per stage were 4, 10, 19 and 16 rows outside the cut generations and 2, 1, 4 and 8 inside them (A18), and 4, 15 and 11 plus 0, 3 and 5 (F); such a row is UNVERIFIED, not FAILED, and a timeout is a property of this machine and its load, not of the model, so the proof rates are lower bounds on what the gate would verify with more time. A few rows changed the theorem statement (`statement_not_preserved`: 1 for the A18 0.8B, 3 for the A18 9B, 3 for the F 2B, 2 for the F 9B) or imported an unknown module (1 for the A18 9B, 1 for the F 9B); these counts are after the `truncated` bucket takes precedence (by raw reason the F 9B has 3 and 3, of which 1 and 2 are cut generations).

What this does not show. (i) The gate's limits stand: binder annotations are not compared, a definitional match counts as VERIFIED (none occurred), the external red-team review was not completed (D68), and the lexical rules reject some harmless constructs; a hand check of 10 genuine proofs gave 9 VERIFIED and 1 timeout (D68). We have no second, independent checker for the VERIFIED proofs, so the false-accept rate of the gate is not measured. (ii) miniF2F has been public since 2021 and may be in the models' training data: proof rates are upper bounds on capability, not clean estimates. (iii) The two sets are different statement sets and are not paired; "valid" against "test" differences between 0.309 and 0.332 for the 9B are within the intervals. (iv) The Laya was not applied to Lean. (v) The chip-second attribution of Lean costs splits chunk wall time by generated tokens and is not comparable with the prompt-plus-generated attribution of Python and math (D72).

7.12 Revision 3.10.0: which visible Rust tests should the gate see (A15.3)

Order of events. Addendum A15.3 was committed before its results (`f4af1e4`, 2026-10-10 07:35:49), the selection code in `e556339` (07:38:01) and the results in `b2920c8` (08:20:49). It states one count seen before the plan: on R′ (67 of 69 tasks with at least two test blocks) the first test block gave 14, 12 and 8 confident-wrong answers (2B, 4B, 9B) and the first and the last block 8, 4 and 3. It is exploratory, CPU-only, on stored answers; the Rust compile uses edition 2021 (D58), so the stored edition-2015 verdicts are recomputed. It also runs the R′ arm of A15.2 that D58 reported as never run. Numbers are read from `results/gwenlaya_v4/edition/rust_gate_selection.json` by `scripts/make_tables_3100.py`.

Rules. At a fixed number k of visible units (an assertion statement on E, a `\#[test]` block on R′) the gate sees the `first` k units in file order (the current gate), `first_last` (the first $k-1$ and the last), `longest` (the k longest by text) or `spread` (evenly spread, containing the first and last for $k \geq 2$); $k = 1, 2, 3$ on the 505 of 507 E tasks (and 68 of 69 R′ tasks) that have at least 3 units. The logical sanity rule (an answer that passes the hidden suite must pass every subset) produced 0 violations on either set.

Table 39: A15.3, Rust gate with k visible test units chosen by four rules, on the stored answers (`rust_gate_selection.json`, edition 2021 for every compile). Cell = confident-wrong count (VERIFIED and hidden-wrong) / VERIFIED answers (rate of the confident-wrong count over the tasks of the set). E = MultiPL-E Rust, 505 of 507 tasks (units = assertion statements); R′ = Exercism, 68 of 69 tasks (units = `\#[test]` blocks). `first` is the registered gate order; `first\_last` at $k = 1$ is the last unit alone and `spread` at $k = 1$ the middle one.

set	tier	k	first	first_last	longest	spread
E	2B	1	32 / 185 (0.063)	47 / 200 (0.093)	36 / 189 (0.071)	42 / 195 (0.083)
		2	11 / 164 (0.022)	16 / 169 (0.032)	17 / 170 (0.034)	16 / 169 (0.032)

set	tier	k	first	first_last	longest	spread
		3	6 / 159 (0.012)	5 / 158 (0.010)	8 / 161 (0.016)	7 / 160 (0.014)
E	4B	1	34 / 301 (0.067)	43 / 310 (0.085)	36 / 303 (0.071)	45 / 312 (0.089)
		2	21 / 288 (0.042)	15 / 282 (0.030)	21 / 288 (0.042)	15 / 282 (0.030)
		3	11 / 278 (0.022)	7 / 274 (0.014)	9 / 276 (0.018)	8 / 275 (0.016)
E	9B	1	31 / 328 (0.061)	40 / 337 (0.079)	33 / 330 (0.065)	34 / 331 (0.067)
		2	16 / 313 (0.032)	14 / 311 (0.028)	18 / 315 (0.036)	14 / 311 (0.028)
		3	7 / 304 (0.014)	7 / 304 (0.014)	9 / 306 (0.018)	7 / 304 (0.014)
R′	2B	1	15 / 20 (0.221)	11 / 16 (0.162)	7 / 12 (0.103)	8 / 13 (0.118)
		2	10 / 15 (0.147)	8 / 13 (0.118)	4 / 9 (0.059)	8 / 13 (0.118)
		3	6 / 11 (0.088)	4 / 9 (0.059)	4 / 9 (0.059)	4 / 9 (0.059)
R′	4B	1	12 / 40 (0.176)	6 / 34 (0.088)	6 / 34 (0.088)	10 / 38 (0.147)
		2	8 / 36 (0.118)	4 / 32 (0.059)	6 / 34 (0.088)	4 / 32 (0.059)
		3	7 / 35 (0.103)	3 / 31 (0.044)	6 / 34 (0.088)	2 / 30 (0.029)
R′	9B	1	8 / 44 (0.118)	5 / 41 (0.074)	3 / 39 (0.044)	3 / 39 (0.044)
		2	8 / 44 (0.118)	3 / 39 (0.044)	3 / 39 (0.044)	3 / 39 (0.044)
		3	7 / 43 (0.103)	3 / 39 (0.044)	3 / 39 (0.044)	2 / 38 (0.029)

Table 40: A15.3, paired difference in confident-wrong rate, rule − `first` at the same k (95% paired cluster bootstrap, 10,000 resamples, seed 0; `rust_gate_selection.json`, key `confident\_wrong\_minus\_first`). Negative = fewer confident-wrong answers than `first`. Descriptive; no multiplicity correction.

set	tier	k	first_last − first	longest − first	spread − first
E	2B	1	+0.030 [+0.004, +0.057]	+0.008 [−0.016, +0.032]	+0.020 [−0.008, +0.050]
		2	+0.010 [−0.006, +0.026]	+0.012 [−0.006, +0.030]	+0.010 [−0.006, +0.026]
		3	−0.002 [−0.010, +0.004]	+0.004 [−0.008, +0.016]	+0.002 [−0.008, +0.012]
E	4B	1	+0.018 [−0.008, +0.046]	+0.004 [−0.018, +0.026]	+0.022 [−0.002, +0.048]
		2	−0.012 [−0.032, +0.006]	+0.000 [−0.018, +0.018]	−0.012 [−0.032, +0.006]
		3	−0.008 [−0.020, +0.002]	−0.004 [−0.014, +0.006]	−0.006 [−0.018, +0.006]
E	9B	1	+0.018 [−0.008, +0.044]	+0.004 [−0.016, +0.022]	+0.006 [−0.018, +0.030]
		2	−0.004 [−0.020, +0.012]	+0.004 [−0.010, +0.020]	−0.004 [−0.020, +0.012]
		3	+0.000 [−0.006, +0.006]	+0.004 [−0.006, +0.016]	+0.000 [−0.008, +0.008]
R′	2B	1	−0.059 [−0.147, +0.029]	−0.118 [−0.206, −0.044]	−0.103 [−0.191, −0.029]
		2	−0.029 [−0.118, +0.059]	−0.088 [−0.162, −0.015]	−0.029 [−0.118, +0.059]
		3	−0.029 [−0.088, +0.029]	−0.029 [−0.088, +0.029]	−0.029 [−0.103, +0.044]
R′	4B	1	−0.088 [−0.176, +0.000]	−0.088 [−0.176, +0.000]	−0.029 [−0.103, +0.044]
		2	−0.059 [−0.132, +0.000]	−0.029 [−0.118, +0.044]	−0.059 [−0.132, +0.000]
		3	−0.059 [−0.132, +0.015]	−0.015 [−0.088, +0.059]	−0.074 [−0.147, +0.000]
R′	9B	1	−0.044 [−0.118, +0.029]	−0.074 [−0.147, −0.015]	−0.074 [−0.132, −0.015]
		2	−0.074 [−0.147, −0.015]	−0.074 [−0.147, −0.015]	−0.074 [−0.147, −0.015]
		3	−0.059 [−0.118, −0.015]	−0.059 [−0.118, −0.015]	−0.074 [−0.132, −0.015]

E (Tables 39 and 40). The `first` rule reproduces the dial of A15.2 within a task (4B confident-wrong 0.067, 0.042, 0.022 at $k = 1, 2, 3$ here against 0.067, 0.044, 0.024 in 3.7.0, which used edition 2015; 9B 0.061, 0.032, 0.014 against the same three; which item differs was not traced). The four rules differ by little: at $k = 2, 3$ no paired difference from `first` has an interval excluding zero on any tier, with points between −0.012 and +0.012; at $k = 1$ `first_last`, the last unit alone, is worse than the first on the 2B (+0.030 [+0.004, +0.057]). The rule the plan selects on E (lowest confident-wrong rate averaged over the three tiers and $k = 2, 3$, ties broken by coverage) is `first_last` with 0.0211, against 0.0238 for `first`, 0.0271 for `longest` and 0.0221 for `spread` (`chosen\_on\_E`); a difference of 0.0026 with no interval, so **choosing a rule on E is not supported by the data; the dial in k (more tests) matters far more than the rule**.

R′ (68 tasks). The picture is different and the intervals are wide. With `first` at $k = 1$ the confident-wrong counts are 15, 12 and 8 (rates 0.221, 0.176, 0.118; the plan's 14, 12, 8 was counted

on 67 tasks with at least two test blocks, here 68 tasks with at least three and edition 2021). The other rules give fewer: `first_last` 11, 6, 5; `longest` 7, 6, 3; `spread` 8, 10, 3 (2B, 4B, 9B). Paired differences from `first` exclude zero for `longest` and `spread` on the 2B (-0.118 [-0.206, -0.044] and -0.103 [-0.191, -0.029] at $k = 1$; `longest` also at $k = 2$, -0.088 [-0.162, -0.015]) and for all three rules on the 9B at $k = 1$ to 3 except `first_last` at $k = 1$; on the 4B the upper bounds of several intervals are exactly 0 (the interval touches zero). Because the R′ first-versus-first-and-last count was known before the plan, **these R′ differences confirm nothing**; they and the E result are consistent with the Exercism test files placing the degenerate case first, a hypothesis that was not tested. The 4B R′ rows also carry one task (`rs/exercism/allergies`) that edition 2021 changes (A15.1). Limits: stored answers, one greedy sample, 505 and 68 tasks, 4B and 9B Rust answers produced from prompts that mostly show no test (D63), assertions beyond k are hidden tests (precision partly measures overlap with the ground truth), and the frozen Laya and the verdict policy were not touched.

7.13 Revision 3.10.0: runtime integration, quantisation shift and replay (A19, A20; system changes)

These are code changes and two small descriptive measurements, written up from `docs/LAYA_RUNTIME.md`; the paper does not claim the integrated system works.

- **Frozen Laya as runtime router and scorer** (`gwaya/laya_runtime.py`, `165fd37`, `d22c193`; opt-in through `GWENLAYA_LAYA_DIR`). The frozen ModernBERT-large router and calibrator are loaded with every file's sha256 checked against `docs/LAYA_FREEZE.txt` (fp32, CPU, LoRA merged); the A13b policy is `route_policy="start_at"` (cheapest tier with router $p \geq \tau = 0.5$, then cascade up on the gate); the calibrator's raw score goes through the registered isotonic map; VERIFIED always comes from the gate; Laya is on for Python and math, off for Rust, and refused for Lean. Failure modes fail closed: a candidate cut by the 512-token window (`truncated_candidate`) or missing log-prob signals vetoes LIKELY_CORRECT. Parity against the stored TPU predictions on 20 router and 20 calibrator rows: maximum absolute difference 0.0038 and 0.0029 (tolerance 0.02; `parity.json`); 9 of the 20 calibrator rows were truncated candidates.
- **The registered program-of-thought math gate without a gold answer** (`gwaya/math_pot_runtime.py`, `4f9d2d2`). For a math task with no gold answer the tier's boxed answer is compared with the value printed by a program the same tier wrote (A10); a mismatch, crash, timeout or missing boxed answer is UNVERIFIED, never FAILED. The program is generated lazily (none when there is no boxed answer), a deliberate difference from the study's cost model; verdicts are unchanged. A consistency check of the wiring on 40 stored (tier, task) items returns the stored verdict in 40 of 40 (`math_pot_runtime/consistency.json`). Its measured precision is the A10 and D62 numbers on E (easy for these tiers), not a measurement of the wiring.
- **A llama.cpp tier with log-probs** (`gwaya/llama_tier.py`, `26bb89d`; `scripts/gwenlaya_serve.sh`). It serves Q4_K_M GGUFs through `llama-server` and returns mean and minimum token log-probs (Ollama 0.1.44 returns none, so the calibrator could never support LIKELY_CORRECT with the default ladder). Failures raise. With the backend and Laya both on, every scorer evidence is labelled `quant_shift: true`; this is a label, not a correction. A live smoke test of 3 tasks (`live_smoke.json`) shows the wiring runs (a Python task with tests VERIFIED, a Python task without tests LIKELY_WRONG at $p = 0.43$ under exploratory thresholds, a math task LIKELY_CORRECT at $p = 0.97$; 12 to 97 s per task on a shared 8-core machine); it is a plumbing test, not an evaluation.

A19, the Laya's shift between bf16 and Q4_K_M answers (Table 41). Plan:

Table 41: A19, the frozen Laya's output on bf16 (vLLM-TPU) against Q4_K_M (llama.cpp b11476, CPU) answers of Qwen3.5-2B to the same 80 night tasks (`quant\_shift\_probe.json`; Q4_K_M minus bf16; paired cluster bootstrap, 95% CI). The tasks are in E, which the Laya trained on, so absolute values are in-sample; only the paired difference is read. Math pairs differ in protocol (the night math used the code-only prompt path), not only in precision.

domain	pairs	mean Δp [CI]	mean $\lvert\Delta p\rvert$ [CI]	Spearman of p [CI]	gate verdict differs	identical text	mean $\lvert\Delta p\rvert$, identical text
Python (primary)	28	+0.096 [−0.020, +0.230]	0.162 [0.054, 0.287]	0.64 [0.28, 0.90]	5 of 28	9 of 28	0.030
Rust (gate-only domain)	26	−0.024 [−0.174, +0.126]	0.177 [0.042, 0.326]	0.65 [0.22, 0.91]	5 of 26	4 of 26	0.000
math (protocol-confounded)	26	−0.182 [−0.286, −0.081]	0.239 [0.158, 0.325]	0.79 [0.58, 0.90]	0 of 26	0 of 26	n/a

Table 42: A20, replay of the registered E′ router policy through the runtime loop `GwenLaya.answer` on stored inputs (`system\_replay\_eprime.json`). No model ran; the checker returns the stored gate verdict and the clock the stored accelerator-seconds. Agreement is counted per task on start tier, final tier, gate-verdict path, cost and answered flag.

	E′ (math, Python)	R′ (Rust; orchestration check)
tasks / all five items agree	828 / 828	69 / 69
disagreements listed	0	0
cost ratio vs always-9B, runtime / registered	0.7879 / 0.7879	0.9872 / 0.9872
accuracy change vs always-9B, registered quantity (points)	0.97	0.00
answered accuracy of the runtime output, policy / 9B-only	0.621 / 0.581	0.536 / 0.536
runtime verdicts	VERIFIED 547, ABSTAIN 235, LIKELY_WRONG 46	VERIFIED 45, LIKELY_WRONG 24
scorer feature mismatches vs stored rows	0	0

Table 43: A22, candidate-first layout (v2) against the frozen layout (v1), one training run, one seed (`layout\_a22.json`). AUROC of P(correct); paired cluster bootstrap, 95% CI, cluster = problem. The registered rule calls v2 better only if ΔAUROC (v2 − v1) has a CI above zero on E′ *and* on R′.

set	slice	n	AUROC v1 / v2 / LR	ΔAUROC v2 − v1 [CI]	ΔAURC v2 − v1 [CI]	ΔAUROC v2 − LR [CI]
E′	pooled	2484	0.941 / 0.932 / 0.929	−0.010 [−0.016, −0.004]	+0.013 [+0.002, +0.024]	+0.003 [−0.009, +0.015]
E′	2B	828	0.923 / 0.920 / 0.918	−0.004 [−0.013, +0.006]	+0.016 [+0.004, +0.030]	+0.002 [−0.016, +0.019]
E′	4B	828	0.952 / 0.937 / 0.935	−0.015 [−0.025, −0.006]	+0.014 [−0.007, +0.037]	+0.001 [−0.016, +0.019]
E′	9B	828	0.945 / 0.937 / 0.933	−0.009 [−0.018, +0.000]	+0.007 [−0.010, +0.023]	+0.004 [−0.011, +0.020]
R′	pooled	207	0.862 / 0.867 / 0.921	+0.004 [−0.027, +0.034]	+0.016 [−0.029, +0.070]	−0.054 [−0.104, −0.010]
R′	2B	69	0.815 / 0.841 / 0.968	+0.026 [−0.004, +0.071]	−0.022 [−0.077, +0.020]	−0.127 [−0.228, −0.044]
R′	4B	69	0.815 / 0.859 / 0.885	+0.044 [−0.007, +0.105]	−0.029 [−0.107, +0.041]	−0.026 [−0.103, +0.046]
R′	9B	69	0.891 / 0.882 / 0.925	−0.009 [−0.074, +0.059]	+0.048 [−0.030, +0.135]	−0.043 [−0.135, +0.030]

Registered rule (`decision`): lower CI bound of ΔAUROC −0.0161 on E′ and −0.0270 on R′; v2 better = false; window explanation supported = false.

Table 44: Ledger lines added since 3.9.0 (`spend\_summary\_3100.json`, from `spend\_ledger.jsonl`); USD is the list-price estimate of the ledger, not the billed amount. The purpose column is inferred from the item names and the commit log, not from VM logs.

ledger time	item	purpose	seconds	USD
2026-10-10 19:11	`tpu_f-9b-185104`	F generation, 9B (v5litepod-4)	1,016	1.3547
2026-10-10 19:28	`tpu_f-small-185104`	F generation, 2B and 4B	2,139	0.7130
2026-10-11 15:35	`tpu_f-laya-152622`	frozen Laya prediction on F	444	0.1480
2026-10-11 15:54	`tpu_laya-layout-152621`	A22 candidate-first layout training and prediction	1,529	0.5097
	added by revision 3.10.0 (4 lines)			2.7254
	ledger total (31 lines; 27 lines, 19.2801 at 3.9.0)			22.0055

addendum A19 (`1bb7543`, 14:18:21), results `73d1a15` (14:41:33). The 80 night tasks (Python 28, Rust 26, math 26) answered by Qwen3.5-2B Q4_K_M under llama.cpp b11476 on CPU are paired with the stored bf16 training rows of the same tasks and tier; the frozen Laya ran through the runtime path on both sides. **Descriptive, in-sample (the tasks are in E), 2B only, no 4B or 9B pairs exist, one greedy sample per side.** On the 28 Python pairs the paired difference in the calibrated p (Q4_K_M minus bf16) is +0.096 [−0.020, +0.230], the mean absolute difference 0.162 [0.054, 0.287] (maximum 0.913), the rank correlation 0.64 [0.28, 0.90]; the gate verdict changed for 5 of 28 (4 failed to verified, 1 verified to failed); 9 of 28 answers have identical text and for those the mean absolute difference is 0.030, so most of the shift comes from the answers differing, not from the log-prob signal. The two sides differ in the whole serving stack (vLLM-TPU bf16 against llama.cpp on CPU; log-prob definitions and rounding may differ), so the measured shift is that of the local serving stack against the training stack, not of 4-bit weights alone. The math pairs are protocol-confounded (the night math used the code-only prompt path; none boxed) and are not a quantisation contrast. **Not shown:** that the calibration holds or fails under quantisation, anything for 4B or 9B, anything about accuracy, or that the exploratory thresholds (`thresholds_C.json`, in-sample on C, never registered) survive.

A20, the runtime loop reproduces the registered E′ policy on stored inputs (Table 42). Plan `cca2049` (15:01:14), results `294054e` (15:04:15). `GwenLaya.answer` was fed the stored E′ greedy answers, the stored frozen-router probabilities, the stored calibrator scores and the stored gate verdicts, with an injected clock charging the stored accelerator-seconds; the per-task outcome (start tier, final tier, gate-verdict path, cost, answered flag) equals the offline `run_policy` outcome for **all 828 tasks** (5 of 5 items), with the registered cost ratios recomputed (0.7879 with the first chunk, 0.8946 without) and the accuracy change of +0.97 points. One difference of definition was stated in advance: the registered accuracy counts the final candidate even where no gate verified, while the runtime withholds such answers (547 VERIFIED, 235 ABSTAIN, 46 LIKELY_WRONG); its answered accuracy is 0.621 against 0.581 for the 9B-only arm under the same rule (+3.99 points), a different quantity that is not a new claim. The replay checks orchestration only: the router, calibrator and gate outputs are the stored ones, so it says nothing about live tiers, quantised tiers or the model weights (parity does that). The same replay on R′ agrees on 69 of 69 (cost ratio 0.987, accuracy change 0).

Explicitly not validated. (i) The integrated system on Q4_K_M tiers: calibration under the A19 shift, accuracy, LIKELY_CORRECT precision, abstention behaviour and cost have no evaluation; the A19 probe has 28 pairs on one tier. (ii) End-to-end accuracy of the routed-and-scored system on live tiers of any precision (the live smoke has $n = 3$). (iii) Any threshold: τ_{hi} and τ_{lo} default to $\pm\infty$ (Laya never decides LIKELY_CORRECT or LIKELY_WRONG), and the exploratory thresholds are in-sample on C. (iv) Behaviour under candidate truncation and missing log-probs beyond the fail-closed veto. (v) The 4B path of the llama.cpp tier (unit-tested with a fake server only) and a 9B tier (no server script); concurrency; latency on a free machine. (vi) Rust and Lean (Laya off and refused). (vii) A tier map from a non-Qwen3.5 model to a trained tier name is an untested assumption. (viii) The lazy program-of-thought cost against the registered charge.

7.14 Revision 3.10.0: does the candidate's position in the window explain the text signal (A22)

Order of events. Addendum A22 was committed before the run (`2a76e2f`, 2026-10-10 15:43:58) with its decision rule; the layout code and launch scripts followed (`3134a94`, `a22f173`, `2e41bcb`, 15:52); the TPU run (ledger item `tpu_laya-layout-152621`, 1,529 s, $0.5097) and its scoring were on 2026-10-11 and the result is in `b98a627` (15:59:42). **The experiment.** The frozen layout renders domain, tier, signals, prompt, candidate and cuts at 512 tokens, so a long prompt hides the candidate. v2 renders the candidate first (domain, tier, signals, candidate, prompt) and is otherwise the calibrator recipe unchanged (same 9,243 training rows, 3 epochs, LoRA r = 16,

seed 1234, 16 numeric features). The router was not retrained. Scoring is on the same stored rows as v1: E′ (Python and math, 828 tasks × 3 tiers), R′ (69 × 3), C for the isotonic map. Decision rule, fixed in advance: v2 is better only if the paired AUROC difference v2 − v1 has a 95% interval above zero on *both* E′ and R′.

Result (Table 43): v2 is not better. E′ pooled ΔAUROC −0.010 [−0.016, −0.004] (v1 0.941, v2 0.932): the interval is below zero, i.e. v2 is slightly worse; R′ pooled +0.004 [−0.027, +0.034] (0.862 against 0.867): indistinguishable. The 2B and 4B R′ points are +0.026 and +0.044 with intervals including zero. v2 is +0.003 [−0.009, +0.015] above the hand-feature regression on E′ and −0.054 [−0.104, −0.010] below it on R′. By the registered rule the explanation that the candidate's position in the window is why the text carries no signal on R′ is **not supported by this experiment**; v1 stays the frozen model; v2 was not frozen and was not evaluated on F (per the plan that step was conditional on passing the rule).

What was not done (D74). The plan promised the share of rows whose candidate is at least partly and fully inside the 512-token window under each layout, computed with the ModernBERT tokenizer, so that "v1 was blind on x% of rows" would be a measured number; the analysis environment has no tokenizer for it (offline), so **these fractions were not computed**. The earlier statement that the candidate lies entirely outside the window for 103 of 207 R′ rows comes from the 3.7.0 audit, not from this run. Limits as registered: single seed (training noise is not separated from layout), no shuffled-label control (the control of D50 failed for v1), E′ and R′ are not pristine, and a null result does not show that text is useless; the `middle\_truncate` layout was implemented and unit-tested but not run.

7.15 Revision 3.9.0: the Python depth dial (A17)

Order of events. Plan addendum A17 (`docs/ANALYSIS_PLAN_E_TPU.md`) was committed in `6d229b1` (2026-10-09 21:52:55) before it was run and with no Python depth result seen; the script and its splitter tests followed in `499b9eb` (21:54:18) and the results in `f19e283` (22:53:02). Like A15.2 it is exploratory, CPU-only, on stored answers, and tests no hypothesis. Numbers are read from `results/gwenlaya_v4/edition/python_gate_depth.json` by `scripts/make_tables_390.py`.

Construction. The Python hidden tests of E are generated suites with one `inputs` list and one `results` list. The k-gate is the same hidden test text with both lists cut to their first k elements ($k = 1, 2, 4, 8$), located with `ast`; 524 of the 529 tasks have at least 8 pairs and are used at every k (2 have no such lists, 3 fewer than 8). The logical sanity check fixed in A17 (every answer that passes the full hidden tests must pass every k-gate) found no violation for either tier. Tiers 4B and 9B. The stored one-assertion gate is reported on the same tasks; at $k = 1$ the A17 gate is a different form of test from the stored one and gives almost the same numbers (Table 45).

What the numbers show. As in the Rust dial (A15.2, Table 29), each added visible test lowers coverage and raises precision, and the confident-wrong rate over all tasks falls monotonically in both tiers: 4B 0.139, 0.115, 0.082, 0.036 and 9B 0.147, 0.126, 0.090, 0.046 for $k = 1, 2, 4, 8$. Coverage falls from 0.815 to 0.712 (4B) and from 0.872 to 0.771 (9B); precision rises from 0.829 to 0.949 and from 0.832 to 0.941. The intervals at $k = 1$ and $k = 8$ do not overlap for either tier; they are marginal bootstrap intervals, not a paired test. On Rust, three visible assertions brought the rate from 0.067 to 0.024 (4B) and from 0.061 to 0.014 (9B); Python starts from a higher one-test rate and needs more tests for a similar level. With eight tests the 4B precision point estimate (0.949) reaches about the registration's 0.95 flag point; the 9B's (0.941) does not. This is a dial on the stored answers, not a threshold and not a recommendation for a k.

Correction: the prompts did show tests (D63). A17 says "Python answers were generated against prompts that showed no tests". That is false, and I verified it on the task file (`prompt_test_visibility_390.json`): all 373 MBPP prompts of E (368 of them in the dial's task set) contain the gate's assert word for word, and 71 of the 156 HumanEval prompts carry doctest examples. For 344 of the 368 MBPP tasks in the dial the first hidden input is exactly

Table 45: A17, the Python visible-test depth dial on the stored E answers (`python_gate_depth.json`). 524 of 529 Python tasks have at least 8 input/result pairs and are used at every k (excluded: 2 with no pairs, 3 with fewer than 8). The k-gate is the hidden test text with both lists cut to their first k elements; "stored" is the one-assertion gate as stored in E, on the same tasks. Coverage = share of tasks verified; precision = share of verified answers that pass the hidden tests; CWR = verified and wrong, over all tasks. 95% cluster-bootstrap CIs (10,000 resamples, seed 0; cluster = problem), marginal per k, not a paired test. Descriptive; no hypothesis is tested. Most of these prompts showed the model a test (Section 7.15).

Tier	gate	coverage [CI]	precision [CI]	CWR [CI]
4B	stored (1)	0.813 [0.779, 0.845]	0.831 [0.795, 0.866]	0.137 [0.109, 0.168]
	$k = 1$	0.815 [0.781, 0.847]	0.829 [0.793, 0.864]	0.139 [0.111, 0.170]
	$k = 2$	0.790 [0.754, 0.824]	0.855 [0.821, 0.888]	0.115 [0.088, 0.143]
	$k = 4$	0.758 [0.721, 0.794]	0.892 [0.861, 0.921]	0.082 [0.059, 0.107]
	$k = 8$	0.712 [0.672, 0.750]	0.949 [0.926, 0.970]	0.036 [0.021, 0.053]
9B	stored (1)	0.870 [0.842, 0.899]	0.831 [0.796, 0.866]	0.147 [0.116, 0.177]
	$k = 1$	0.872 [0.844, 0.901]	0.832 [0.797, 0.866]	0.147 [0.116, 0.177]
	$k = 2$	0.851 [0.821, 0.880]	0.852 [0.818, 0.885]	0.126 [0.097, 0.155]
	$k = 4$	0.815 [0.781, 0.847]	0.890 [0.859, 0.919]	0.090 [0.067, 0.115]
	$k = 8$	0.771 [0.735, 0.807]	0.941 [0.916, 0.963]	0.046 [0.029, 0.065]

the argument of that printed assert (6 differ, 18 have an assert form my check does not parse), so at $k = 1$ the gate mostly re-runs the one test the model was shown. Two consequences. (i) The one-test gate's precision on Python measures, for most tasks, whether the model passes a test it could read, which is a weaker check than an unseen test; the dial's later tests are mostly unseen. (ii) The A17 limit that this "describes the gate on those answers, not what a model would write if it saw the tests" should read: not what a model would write if it saw more tests. I do not know in which direction a cleaner design (no test in the prompt) would move the dial. The same misstatement affected the 3.7.0 Rust limit (Section 7.8). A deviation entry, D63, records this. (The registration itself defines the Python gate as the first public test, Section 1 of the registration; for MBPP that test is the one printed in the prompt, so the design was as registered and the error is in the A17 text.)

Limits. Stored answers, one greedy sample, the burned E set, public benchmarks, 4B and 9B only, no R′ or E′; tests beyond k are hidden tests, so k = all is the hidden verdict by construction.

7.16 Revision 3.9.0: gate strength and the verdict policy (system changes)

These are code changes, not results. `gwaya/domains/strength.py` and `checkers._apply_strength` (`da7d708`) record with every Python and Rust gate verdict how many visible tests stood behind it (`visible_tests`: Python, the number of input/result pairs of a generated suite or else of `assert` statements; Rust, `\#[test]` functions or else assert macros, after stripping comments and strings, fixed in `dfd41aa`). The count is a static lower bound and says nothing about test quality. A task payload may set `min_visible_tests`; a pass below it is then UNVERIFIED (fail-closed). `gwaya/gwenlaya.py` (`dfd41aa`) adds a verdict policy, written into every output's evidence:

- **Weak gate** (Python, Rust; **off by default**). With $k_{\min}$ configured for a domain (argument or `GWENLAYA_MIN_VISIBLE_TESTS`), a gate pass backed by fewer than $k_{\min}$ visible tests, or by an unknown count, is not reported as VERIFIED; it takes the calibrator path of an unverified answer (LIKELY_CORRECT, LIKELY_WRONG only through the low threshold, or escalate/abstain). No $k_{\min}$ has been chosen, because no held-out data exist for choosing it.

- **Lean.** A Lean task without a non-empty formal statement is never VERIFIED (the checker could not tell the task's theorem from an easier one the model wrote), and a Lean answer is never LIKELY_CORRECT (the frozen Laya never saw Lean, so only the kernel can vouch).

Table 46: Exploratory, not pre-registered: what the weak-gate policy would do to the stored gate verdicts (`verdict_policy_exploratory.json`, key `stored_gate`, Rust counts after the counting fix). "1 test" = verified answers whose gate payload has exactly one visible test. "Wrong among verified" = share of verified answers that fail the hidden tests (no policy). "Kept at $k_{\min} = 2$" = verified answers that would stay VERIFIED; the others would be demoted to the calibrator path, which is not counted as wrong. The policy is off by default and no $k_{\min}$ is recommended; E is an evaluation set.

Set	Tier	tasks	verified	1 test	wrong among verified	kept at $k_{\min} = 2$
E Python	2B	529	313	313	0.240	0
E Python	4B	529	430	430	0.172	0
E Python	9B	529	460	460	0.167	0
E Rust	2B	507	186	186	0.172	0
E Rust	4B	507	302	302	0.113	0
E Rust	9B	507	329	329	0.094	0
R′ Rust	2B	69	21	21	0.714	0
R′ Rust	4B	69	40	39	0.300	1
R′ Rust	9B	69	45	43	0.178	2

What the stored data say (exploratory, not pre-registered; Table 46). `scripts/verdict_policy_exploratory.py` counted the visible tests in each stored gate payload. Every VERIFIED of the stored E gate, Python and Rust, all tiers, rests on exactly one visible test; on R′ three verified answers (one 4B, two 9B) rest on more. So any $k_{\min} \geq 2$ would demote every stored E VERIFIED to the calibrator path, and on R′ keep three. The table's "wrong among verified" is read from the stored rows without the serial-recheck overlay of A8, so it can differ by one task from Table 5 (4B Python: 0.172 here, 0.170 from the published precision 0.830). Nothing here shows that the policy lowers confident-wrong answers for a user: a user supplies their own tests, and E, which set no default, is an evaluation set. What A15.2 and A17 suggest is only that more visible tests make VERIFIED more trustworthy on these answers. None of the published results uses the policy. Because it changes what VERIFIED means when it is switched on, it is recorded as deviation D64.

7.17 Revision 3.9.0: the Lean 4 gate (system changes; no results)

The registration (Section 1) lists a Lean 4 gate ("`lake env lean` kernel check + `\#print axioms` audit; precision = 1 by construction") and miniF2F-lean4 (244 statements) as a secondary set. Lean was dropped from the night run (D7) and had produced no result up to 3.9.0. **Superseded in 3.10.0:** A18 was generated and scored and A21 added a second set (Section 7.11); the text of this subsection is that of 3.9.0, kept for the record, and its statements about A18 being in progress, a re-run of the sampling and no scored proof are withdrawn (Section 2.1).

- **The gate was unusable.** Two defects made every miniF2F proof impossible to verify: the checker never passed the pinned Mathlib project (Lean v4.7.0, Mathlib `a45ae637`) to the oracle, so every Mathlib import was UNVERIFIED (`aa572bb`); and the loader kept the dataset's trailing `:= sorry` inside the statement the gate requires verbatim, and `sorry` is rejected (`d97e390`).
- **Soundness holes.** An internal audit of the repaired gate (its findings are in the A18 amendment, `f318f71`, and reproduced as tests; no separate report is committed) found that "precision 1 by construction" did not hold: the statement check was a raw substring

test (the statement inside a comment or a string, next to an easier theorem, passed); `variable` hypotheses, local notation and instances and namespace shadowing could change the meaning of an intact statement; a `\#eval` hidden in a string could print fake axiom-audit lines; and a task with no formal statement accepted any compiling file.

- **The hardened gate** (`gwaya/lean_gate.py`, `24bc5c0`). The reference (task header, formal statement, `by sorry`) is compiled alone; the candidate is compiled separately; a *trusted second Lean process* with fixed source reads both compiled modules as data, replays every candidate declaration through the kernel, and accepts only if the target is a theorem whose type equals the reference type (exact expression equality, or else kernel definitional equality; which one is recorded) and whose `collectAxioms` closure is within `propext`, `Classical.choice` and `Quot.sound`. Its output lines carry a per-run random nonce, so candidate output cannot spoof them. Lean-aware lexical rules (comments and strings stripped first) reject `variable`, notation, instances, attributes, namespaces, metaprogramming, IO and escape hatches. `tests/test_lean_gate.py` reproduces each audit exploit; in my run for this revision all 94 tests passed, including the 13 that use the real pinned project (four genuine miniF2F proofs VERIFIED with an exact match, a wrong proof FAILED, and each exploit rejected by the trusted process alone with the lexical rules bypassed; `results/gwenlaya_v4/lean_a18/lean_gate_tests_390.log`).
- **Limits of the hardened gate.** (i) Binder annotations (explicit or implicit arguments) are not compared: both the expression equality and the definitional-equality check ignore them. (ii) A definitional (not exact) match counts as VERIFIED; A18 results will report exact and definitional matches separately. (iii) The external red-team review of the hardened gate was not completed. (iv) The strict lexical rules reject some harmless constructs (for example any `namespace`, `variable` or local instance in a proof), so some valid proofs will be FAILED. (v) A check is slow: in my run a VERIFIED genuine proof took 12 to 21 s per check at a load average of about 3 on 8 cores (log above); it will be longer under heavier load. (As of 3.9.0 no proof had been scored with it; see Section 7.11.)
- **A18.** Plan addendum A18 (`5b53ce8`, before any Lean generation) and its amendment (`f318f71`, before any Lean generation; score only with the hardened gate) pre-register a descriptive Lean miniF2F run with the 2B, 4B and 9B tiers (greedy, and $k = 8$ samples). The sanity check (`lean_sanity.json`) keeps 223 of 244 statements: 17 are commented out in the dataset and 4 do not elaborate under the pinned Mathlib. The generation began after the amendment and is in progress (the $k = 8$ sampling is being re-run after an engine-seed fix, `302ab95`). **No Lean result is reported in this revision.** Its ledger lines are counted in Section 11. Replacing the registered Lean check by this gate is deviation D65.

7.18 Revision 3.8.0: a text-only ablation of Laya's calibrator (A16)

Order of events. Plan addendum A16 (`docs/ANALYSIS_PLAN_E_TPU.md`; commit `3d96b6f`) (committed 20:40:12) preceded everything that was run: nothing had been trained or scored when it was committed. Disclosure: the TPU VM of the ablation (ledger item `tpu_laya-textonly-204124`, created about 20:41:24) was created about 4 seconds *before* the training-code commit `df11747` (20:41:28); the code that ran was shipped from the working tree and committed seconds later. The training code (`df11747`) and analysis code (`a8f203d`, 20:42:02) followed, then the results commit (`2890b51`). It is post hoc with respect to the E′ and R′ results of Laya (D59), exploratory, run once with one seed (D60), and no hypothesis is tested; intervals are descriptive.

What was run. The calibrator recipe of the frozen Laya (ModernBERT-large, LoRA rank 16 on `Wqkv`, `Wo`, `Wi`; same training rows and validation hash; `-text-only`) with the input reduced to domain, tier, prompt and candidate text: the `[signals]` line is removed and no numeric feature reaches the head (`head_config_textonly.json`: `n_features` 0, `text_only` true). It trained

1,566 steps (the same count as the frozen Laya); its validation AUROC is 0.855 (n = 903) against 0.927 for the full Laya (`run_manifest_textonly.json`, `run_manifest_calibrator.json`). An isotonic map was fitted on C only. It is a different model from the frozen Laya, which was not touched. Numbers are read from `textonly_a16.json` by `scripts/make_tables_380.py`.

Table 47: Text-only ablation of Laya's calibrator (A16; `textonly\_a16.json`), AUROC of P(correct), one seed. Isotonic map fitted on C only. 95% paired cluster-bootstrap CIs (10,000 resamples, seed 0); descriptive, no multiplicity correction, no hypothesis is tested. Δ columns are differences of AUROC: text-only minus raw mean log-prob; text-only minus the hand-feature logistic regression; the frozen full Laya minus text-only. On R′ the log-prob baseline ranks the one answer per tier with a missing `mean\_logprob` lowest (treated as −20); the hand-feature regression uses 0; see the text for the other convention.

Set	Tier	n	text-only AUROC [CI]	text − log-prob	text − hand LR	full − text
E′	9B	828	0.870 [0.843, 0.895]	+0.134 [+0.099, +0.171]	−0.063 [−0.088, −0.039]	+0.076 [+0.056, +0.096]
E′	4B	828	0.869 [0.843, 0.893]	+0.130 [+0.096, +0.166]	−0.067 [−0.093, −0.041]	+0.082 [+0.062, +0.104]
E′	2B	828	0.839 [0.811, 0.865]	+0.086 [+0.051, +0.122]	−0.079 [−0.107, −0.052]	+0.084 [+0.063, +0.106]
R′	9B	69	0.588 [0.455, 0.713]	−0.147 [−0.306, +0.011]	−0.338 [−0.480, −0.201]	+0.299 [+0.153, +0.444]
R′	4B	69	0.555 [0.418, 0.684]	−0.110 [−0.270, +0.049]	−0.330 [−0.480, −0.184]	+0.263 [+0.111, +0.422]
R′	2B	69	0.405 [0.180, 0.639]	−0.370 [−0.605, −0.131]	−0.563 [−0.771, −0.343]	+0.410 [+0.225, +0.598]

Table 48: A16, text-only AUROC within domain on E′ (`textonly\_a16.json`, keys `auroc\_text.math` and `auroc\_text.python`); 95% CI. The domain token is in the text, so a within-domain AUROC above 0.5 rules out a domain prior only, not a source or difficulty prior.

Tier	math [CI]	Python [CI]
9B	0.895 [0.866, 0.922]	0.704 [0.625, 0.779]
4B	0.891 [0.862, 0.918]	0.724 [0.647, 0.796]
2B	0.846 [0.814, 0.876]	0.747 [0.672, 0.817]

What the numbers show. On E′ (n = 828) text-only AUROC is 0.839, 0.869 and 0.870 on the 2B, 4B and 9B, with every interval well above 0.5 (Table 47). It is above the raw log-prob (+0.086, +0.130, +0.134) and below the hand-feature logistic regression (−0.079, −0.067, −0.063); the full Laya is above it by +0.084, +0.082 and +0.076; every one of these intervals excludes zero. Within domain (Table 48) it remains above 0.5 for math (0.846, 0.891, 0.895) and for Python (0.747, 0.724, 0.704; Python has 169 tasks and the intervals are wide). On the fresh Rust set R′ (n = 69) text-only is at chance: AUROC 0.405, 0.555 and 0.588 (2B, 4B, 9B), all intervals include 0.5 (lowest lower bound 0.180, highest upper bound 0.713). The raw log-prob baseline ranks the one R′ answer per tier whose `mean_logprob` is missing (3 rows in all) as the *lowest* (treated as −20); the hand-feature regression uses 0 for it. With the other convention (missing treated as 0 for the log-prob baseline) the three differences from the log-prob would be −0.354, −0.085, −0.116 (2B, 4B, 9B; the auditor's recomputation, not a committed file), so the 4B and 9B intervals include zero either way. Text-only is below the raw log-prob in point terms (the 2B interval, −0.605 to −0.131, excludes zero; the 4B and 9B intervals include it, upper bounds +0.049 and +0.011), and below the hand-feature regression (−0.563, −0.330, −0.338) and the full Laya (+0.410, +0.263, +0.299), all with intervals excluding zero.

Reading (hypotheses; none was tested by an ablation of its cause). (i) In distribution the encoder reads something predictive of correctness from the prompt and candidate text. The within-domain result rules out a domain prior only; it does not rule out a source or difficulty prior (for example GSM8K against MATH, or answer form), which would predict correctness without being specific to the candidate. (ii) That signal does not transfer to the Exercism-style Rust prompts of R′. Possible reasons, untested here: long prompts push the candidate out of the

512-token window; the Rust training prompts came only from MultiPL-E (E, in the training set); the prompt style differs. A text-only score at chance on 69 tasks is also compatible with a small true effect. (iii) Hypothesis: most of what Laya gains on R′ comes from its gate and log-prob inputs. This is consistent with the numbers: the full Laya exceeds the text-only model by 0.26 to 0.41 AUROC on R′ (0.076 to 0.084 on E′). The converse ablation (numeric inputs only, no text) is essentially the hand-feature regression, which is a different model; no causal decomposition was run.

Limits. One seed, one training run; AUROC only is the focus (AURC, ECE and Brier are in the file); the candidate was cut at 2,000 characters and the input at 512 tokens, as for the frozen Laya; no shuffled-label control was run for this model, and the control of Section 7.3 failed for the frozen one, so the ablation inherits that unresolved question; the ablation does not remove the text from the full model.

7.19 Deviations of revision 3.6.0

Table 49: Deviations D48–D52 of revision 3.6.0 (full text in `docs/DEVIATIONS.md`, committed in `d217f1e` before E′ generation).

#	Registered / before	Done here	When
D48	router-train, calibration and evaluation split by the reserved 20-bucket rule	new pool split by cluster hash mod 10 (0–5 train, 6 C, 7–9 E′) to leave a usable training set	before generation of E′
D49	training only on data disjoint from the evaluation set (E burned)	all 1,536 E tasks in the Laya training set; E′ shares no cluster, task id or prompt hash with training (0/0/0); C 0/0/1	before generation of E′
D50	shuffled-label control AUROC in 0.4–0.6 on C, else stop (A13b)	control scored 0.693 (calibrator) and 0.572 / 0.637 / 0.389 (router); untrained twin 0.480 pooled, 0.239–0.514 in strata, router 0.653 / 0.482 / 0.575; criterion withdrawn and replaced by a direct leakage check. Applied as written, E′ would not have been generated; the offered reading is not proven	after seeing the C control scores, before generation of E′
D51	tiers 2B/4B/9B/27B; generator LoRA probe	27B abandoned (did not load in 33 min; 8-chip slice blocked by the 4-chip per-zone quota); LoRA probe finished no step in 25 min	measured, during A13b
D52	one Laya training run	first run lost (save of XLA tensors failed, router tiers not passed), repeated after fixes; about $0.9 spent on the lost run	after the first run

7.20 Spend of this revision

7.21 Deviations and spend of revision 3.7.0

7.22 Deviations of revision 3.9.0

7.23 Deviations of revision 3.10.0

8 Hypothesis status

Table 55 restates `docs/RESULTS_VERDICT.md`. The probability column is the **red-team prior** written before any data (`docs/PROPOSAL_REVIEW.md`, copied into `docs/DEVIATIONS.md`). It is *not* an estimate from data and was not updated by any result in this paper. No exploratory analysis of Section 6 is a verdict on a registered hypothesis. Where an exploratory analysis is related, the table says so and why it is not the test.

Revision 3.6.0: H1 and H3 were run on a fresh E′ (math, Python) with deviations (Section 7.2); revision 3.7.0 repeated them with the same frozen Laya on a fresh Rust set R′, where neither is supported (Section 7.7); revision 3.10.0 repeated them once more, unchanged, on fresh Python and math sets F (908 tasks), where H1 is

Table 50: Ledger lines added since 3.5.0, with item names as written in the ledger. By name, `p-*` lines are the pool P generation, `laya-probe` the probe, `b27` the 27B attempt, `laya-train` the lost first run, `laya-train2` the repeat, `laya-twin` the untrained twin, `ep-*` the E′ generation and `laya-pred` the prediction; I inferred this mapping from the names and did not check it against the VM logs.

Time	Item	Seconds	USD
2026-10-09 11:50	`tpu_p-9b-112647` (v5litepod-4, us-south1-a)	1,248	1.6640
2026-10-09 12:19	`tpu_p-small-112647` (v5litepod-1, us-west4-a)	2,970	0.9900
2026-10-09 12:21	`tpu_laya-probe-114619` (v5litepod-1, us-west4-a)	1,876	0.6253
2026-10-09 13:06	`tpu_b27-tp4-122313` (v5litepod-4, us-west4-a)	2,463	3.2840
2026-10-09 14:15	`tpu_laya-train-132634` (v5litepod-1, us-west4-a)	2,820	0.9400
2026-10-09 15:25	`tpu_laya-train2-141806` (v5litepod-1, us-west4-a)	3,877	1.2923
2026-10-09 16:37	`tpu_laya-twin-162802` (v5litepod-1, us-west4-a)	445	0.1483
2026-10-09 17:01	`tpu_ep-9b-164158` (v5litepod-4, us-south1-a)	1,000	1.3333
2026-10-09 17:13	`tpu_ep-small-163947` (v5litepod-1, us-west4-a)	1,897	0.6323
2026-10-09 17:33	`tpu_laya-pred-172040` (v5litepod-1, us-west4-a)	413	0.1377
	Added since 3.5.0 (10 lines)		**11.0472**
	Lines 1–11 (as in 3.5.0)		4.9136
	Total (21 lines, sum of `usd\_estimate`)		**15.9608**

Table 51: Deviations D53–D58 of revision 3.7.0 (full text in `docs/DEVIATIONS.md`).

#	Registered / before	Done here	When
D53	no Rust evaluation after E′	R′ built post hoc, after the E′ results were known; A14 committed before any model generated for it	before R′ generation
D54	tasks fixed before any check	69 of 106 exercises kept by a reference-solution and starter-stub check with the real checker (latency $\leq$ 4 s); no model answer used in the filter	before R′ generation
D55	Rust harness as validated in 3.3.1	found: `rustc` without `-edition` (edition 2015) in all published runs; 2021-prelude exercises dropped from R′; measured in A15.1 (see D58)	while building R′
D56	τ chosen on a calibration split	$\tau = 0.5$ from C (math, Python) reused on Rust, not re-chosen	before R′ generation
D57	—	three ledger lines added; field names `E\_prime` in `laya\_rfresh.json` are inherited from the script	after the results
D58	decision rule of A15 (written before the results): published numbers keep the label 2015; future default 2021; re-analyse if a confirmed flip rate exceeds 2%	oracle default changed to 2021 (`DEFAULT\_RUST\_EDITION`); confirmed flips 1/507 (E 4B), 1/507 (E 9B), 1/69 (R′ 4B), none pass-to-fail, so no re-analysis. Depth dial run as $k = 1, 2, 3$ on E Rust only, where A15 said $k = 1, 2, 4$ on Python and Rust of E and R′	after A15 (before the results)

Table 52: Ledger lines added since 3.6.0. By name, `rf-*` are the R′ generations (`-9b-` on the four-chip slice) and `laya-pred-r` the Laya prediction on R′; inferred from the names, not checked against the VM logs.

Time	Item	Seconds	USD
2026-10-09 18:39	`tpu_rf-9b-182548` (v5litepod-4, us-south1-a)	640	0.8533
2026-10-09 18:42	`tpu_rf-small-182548` (v5litepod-1, us-west4-a)	834	0.2780
2026-10-09 18:54	`tpu_laya-pred-r-184359` (v5litepod-1, us-west4-a)	450	0.1500
	Added since 3.6.0 (3 lines)		**1.2813**
	Lines 1–21 (as in 3.6.0)		15.9608
	Total (24 lines, sum of `usd\_estimate`)		**17.2421**

Table 53: Deviations D62–D65 (full text in `docs/DEVIATIONS.md`). D62 was recorded with the scorer commits; D63–D65 are added with this revision.

#	Registered / before	Done here	When
D62	registered math scorer (`math_check.py` at `78cf88b`)	normalization of notational equivalences, fail-closed; exploratory re-score of stored E math (accuracy and math-gate numbers in Table 10); registered numbers stay primary; not propagated to A4, A5, A11, A12 or Laya labels	post hoc, after inspecting the stored outputs
D63	A17 limit: "answers were generated against prompts that showed no tests"; 3.7.0 A15.2 limit: Rust prompts "showed one test"	both wrong (the Rust one in part): all 373 MBPP prompts print the gate assert verbatim, 71 of 156 HumanEval prompts carry doctests; no Rust prompt contains the gate assertion literally, but in 87 of 155 HumanEval-Rust prompts the gate's first call and its expected value are shown as a doc-comment example (about 17% of E Rust); none of the 352 MBPP-Rust prompts shows a test. A17 numbers unchanged; their reading corrected	after the A17 result
D64	VERIFIED = all gate-visible checks passed	opt-in weak-gate policy (off by default; $k_{\min}$ not chosen) can demote a pass to the calibrator path; Lean never LIKELY_CORRECT; Lean without a formal statement never VERIFIED. No published result uses it	after A15.2 and A17
D65	Lean gate = `lake env lean` + `\#print axioms`, precision 1 by construction	hardened gate: trusted second Lean process, elaborated-type comparison (exact or definitional, recorded), `collectAxioms`, strict lexical rules; binder annotations not compared; external red-team review not completed. A18 is scored only with it (amendment `f318f71`)	before any Lean proof was scored

supported and H3 is not (Section 7.10). The following paragraph, from 3.5.0, explains why the E generations themselves cannot be used.

H1 and H3 cannot be run as registered on these E generations. The registration's freeze rule (Section 6) requires the calibrator, all thresholds and the E manifest hash to be fixed in a signed addendum *before E is generated*, and it forbids E-based refitting. E has now been generated, and its outcomes analysed and published, before split C or the Laya head exist. A registered test therefore needs one of two things:

- fresh generations on a new evaluation set E′ after the freeze (for example, problems released after the models' training cutoff, which would also reduce contamination); or
- a recorded deviation stating that the E generations predate the freeze and that their outcomes were seen, in which case any H1/H3 run on them is a deviated test.

H13 (budget). The decision rule has two parts: the ledger total must be $\leq$ \$40 planned and $\leq$ \$50 hard, and no stage may exceed 1.25× its cap twice. The ledger total read for this revision is \$22.0055 (31 lines; \$11.0472 of it added in 3.6.0, \$1.2813 in 3.7.0, \$0.5160 in 3.8.0, \$1.5220 in 3.9.0 for the A18 Lean generation and \$2.7254 in 3.10.0 for F and A22; Section 11), so H13 is not refuted on an interim basis. Three caveats apply:

1. The GPU stage launcher is designed to refuse a stage that would push the ledger over its cap, so H13 was expected to hold largely by construction (red-team prior 0.75, "close to tautological").
2. The TPU runs recorded up to v3.3.1 were launched by separate scripts with a run-time limit but no ledger check, so the low total reflects low usage. Since commit `39a5a0a`, the committed driver `deploy/tpu/drive.sh` is a dry run by default. Before it creates anything, it checks the ledger total plus the run's worst case against a cap. The driver log of the second E generation run shows this check: ledger \$1.9809, worst case \$2.5, cap \$45 (`egen/drive_e_gen2.log` in the session scratchpad; not committed and not in the lake). I did not see the driver log of the first E run, so I do not claim that run was checked. I also

Table 54: Deviations D66–D77 (full text in `docs/DEVIATIONS.md`). D66–D68 were recorded with the A18 scoring and D69–D73 with the F scoring and are shown here because their results first appear in this revision; D74–D77 are added with it.

#	Registered / before	Done here	When
D66	A18 tiers 2B, 4B, 9B; $k = 8$ draws with a per-request seed	0.8B added (shares the chip run); engine-level seed because vLLM-TPU rejects per-request seeds; the sampling still produced no rows, so no any-of-8 result	before any Lean output
D67	Lean checks at the gate's default 120 s per stage	first pass timed out on 4, 10, 22, 18 greedy checks under load 10–17; re-scored at 300 s; the second pass is reported	after seeing timeouts, before reading proof rates
D68	A18 reports proofs verified	exact versus definitional match reported; binder annotations not compared; external red-team review not completed; hand check of 10 proofs (9 VERIFIED, 1 timeout)	during scoring
D69	F scored once	Python and math scored with the E′ recipe under load; every non-passing row re-run serially (1 flip per tier); passing rows not re-run, so unstable passes are not excluded	after generation
D70	A12 "without warm-up" = first 192 tasks	on F that drops the first answer chunk of the Python file and the first program chunk of the math file (384 of 908 tasks, a domain-unbalanced subset); only cost ratios are read from it	at analysis
D71	A21 conventions (pooled math, bootstrap, supported = CI only)	pooled math with and without competition shown; cluster bootstrap within domain; "supported" also requires Holm $p < 0.05$; cascade charges programs of every tier consulted; always-9B shown plain and with its program	at analysis
D72	Lean cost as A18	chunk wall time split by generated tokens, 9B × 4 chips; not comparable with Python and math seconds; F timeouts reported, not re-run	at scoring
D73	the E′ Laya launcher	F launcher with sha256 checks of the six frozen files and of the row files; label fields present as in E′, checked unread	before prediction
D74	A22 reports token-window fractions	not computed: no tokenizer in the analysis environment; the registered rule does not use them	after the result
D75	—	A18 results committed now, copied from the data disk; three scoring passes; $k = 8$ sampling never produced rows; the 3.9.0 "sampling is being re-run" is withdrawn	this revision
D76	A21 27B probe; Laya on Lean; A12 no-warm-up reading of the router	none of these ran or was computed	at analysis
D77	—	Laya runtime, PoT gate without a gold answer and llama.cpp tier integrated as opt-in system changes; descriptive checks only; not-validated list stated	after A20

did not inspect driver logs for the program-of-thought and 9B runs of 3.5.0, nor for any run of 3.6.0, 3.7.0, 3.8.0, the A18 Lean generation or the F and A22 runs of 3.10.0, so I make no claim that they were ledger-checked.

3. In either case it is not evidence about the GwenLaya system. H13 is a whole-study claim, the planned GCS ledger `runs/LEDGER.jsonl` still does not exist (read-only listing), and the verdict is interim.

H15 (preemption). `tests/test_run_study.py::test_resume_after_kill_mid_run` passes, but it simulates a death by appending a torn line. It sends no SIGKILL, so the probability stays at the prior.

Table 55: Hypothesis status after revision 3.10.0 (H1 and H3 as in 3.6.0 for E′, with the Rust result of 3.7.0 and the F replication of 3.10.0; no status changed in 3.8.0 or 3.9.0, and the F result repeats the E′ pattern). "Prior" = red-team prior that the hypothesis would be supported, written before any data; not updated by evidence.

H	Role	Status	P(true)	Basis
H1	primary	**supported on E′ (math, Python), deviated test (3.6.0), and replicated on F (Python, math; 3.10.0); NOT supported on Rust R′ (3.7.0)**	0.45 (prior)	**F (908 tasks, frozen Laya, same A13b metric): 9B ΔAURC** −0.104 [−0.132, −0.078]**, Holm** $p =$ 0.0002**; also 4B and 2B. Within Python alone the 9B interval includes zero (**−0.036 [−0.092, +0.019]**). R′ (69 Rust tasks): 9B ΔAURC** −0.070 [−0.180, +0.041]**, Holm** $p = 0.20$**; not supported on any tier; underpowered, not evidence of no effect.** E′ (828 tasks), Laya frozen before generation; 9B ΔAURC vs raw log-prob −0.078 [−0.106, −0.053], Holm $p =$ 0.0002; also 4B, 2B. Metric is AURC, not registered CWR at matched coverage. Carries D48, D49, D50 (failed shuffled-label control, withdrawn). A4 and A10 remain exploratory analogues
H3	primary	**not supported** on E′ (3.6.0), on R′ (3.7.0) and on F (3.10.0)	0.50 (prior)	**F (908 tasks): 9B** −0.002 [−0.022, +0.016]**, Holm** $p = 0.43$**; not supported on any tier. R′: difference exactly 0 on every tier (Laya's top-**k** is the gate-verified set).** E′: Δ selective risk Laya − gate at the gate's coverage −0.012 [−0.023, +0.011] (9B); CIs include zero on all tiers. Registered cost-per-correct metric not used; D48–D50 apply. A5, A10 and A12 are not H3. The router cost ratio (0.79) is exploratory
H4	secondary	not run	0.15 (prior)	no Laya head; A2 is raw confidence only; A11 is a cross-fitted logistic score, not the Laya head
H5	secondary	not run	0.25 (prior)	no Laya head (A11 does not test it)
H6	secondary	not run	0.40 (prior)	no Laya calibrator (A11 is an exploratory cross-fitted substitute on burned E)
H8	secondary	not run	0.30 (prior)	needs GPU (D11); A7 is not a quantisation test
H9	secondary	deferred	0.25 (prior)	D8
H13	systems	not refuted (interim)	0.75 (prior)	ledger $22.0055 of $50 (list-price estimate; 31 lines); not evidence about the system, Sec. 8
H14	systems	not run	0.50 (prior)	needs E1 (D12)
H15	systems	not run	0.70 (prior)	offline test is not the registered SIGKILL check
H2	exploratory	not run	0.08 (prior)	needs GL and B5
H7	exploratory	not run	0.10 (prior)	registered arms absent; F: 4B/9B Python gate precision 0.709/0.729 and math 0.977/0.991 (Table 32); descriptive analogue A3/A10 (4B gate, TPU) has precision 0.830 Python, 0.887 Rust, 0.943 math (math CI includes 0.95), point estimates below the 0.95 flag; under the exploratory

9 TPU feasibility findings

- **Qwen2.5 works.** In smoke run #1 (on demand, us-central1-a; logs under `gs://socrateai-datalake-gen-lang-client-0625573011/gwenlaya_v4/tpu_smoke/gwenlaya-tpu-smoke-215529/`), `Qwen/Qwen2.5-Coder-1.5B-Instruct` loaded and generated with vLLM 0.31.0's TPU backend: load 208.9 s, then 442 completion tokens in 0.61 s. The gate on that VM returned UNVERIFIED (no bubblewrap there), which is the designed fail-closed behaviour.
- **Qwen3.5 0.8B, 2B and 4B generate on one v5e chip with lazy compilation.** In run #1, `Qwen/Qwen3.5-4B` did not finish loading within 1,500 s. Qwen3.5 is not JAX-native in tpu-inference and runs through the PyTorch fallback. It runs when up-front precompilation is skipped (`SKIP_JAX_PRECOMPILE=1`). With that setting:
  - Qwen3.5-0.8B generated 16 night tasks in run #2 (`results/gwenlaya_v4/tpu_run2/res_qwen35_0p8b.jsonl.summary.json`). The run #2 scores are not used, because its TPU-host gate self-test reported `python_ok False`.
  - Qwen3.5-2B and 4B generated all 80 night tasks in run #3.
  - The same two models generated all 1,536 E tasks in this revision.
- **Concurrency limit for the 4B.** On the 16 GiB chip the 4B (hybrid full/linear attention; vLLM sizes a "mamba" recurrent-state pool) failed to start with `max_num_seqs` 96 ("Cannot fit both KV pools"). It started at 32, where the engine re-split the KV cache for 11 concurrent requests. The 2B ran at 96 configured and 54 effective (Section 6).
- **Log-probabilities need `logprobs=1`.** The first E pass returned none (Section 6). The committed generator requests `logprobs=1` and fails fast if none come back.
- **No TPU LoRA path was found for Qwen3.5 dense or Qwen3.8** in the stacks we inspected. Serving Qwen2.5 with `enable_lora` failed at engine initialisation, and this is the only vLLM-TPU LoRA attempt. Tunix 0.1.7 lists Qwen2/Qwen3; MaxText lists Qwen2.5/Qwen3 and Qwen3.5 mixture-of-experts scripts. (3.5.0 concluded that the Laya head and SFT would therefore stay on the GPU/CPU plan; in 3.6.0 the Laya head was in fact trained on the TPU with torch_xla and peft, Section 7; a torch_xla and peft LoRA probe of Qwen3.5-0.8B finished no step in 25 minutes.)

10 Descriptive TPU run #3 (history; not a registered arm)

What ran. TPU run #3 used one on-demand TPU v5e chip (us-west4-a), vLLM-TPU 0.31.0, bf16 weights, greedy decoding, `max_tokens` 1024 and non-thinking mode, with the D28 math prompt and TPU-specific code system prompts (via `llm.chat`, with no code prefill). It generated the 80 night tasks for three models (`results/gwenlaya_v4/tpu_run3/`). Scoring on the TPU host failed for every Python item, for a reason that was not recorded (the gate self-test on the same host passed). All rows were therefore re-scored on our machine.

Reading (corrected wording). Qwen3.5-2B in bf16 under vLLM on TPU, with a different code system prompt and no code prefill, gives the same Python and Rust pass counts as the q4 CPU arm. Item agreement is 22 of 28 for Python and 24 of 26 for Rust. Hardware, quantisation, engine and code prompt all changed, so this is a loose consistency check, not an isolation of any one factor. The same-protocol comparison A7 (Section 6.5) supersedes it.

Table 56: Descriptive TPU run #3 (one v5e chip, bf16, greedy, one sample, chat prompts; not a registered arm, not a test). Items passing the checks out of 28 Python, 26 Rust and 26 math items; "trunc." = math items cut at 1,024 tokens (all UNVERIFIED). Load, generation time, tokens and tok/s are the run's own self-reported `SUMMARY` fields.

Model	Python	Rust	Math	trunc.	load s	gen. s	tokens	tok/s
Qwen3.5-2B	11/28	6/26	16/26	9	106.2	99.7	25,568	256.3
Qwen3.5-4B	20/28	13/26	16/26	9	124.3	117.4	25,852	220.2
Qwen2.5-Coder-1.5B-Instruct	18/28	15/26	8/26	4	186.3	8.7	18,219	2,094.2

11 Actual GCP spend

The figures below are read from `/mnt/data/home/xavkal/gwaya-data/spend_ledger.jsonl`. The ledger was last written at 2026-10-11 15:54:04 when it was read by `scripts/make_tables_3100.py` for this revision on 2026-10-11 (`spend_summary_3100.json`; the values below are macros written by that script, so they show the ledger as of the build). **Revision 3.10.0:** the ledger has 31 lines and the sum of `usd_estimate` is **22.0055**, 44.0% of the $50 cap. The 4 lines after the 27th, `tpu_f-9b-185104` (v5litepod-4, 1,016 s, $1.3547), `tpu_f-small-185104` (v5litepod-1, 2,139 s, $0.7130), `tpu_f-laya-152622` (v5litepod-1, 444 s, $0.1480), `tpu_laya-layout-152621` (v5litepod-1, 1,529 s, $0.5097), total $2.7254 (Table 44): the F generation on the 9B (v5litepod-4, 2026-10-10 19:11) and on the 2B and 4B (19:28), the frozen Laya's prediction on F (2026-10-11 15:35) and the A22 candidate-first training and prediction run (15:54). Only items dated from the evening of 2026-10-10 are attributed to this revision; the two A18 Lean lines ($1.5220, 2026-10-10 07:48 and 07:59) belong to 3.9.0, where they were first counted. All scoring, re-checking, analysis, the A15.3 selection, the A19 and A20 runs, and the Lean scoring ran locally and cost nothing on GCP. **As of 3.9.0** the ledger had 27 lines and $19.2801 (the 25 lines of 3.8.0 plus the two A18 generation lines, $1.5220). **As of 3.8.0** the ledger had 25 lines, and the sum of `usd_estimate`, recomputed by `scripts/make_tables_380.py` (`spend_summary_380.json`), was **17.7581**. **Revision 3.8.0 added one line, `tpu_laya-textonly-204124` (v5litepod-1, us-west4-a, 1,548 s, $0.5160), the text-only ablation run (the ledger line was written at 2026-10-09 21:09:25).** The first 24 lines total 17.2421, as in 3.7.x. Revision 3.6.0 showed 21 lines and 15.9608; **the 3 lines added by revision 3.7.0 total 1.2813** (Table 52: the R′ generations and the Laya prediction on R′). Revision 3.5.0 showed 11 lines and 4.9136 (the first 11 lines, as listed below); **the 10 lines added by revision 3.6.0 total 11.0472** and are listed in Table 50. Revision 3.4.0 showed 8 lines and 2.5289; the three lines it lacked are the last three rows below, totalling 2.3847. The tables below are those of 3.5.0 and are unchanged.

Time	Item	Seconds	USD
2026-10-07 21:54	TPU smoke, attempt 1 (spot, us-west4-a; preempted)	240	0.0329
2026-10-07 22:38	TPU smoke (on demand, us-central1-a)	2,466	0.8220
2026-10-08 00:35	L1 upload to data lake (6 objects)	13	0.0
2026-10-08 07:09	TPU run #2 (on demand, us-central1-a)	1,073	0.3577
2026-10-08 07:36	TPU run #3 (on demand, us-west4-a)	982	0.3273
2026-10-08 21:36	E generation, first pass (on demand, us-west4-a; no log-probabilities, set aside)	963	0.3210
2026-10-08 21:44	**orphan VM**: created 21:38:23 by a driver the maintainer killed by mistake, deleted 21:44:35 (us-central1-a)	360	0.1200
2026-10-08 22:20	E generation, second pass, used here (on demand, us-west4-a)	1,644	0.5480
2026-10-09 08:47	program-of-thought generation, 2B and 4B (on demand, us-west4-a, v5litepod-1)	990	0.3300
2026-10-09 09:07	9B smoke test (on demand, us-west4-a, **v5litepod-4**), 18 tasks, not analysed	587	0.7827
2026-10-09 09:37	9B full run (on demand, us-west4-a, **v5litepod-4**)	954	1.2720
	Total (sum of `usd_estimate`)		**4.9136**

All non-zero spend is TPU use, priced from Cloud Billing Catalog list prices: spot v5e $0.494237 and on-demand v5e $1.20 per chip-hour, as recorded in each line's `price_source`. All lines are v5litepod-1 (one chip) except the two 9B lines, which are v5litepod-4 (four chips, so seconds $\times$ 4 $\times$ 1.20/3600). The two 9B lines together cost $2.0547, about $2.05. In 3.6.0 the largest single lines are the 27B attempt ($3.284, no result), the pool-P 9B generation ($1.664), the repeated Laya training ($1.2923) and the E$'$ 9B generation ($1.3333); the lost first training run is $0.94. The cap is $50 hard (H13); the total, 22.0055, is 44.0% of it (17.7581, 35.5%, as of 3.8.0; 19.2801, 38.6%, as of 3.9.0). The 27B attempt and the generator-LoRA probe (ledger item `tpu_laya-probe-114619`, $0.6253, if that item is the probe; I did not verify which probe it covers) produced no usable result.

- The ablation analysis (bootstrap, tables) ran locally and cost nothing on GCP; only its training and prediction run is on the ledger.
- The three E-generation lines (first pass, orphan VM, second pass) total $0.989. About $0.44 of that (first pass plus orphan) produced nothing used here.
- The orphan line is an upper bound, since billing may start only when the VM is ready.
- The billed amount is TBD until the billing export is read.
- All scoring, re-checking and analysis ran locally and cost nothing on GCP. This includes the A15 edition re-check and depth dial of 3.7.0: the ledger was re-read after them (24 lines, unchanged). In 3.10.0 the same holds for the F scoring and analysis, the A15.3 selection and the Lean scoring.
- The list-price estimates of every ledger item, including the four newest, are not the billed amount (TBD until the billing export is read).

12 Reproducibility

Code.

- **Earlier releases.** v3.2.0 = `825d1ba`. Tag `v3.3.0` points to commit `200c7f4`, the merge of the night branch that contains `ebd032f`. v3.3.1 = `4914b5c` on `main`, with the Rust fix PR #4 (`8cb37ad`) and the math/Python fixes `2c1f6b1`.

- **This revision** is on branch `bench/tpu-e-eval`:
  - `39a5a0a`: TPU batch generation, importer, capped driver, the earlier TPU and validation scripts, and the claim audit.
  - `66e2313`: memoised and optionally parallel checks.
  - `8ee9da3`: analysis plan.
  - `5e10ad2`: analysis script.
  - `40dcae5`, `5ef8e43`: overlay, A7 restriction, S1 and the plan amendments.
  - `f80ac16`: results.
- **Revision 3.5.0** is on branch `feat/math-gate` (HEAD `227eaf8` when this text was written; the paper edits of 3.5.0 are not yet committed):
  - `803f350`: program-of-thought math gate, charged for its generation (A10).
  - `4af1fe9`, `fa8826b`: plan addendum A11 (written before fitting) and its analysis script.
  - `cb5c6b5`: chips-aware chip-seconds for multi-chip slices; plan addendum A12 written before results.
  - `227eaf8`: A10, A11 and A12 results.
  - `scripts/make_tables_350.py` writes `papers/tables_a10_a12_part1.tex` to `part3.tex` from the three result JSON files (not committed yet).
- **Revision 3.6.0** is on branch `feat/laya` (HEAD `8af8776` when this text was written; the paper edits of 3.6.0, `scripts/make_tables_360.py`, its outputs (`papers/tables_laya360*.tex`, `results/gwenlaya_v4/laya/controls_C.json`, `spend_summary_360.json`) and `deploy/tpu/run_laya_predict.sh` are not committed):
  - `20219bc`: plan addendum A13, written before any of it was generated.
  - `d470de3`: pool-P builder, Laya data builder, `train_laya.py` on TPU, generator-LoRA probe.
  - `65ce98f`, `b712e08`: plan addendum A13b and `analyze_laya.py` with tests.
  - `d217f1e` (2026-10-09 16:39:32 +0200): freeze file, isotonic map, D48–D52, committed before E′ generation.
  - `8af8776` (17:37:55): results on E′, router policy, controls, twin.
- **Revision 3.7.0** is on branch `feat/rust-fresh`; the paper edits of 3.7.0, `scripts/make_tables_370.py`, its outputs (`papers/tables_rfresh370*.tex`, `results/gwenlaya_v4/laya/rfresh_gate_laya_overlap.json`, `spend_summary_370.json`) and the D53–D58 text of `docs/DEVIATIONS.md` are not committed when this text was written (the A15 commits below are committed):
  - `0f47b15` (2026-10-09 18:25:26 +0200): plan addendum A14, `scripts/data/build_rust_pool.py`, `tests/test_build_rust_pool.py`, `rust_fresh_report.json`; written before any model generated for R′.
  - `ecdd44e` (18:25:36): task-file hash in A14.
  - `790e224` (18:53:27): results on R′ (`laya_rfresh.json`, predictions, rows, `scripts/rust_fresh_summary.py`).
  - `f0f514b` (18:57:22): oracle edition option, `scripts/rust_edition_sensitivity.py`, plan addendum A15, written before the A15 results. `8d3323f` (19:14:39): oracle default 2021 (D58), edition results, depth-dial script. `4fe5405` (19:21:28): depth-dial results.

- **Revision 3.9.0** is on branch `feat/py-depth`; the paper edits, `scripts/make_tables_390.py`, its test and outputs, the D63–D65 text and the A17 erratum are in the revision commit. Sources, all committed before it:
  - `6d229b1` (2026-10-09 21:52:55 +0200): plan addendum A17; `499b9eb` (21:54:18): `scripts/python_gate_depth.py` and tests; `f19e283` (22:53:02): A17 results.
  - `da7d708` (22:53:02): gate evidence strength and opt-in `min_visible_tests`; `dfd41aa` (23:57:40): verdict policy, Rust count fix, exploratory policy count.
  - `aa572bb`, `d97e390`: Lean gate repairs; `5b53ce8`: plan addendum A18; `cc3969e`, `ff97b0b`: A18 wiring and launcher; `16a7abd`: A18 sanity; `f318f71` (2026-10-10 00:33:22): A18 amendment; `24bc5c0` (01:01:28): hardened Lean gate; `302ab95`: engine-level seed for the A18 sampling (generation in progress).
  - `f837c9f`, `2305d90`, `5e14bd8`, merged in `becb57f` (01:18:58): D62 scorer normalization and exploratory re-score.
- **Revision 3.10.0** is on branch `release/3.10.0` (from `b98a627`); the paper edits, `scripts/make_tables_3100.py` and its outputs, the A18 test-split result files, D74–D77 and the A21/A22 text are in this revision's commits. Sources, all committed before it:
  - A15.3: `f4af1e4` (2026-10-10 07:35:49, plan), `e556339` (07:38:01), `b2920c8` (08:20:49, results).
  - A18/A21 Lean: `feb03b6` (13:30:13, D66–D68, scorer options), `046ac08` (16:01:02, match-type fix); F Lean scoring in `99bcc55` (2026-10-11 14:30:58).
  - Runtime: `8626ed4`, `4f9d2d2` (13:53:52, PoT gate), `165fd37`, `d22c193` (14:00:19, Laya runtime and parity), `26bb89d`, `ae7e7b2` (14:14:32, llama.cpp tier and smoke), A19 `1bb7543` (14:18:21) and `73d1a15` (14:41:33), A20 `cca2049` (15:01:14) and `294054e` (15:04:15).
  - A21: builder `d06bb18`, `cdc5e6a`; task files `219178d` (17:47:33); plan `d60bdc3` (17:48:53); launcher `fc01629` (17:49:58); scoring `c797a27`, `e83778c` (22:56:38), `0b21edc` (23:27:45); Laya rows `99bcc55`; Laya result `0683fc5` (2026-10-11 15:47:28).
  - A22: plan `2a76e2f` (2026-10-10 15:43:58); code `3134a94`, `a22f173`, `2e41bcb` (15:52); result `b98a627` (2026-10-11 15:59:42).
- **Night CPU arm.** llama.cpp release b11476 (CPU, asset sha256 `2cda5ff9...4b5e`). Qwen3.5-2B-Q4_K_M.gguf from `unsloth/Qwen3.5-2B-GGUF` revision `f6d5376b...`, sha256 `aaf42c8b...9223`; 4B revision `e87f1764...`, sha256 `00fe7986...11a4` (all in `L0_env.json`).

Frozen inputs and results (sha256, all recomputed for this revision).

File	sha256
`experiments/night/manifest_eval.json`	`bd1684dadcf38328fb6fab7619e5ea2f4fbc5ccc0e31022e059360b2d8c615e0`
`tasks_E_night.jsonl` (80 items)	`e842edd13951b4082c4495f7bcc845b032989f4a457e0c5d6e9c44f2b72cd679`
`tasks_E_primary.jsonl` (1,541 items)	`971f44c61bad5044f8487383f8a3ea27bc2c292c565a6ff8ef1254fce1cc7097`
`docs/FREEZE_ADDENDUM_NIGHT_2026-10-07.md`	`ac2970983ef45eb027de65194d42cadd41483dfd35cd2a866efc64247b02376c`
`experiments / night / decontam _ report.json`	`d128e06c5949c86c29537f71a1d547ef953be6aa22f7bc814a280ec4d48f1907`
`night/cache/L2score/rows.jsonl`	`1e35fac792d29d8f82ce05f5cbad0d615bb4db2a6a707d157edff75da0fa69c6`

night/cache/L2score/gens.jsonl (= L2fix)	03f6e2b6af9cebd65d8488b9a35a1f7d3bc70b566397379edc3d95f5291963db
papers/numbers_v4.json at tag v3.3.0	fccb1695538e76eba93669d1a232d24c7373e8aa2ad37d900f0e19c153275995
night / cache / L2fix / rows . jsonl (v3.3.1 re-score)	a5e72614681a81acf379050270677eead4844e4833494c32f7695c485cae70a8
papers/numbers_v4.json at tag v3.3.1 and HEAD	5f3c2a6e1e069663af306f67f4e2ba0e32155044908cba35fd5826f092d8a1c0
results/gwenlaya_v4/rust_harness_validation.json	93291a5a56c050d15fbb98a501e16d575251602e30507e563ff610f24583e8d8
results / gwenlaya _ v4 / tpu _ run3 / summary.json	e38310f6b4e7869c0f84a3cfad874e0820cd77a29f81e173f6e4c0cc80f94648
tasks_E_night.d25.jsonl (80, D25)	504909fc74dc2af8fb3f2491a475df07de8e95e6fdf0f4f69bc22352e95e0952
tasks _ E _ primary . d25 . jsonl (1,536, D25)	2143110c78a3a9f57599dce77021cf4571fc57cf2584017f16ebdd08d5acba66
docs / ANALYSIS _ PLAN _ E _ TPU . md at 8ee9da3 (original; living file since modified)	1f2fa2c11cf5fa52f12c5864f63a61b9b4ddd1ad8086a1946105f9b046020d46
results / gwenlaya _ v4 / e _ tpu / numbers _ e _ tpu . json (regenerated after the re-audit; working tree)	9197f367f53d28efce0aa6c7de01dd83e67c786a20c4232fc35b07d6171547bc
numbers_e_tpu.json as committed in f80ac16 (superseded)	c76bc506c916c8b78286f4fdfefabc271e3eb58a1fdb0a3d211d4a965e32226b
papers/tables_e_tpu.tex (regenerated)	1aa86421460c261abbfc5e12171975de4497e032d4d6a83731b0516c6aaa58eb
e_tpu/data/rows_tiers.jsonl (= etpu/tiers/rows.jsonl)	b1bf6df59f05a03f771ac4cf667d74da90f6889f3129dfb4d750df29a1406277
e_tpu/data/rows_coder.jsonl	6898d5ae60c20fd1a1defe5be2bb0b5337b4832f9710074bc707b8fc1bae82a9
e_tpu/data/gens_summary.jsonl	46d02306bc84ad9749f384aaaf7b6cb09f0f96ffe503abd9ce8bcb0d9e41c98c
e_tpu/overlay/rows_overlay_tiers.jsonl	54e9ff405358fd7d090e0eba0723f156c6a168304ab326f2e2729ac04b0a94b4
etpu/tiers/gens.jsonl (data disk)	b6362df66cc8b7ffd5177a7eeb8866cf8f0a7d129d70074e4daac46783875fce
raw_qwen3.5-2b-bf16.jsonl (lake, second pass)	e077aa1da6a36e7a0b0de8748e720de5315c0c836dff0c90e307c4e51a201960
raw_qwen3.5-4b-bf16.jsonl (lake, second pass)	d2e1847fff7b19be35e2f20a6fefe246e3bd66dec55dc08b9bdf51e6671970ab
raw_qwen2.5-coder-1.5b-bf16.jsonl (lake, second pass)	df6ee0d466195023526e126abb862d73cb45b2dd969ba8d3215fef24d4ef3c42

Revision 3.5.0 (all recomputed with `sha256sum` on 2026-10-09)

results / gwenlaya _ v4 / e _ tpu _ mathgate/numbers_e_tpu_mathgate.json	ce662f842e64058687f659121fec2979264a587aba68448710d73eebe69d9a1b
results / gwenlaya _ v4 / e _ tpu _ mathgate/abstention_a11.json	9ee7d4dd0da18fb1d10aeb8c7fc4772c186de2d10d05a6e4450b5ebf321fc44c
results / gwenlaya _ v4 / e _ tpu _ mathgate/cascades_a12.json	5d331a630027ab874e237f9e3cbfe387a79ad862a9cfb4ed90a31c47d24e59c8
results / gwenlaya _ v4 / e _ tpu _ mathgate / tables _ e _ tpu _ mathgate . tex (generated, not input)	163944211ac263795b86ba4b378d01128f6bd6f17ff7e39a360d616750c6d8d0
results / gwenlaya _ v4 / e _ tpu _ mathgate/rows_mathgate_2b4b.jsonl (= etpu/mathgate/rows.jsonl)	2a27e0926956d4b4152cd39d4ad22f63641fb3dfb760896335a6018881f0d9de
results / gwenlaya _ v4 / e _ tpu _ mathgate / rows _ 9b . jsonl (= etpu/nine/rows.jsonl)	5303da0da3ecda8259a82b3529980c7300803341968ab39f4ece110df54aa7b2
results / gwenlaya _ v4 / e _ tpu _ mathgate / figures / e _ risk _ coverage . png (= papers/figures_v4/e_risk_coverage_a10.png)	75fce2e39433195424ff844f49f791e6870827b4356cecef52cd9b58ff6c7ff9

results / gwenlaya _ v4 / e _ tpu _ mathgate / figures / e _ cascade _ pareto . png (= papers/figures_v4/e_cascade_pareto_a10.png)	7a3f142a9989afa167e3628a7c2988267b2abd57671317f6ee4a09a59554d19b
gwaya/domains/math_gate.py	6fe1d5a3c2035a355c24b572a8937d2321dee57c8e4a3fe5a1e3db628c58c032
scripts/analyze_abstention.py	900dd40b6c0ce277f1c1f11fa5a457f88be4319730f12cbde87137fc1848788d
scripts/analyze_cascades.py	4e137d9f4fcd65b5477c2668b8d5f9ceb4742d4d179179107d115397bb26f731
scripts/make_tables_350.py	bba450ffeaffea6f90dc3da63d56f3dabab0d9e75bba95b0d5b65ee292433244
papers/tables_a10_a12_part1.tex	4522326e346ae5271b1fb6c1dc42318d76100a13adbf9a1eac781c1d691d27df
papers/tables_a10_a12_part2.tex	9949d94bc2b345465e3f6a2581a71a63fab0bc4c165722443c0211bb3e68cfb6
papers/tables_a10_a12_part3.tex	3e544f5bd3a0e59ad7e63f511253460ddd7ac8bba6727f4575d3d2b4bc5efa02
docs/ANALYSIS_PLAN_E_TPU.md with addenda A10–A12 (at cb5c6b5; as of 3.5.0, living file since modified; the A1–A9 text is unchanged)	92a3f85ce11518b8ba2f32c881d51bfe319ffed738a4703d2f89c4a27c135ce9
docs/DEVIATIONS.md with D43–D47 (working tree; as of 3.5.0, living file since modified)	dac58d8ddddb78fa6182699befd991990d838020009172a3440bbfd44fae88aa
spend_ledger.jsonl (11 lines, read for 3.5.0; a living file, now 25 lines)	ff882615f769fd366afc6c10d3d2f7ed4e52ffc60025ed9b19aa24143943c2d4
etpu/nine/gens.jsonl (data disk)	c01e317819425ff70d3101ec935d8fd04f9bebeecdb6e8ae8105b71e8866e3c6
etpu/mathgate/gens.jsonl (data disk)	3eef7582c6a4c7da1047d0e033ce59ade2e02c209606c50719dffbdebf6d4661
etpu/nine_raw/raw_qwen3.5-9b-bf16.jsonl	2df7659949a3247be99d45471eebe1f7d4610c7d6228618fd4bfdcba9f3edeab
etpu/nine_raw/raw_pot_qwen3.5-9b-bf16.jsonl	39c3f51533dce6802826124212e3d3c9a2e2bacf69cab68edf20a5c989f8e64f
etpu/pot_raw/raw_pot_qwen3.5-2b-bf16.jsonl	adf0156e37f5b70c26dde6723992aec12f722a97d5fb7d0fdd1e51ba385a98ce
etpu/pot_raw/raw_pot_qwen3.5-4b-bf16.jsonl	3abe365ac3a83b72bdf8ae580b84ee27e6c27973bad8f0f20f2581c93375dc26

Revision 3.6.0 (all recomputed with `sha256sum` on 2026-10-09)

docs/LAYA_FREEZE.txt	10bcc78fe4b3856dcb14220e1a47a1082d6a7df670e3905224259efbafc43111
docs/laya_isotonic_C.json	e8ee81dc930f6a795b122bef5151b10c1d507e7055822a8663fbbd62bbbdc76a
results/gwenlaya_v4/laya/laya_eprime.json	ea31b00aef43aa18737d112c3de50e93988d61560ef3037a6604834d51c886c1
results/gwenlaya_v4/laya/router_nowarm.json	899a363f747f47389bcd5cbe74dc24520349d396c0a8a5439a4799a1b3ba2f4d
results / gwenlaya _ v4 / laya / controls_C.json(generated)	a7b561e1cc49ca7f170a2cdfbd497d11a313eb91b64a41ba5a62631b6d491721
results/gwenlaya_v4/laya/spend_summary_360.json(generated)	14385dd6269acc3e26c08c145b13b00631323ddd87493a958f642d69a2ded44a
results / gwenlaya _ v4 / laya / data / pool_manifest.json	b2497d403e3ee6e74946f56db2fe5def69dfe78cf1d009f75efa754b7d7bef01
results / gwenlaya _ v4 / laya / data / preds_C_calibrator.jsonl	82e13f3a189a44359bef0d78c357a600359f79023867944b70f8828d32869fc2
results / gwenlaya _ v4 / laya / data / preds_C_router.jsonl	01cdc400b38c0d9410655b8c4cca1e2cec0e5580a6856f57528dc24cd0049900
results / gwenlaya _ v4 / laya / data / preds_Eprime_calibrator.jsonl	c99229db91b3ab5624643bd512e36bdbcc12873853fc9249cddc4a5afab726fa
results / gwenlaya _ v4 / laya / data / preds_Eprime_router.jsonl	22dc5bb087048c2186cb1946c3529f5c8c71e276e12dde55c780179f6f2c4d24
laya _ v1 / router / head _ config . json(datadisk)	02b38a4c6f8d02d869cfc530ce835bf0625eb9ea30687f68d396142c9c0ae279

laya_v1/calibrator/head_config.json(datadisk)	9d8ef42b932766009c81e9c32e463b913dbf33e2497e0a3eb0f29969fb1f67d5
scripts/data/build_pool_p.py	63caafa0ad96a00028475806391dab739214637619789cc5e01e08baeb5d3199
scripts/data/build_laya_data.py	773cba9142f2517a842edbe335fde174078d108f571c4649ac980acde057e951
scripts/train_laya.py(asof3.6.0;filesincemodifiedby--text-only; seethe3.8.0block)	6b269da846ce5e9c79bb50363858349b7206d6bc445f46645e6e6b494e604cb1
scripts/laya_freeze.py	6a91c56d0a3cc589350a31fe294792a97dedd06e32da81a3163c6299b5e94e73
scripts/analyze_laya.py	baf271410c8293624586e69b4ed368a952f16bffce88c85407a18e52942b251a
scripts/laya_router_warmup.py	fca1ff1a5a1e86c08490c6ba5342dbc53d8facf8cdd75c8954db28b2ec34426e
scripts/tpu/lora_probe.py	6f5d236d31dd6226a5442a7d887562ff9daf45d0a49a8ef59f14306beb1fabdf
deploy/tpu/run_laya_train.sh	b76624acc687e3007061c058d60af49a8a4fce380170916aee6cd30076bf6b86
deploy/tpu/run_laya_twin.sh	ce8714d4546277ab3da87600a43cf85d8d8ca3a307fa1962b363e1436cbdc093
deploy/tpu/run_laya_predict.sh(uncommitted)	5b7a9f986e25451b528dd5df1d37bc82586ca082ba8f0f3b273af9641c95e09f
tests/test_analyze_laya.py	f0c89031b09b8681e680bf8c3d073642be5c20d38fb25f99f89b748895ccae9e
scripts/make_tables_360.py	4f461332f07ead5af8b8115ccbe0141e8067cce02d5339271863f351323abc65
papers/tables_laya360.tex	746242667e9bc074d3b66ceab731d89d4b4321b4cf2754e0b6170041db193860
papers/tables_laya360_spend.tex	c0439e5b8b029309e5fe29348dd3ae3945e5612d862866f4ef08dac8e9c063ed
docs/ANALYSIS_PLAN_E_TPU.mdwithA13,A13b (as of 3.6.0; file since modified, see the 3.7.x block)	bd6e84a6cdce09654f6d248502d96ed052fea427ab845bf496037ad92542f9c3
docs/DEVIATIONS.mdwithD48--D52 (as of 3.6.0; file since modified, see the 3.7.x block)	0518779a8852e148c296c8e6ce3bf346f28878c4fd0506f9efacbefd57588713
spend_ledger.jsonl(21lines, asof3.6.0;alivingfile,nowlonger, seethe3.7.xblock)	f9c527663fed636336e61582dd8eefacef266404561006c62d12f676c399dbdb

Revisions 3.7.0 and 3.7.1 (all recomputed with `sha256sum` on 2026-10-09, after the last edit of this text)

scripts/data/build_rust_pool.py	f6d879f51f4f7d10415678f2d8c68cf20fe10d442d5c3e55f588c694815600b5
tests/test_build_rust_pool.py	59e1abb141250c011793ee841d47cb470894adab51a27a40aa458f43888a6d09
scripts/rust_fresh_summary.py	8440cf7e7cdff6e29eb2ecec1ccf8244c61cc0267cc7b06005022ddbdcb3a2e8
scripts/analyze_laya.py	baf271410c8293624586e69b4ed368a952f16bffce88c85407a18e52942b251a
results/gwenlaya_v4/laya/laya_rfresh.json	f05e1102d1a7b2aea6272fa60cc2b4174e48047d3ebc4a7abcfd989cfdf02883
results/gwenlaya_v4/laya/rust_fresh_summary.json	474f81bfbe630a7faab2b1ce0e032aa827c18a62dd265c701354ad2600a5a46d
results/gwenlaya_v4/laya/rfresh_gate_laya_overlap.json	868a02ddaf84e3e7a2ff1d3eeacc979cc984cf1d2f86bbf537f8b7788c69a222
results/gwenlaya_v4/laya/spend_summary_370.json	61872f3516cd6229b432be4a45d9638a2f90449f24a687e98b2b81e54a009263
results/gwenlaya_v4/laya/isotonic_C_recomputed_rust.json	e8ee81dc930f6a795b122bef5151b10c1d507e7055822a8663fbbd62bbbdc76a
results/gwenlaya_v4/laya/data/tasks_Rust_fresh.jsonl	4d6d03e5a03c409b64c129b37689dbb2091cfd8d9929b47bb9c300e1aa3f3bf3
results/gwenlaya_v4/laya/data/rust_fresh_report.json	a46691356ac80dc6460e9ca14c77698f767fa0e3495cba64d7112480ab90be25

results / gwenlaya _ v4 / laya / data / build_counts_Rfresh.json	038bd07026a47aa480d6cf660449467d65479c144d77d6dd05e90b4014fc38a8
results / gwenlaya _ v4 / laya / data / preds_Rfresh_calibrator.jsonl	44fe1a83701a2aa674c4477f827cfdbe47b6ebb3de6f2191c813dfc7ab5129c9
results / gwenlaya _ v4 / laya / data / preds_Rfresh_router.jsonl	353bd71c20f9ff5a9a2b3e81b16c45284b9c0d8f78ac0ee2ddcaef4f509f3d16
results / gwenlaya _ v4 / laya / data / rows_Rfresh_qwen3.5-2b-bf16.jsonl	568311cb9c5d93839753072bfe7bccb67fe07de9c496bff1fac83c78d0392146
results / gwenlaya _ v4 / laya / data / rows_Rfresh_qwen3.5-4b-bf16.jsonl	8ecca4b93582844116ae4636b1df2b02e40235cf9df10c78a97f568b18132013
results / gwenlaya _ v4 / laya / data / rows_Rfresh_qwen3.5-9b-bf16.jsonl	5728454201aed9e08ab68216235d484a220e742b2b711574549310cc766255a6
results/gwenlaya_v4/edition/rust_edition_sensitivity.json	205da5ea5429e5c07307f41895a2c1669fc739a0eaddf893ff57c81a2122d12a
results/gwenlaya_v4/edition/rust_gate_depth.json	6416c615a6594fd8b65bb47b2c97411924a5a3753f90aea9c92151b1049f28be
scripts/rust_edition_sensitivity.py	68ec9907b0f76daadb4d053e665b02ac4f18c092a487eb9a83e953a5317e76b6
scripts/rust_gate_depth.py	ec7960bdf6e775f10fc7c749029cb52ad62a9b23beb0ce52255ec22c25e27400
tests/test_rust_gate_depth.py	e645602f7417876a7c4839abdde2a7ee9858a6550e3607591838695e32c907f2
gwaya/oracles.py	a9aafb50f0dfb1c730bf007ce7503603a9b65a8da2f5efad275c72606deaa70b
tests / test _ rust _ harness _ hardening.py	5bce1fed39f5ff68fe12b6e40b84ea1716090319cb8f504870ac786bb36214b5
scripts/make_tables_370.py	78795bafd5f1bca820437b71f99f1fcd07d2852d362fe1656ac1a7303d2e2812
papers/tables_rfresh370.tex	7973bf250f1f27cb53ee966e3e9dea3ebaf84f879d96023f27593c8a3ef3601d
papers/tables_rfresh370_spend.tex	34ab389c20c766a91ef7cf6f3823c6de1d582294ce082a43164384165946a089
docs / ANALYSIS _ PLAN _ E _ TPU . md(asof3 . 7 . x ; livingfile , A16addedsince)	ad031a54cab540239711b23bede33eed316a2922e69887a056d7b2747fa997b3
docs / DEVIATIONS . md(asof3 . 7 . x ; livingfile,D59--D61addedsince)	6d7603ba83a8fbde8885eca9de17d8286f10444b4eeb5bf2875caf32a6a8a911
docs/CLAIM_AUDIT_370.md	6c65947aaf8491d613bd12baad5e1c69395a77737908df2c59f2ecda86780722
spend_ledger.jsonl(24lines,asof3. 7 . x ; alivingfile , now25lines , seethe3.8.0block)	b9de2946df1980c22dbf432fbdce10708bcd703f9d46d3e6c6633e31334887f6

Revision 3.8.0 (all recomputed with `sha256sum` *on 2026-10-09, after the last edit of this text)*

scripts/train_laya.py	4d8d1f2714f9bfc439f1c0d17408d40a3bb3361cd3427934ea42b49ebcf67187
deploy/tpu/run_laya_textonly.sh	30a134b7388ae67d4caacead4e6de46407edbad442cab7f7b4770fe81e828866
scripts/analyze_textonly.py	8a73f037517085d074e1e1d4f1982fbd99a8535e4d03385ac94d5dda6a352f86
tests/test_train_laya_textonly.py	6fd36d1c6c24b109d7201be53016a56dbd9435b29974ec489f26944294a020f3
tests/test_analyze_textonly.py	00aea7a8103b67eca4953e5a372f23ca0f537b61bc32977a0a7e2a8c1399f88c
results / gwenlaya _ v4 / laya / textonly_a16.json	109f1c31c24d438d8eaac447ae5d36f98ada588802f372cc673b7bc9d5165465
results / gwenlaya _ v4 / laya / data / preds_textonly_calib.jsonl	298d47d5caec4df6afc384a7b01b34b3388bd9c400130e845efcd588c935cfde
results / gwenlaya _ v4 / laya / data / preds_textonly_eprime.jsonl	7139feff7c7e9b5938fcf21243d77384acd0c123d70d2acf37c8b910362fc655
results / gwenlaya _ v4 / laya / data / preds_textonly_rfresh.jsonl	63028b8ce368dc539829597084c9fe12e8c4c60bb1ac1540c4d45968a164be2f
results / gwenlaya _ v4 / laya / data / run_manifest_textonly.json	4687623e9b3f70808fd0b4393275df4c3a284ba874f4a13020d45416ffea8c3c

results / gwenlaya _ v4 / laya / data / head_config_textonly.json	ba4c94df1c73f00f5777962a59e10e5382246497272165faf121e79c10d407a7
results / gwenlaya _ v4 / laya / data / run_manifest_calibrator.json	fc08588d23f87f1e21f1d21ca88f0000dca3a38141c5624bc90c08570a9996a3
results / gwenlaya _ v4 / laya / spend _ summary_380.json	e7e5b43af3543fd232b1c5c7f78536b72962c90e74650d2ac23ddda93b07b242
scripts/make_tables_380.py	6b99793a1e1b9196efc630ae25e5ebcfc82c8a23a86d6d1f5aed9a526f45e8f7
papers/tables_textonly380.tex	676fb4ee11a05fd3dca5b082dca4463029dfaeefd69aa039ecb5597f691b4e27
docs/ANALYSIS_PLAN_E_TPU.md	1a5a649130b1b93e9c63a06759019edb29b997b4421f72863e319246eca5771c
docs/DEVIATIONS.md	a94083caf3410e85cd6ad6382b344bfb91c7a65d0906af0a0c385c34996d04af
docs/CLAIM_AUDIT_380.md	cbd833357d771e26540968e6955b0b12add4f23a00e343807ab21257e5f1c3a0
spend _ ledger . jsonl(25lines , readfor3 . 8 . 0 ; alivingfile ; includestpu_laya-textonly-204124)	f0e065cf17f9865e3125ff205481623c42b06f3a6a852fef66846c4c7ae211cb

Revision 3.9.0 (written by `scripts/make_tables_390.py` when the tables were generated; living files as of that run)

results / gwenlaya _ v4 / edition / python_gate_depth.json	425e9d107bfce343a9c8f0b1cbc158a136887eb6271971007564cec91baf4b9c
scripts/python_gate_depth.py	ae443caf6344cdf7e9bea0582b56645d772b4ef2a7c07dd0a082f1c9e4fc57e2
tests/test_python_gate_depth.py	aa113acaaef29dfff0031f42654f524fa9e07b98a38435b1f15dbdc0fa84d6ce
results / gwenlaya _ v4 / edition / prompt_test_visibility_390.json	fb5bce4e8823452645575bbf8dc8885717d849c218a389c94c866df78e635206
gwaya/domains/strength.py	df635de31aad9139e279882281531f7eb65fa4288bf7ea54288429c5f2e02d72
gwaya/domains/checkers.py	2aef3bb9665d988a2e623d8b6e5ac2b7d014d1db575aa7894d523b1947c1ff24
gwaya/gwenlaya.py	80d00a6ee6f09164c55b62af22c1a9b802aa621d8faa156fdca13bd318942325
tests/test_gate_strength.py	50ee5f362a76b6ee15c15c007dbff9ace72a935669694b8437a0717eeb90682d
tests / test _ gwenlaya _ verdict _ policy.py	3a712db016782502594d13eaed40b6e251bffe766436311e76a10dcf45c0426e
results / gwenlaya _ v4 / lowtier / verdict_policy_exploratory.json	6352d1950842ca9178ff469ee27c039f0a4c1598e7e401d1bed9e7f63792bd3d
scripts / verdict _ policy _ exploratory.py	dd4a8e929d76c64af15d09ae225fcf12fb9cd4045cd70f045fc5d77d6a7936d6
results / gwenlaya _ v4 / e _ tpu _ mathgate/rescore_exploratory.json	ce2dab98e3751332b52e09ec4a044ea857f52558a22e7035d22c3b69512bff2a
scripts/rescore_math_stored.py	737b1d4ade0439bc302b2693b448320cb04a750ff30439d18700b3393cfb3556
gwaya/domains/math_check.py	690a1658b4e3b3e0d3321dc987cf8efa1f58c9243611c8685534d18bcf1d73a9
tests / test _ math _ check _ normalization.py	f47c9be723fb2d94a7bbd6e6f7426ecc388a8653a6b2075e1ba715b93f78c400
gwaya/lean_gate.py	64377e68a0de8c3ab3e90baa0ca3f2526abc5ade8268741b4f66935e022ba643
tests/test_lean_gate.py	e9ed35d552f7b481b7f67a89e4b49a3ad3b9fb10829eaf8de03ee58077023004
results / gwenlaya _ v4 / lean _ a18 / lean_sanity.json	d828a4ee1d0957895fce403ba6aa1ab3a7ae209640269b2ac7c5132235564264
results / gwenlaya _ v4 / lean _ a18 / tasks_lean.jsonl	be0603bcc6b274cbb6cb9f39663f1af9fef0a782f2922a2739953e0bc88dd6a7
results / gwenlaya _ v4 / lean _ a18 / lean_gate_tests_390.log	6809f9df5e3eb2552bbe1f97a180f38024b691afcffc6ce4b9caec131e6b9ef0
results / gwenlaya _ v4 / lean _ a18 / lean_sanity_summary_390.json	fae07db741a237c3c5a1edbc3050f1dbeb7f73a24177ad34ec7a5e813df01735
results / gwenlaya _ v4 / spend _ summary_390.json	ee47ca819c8ffe1e50bf0ee92680cf0814033e7d7233f64cf42ac42c8d2b4925

scripts/make_tables_390.py	3b35f3f562509f597a238036e7337b0dc340c0a314a126da652c12a6400c9361
tests/test_make_tables_390.py	d8e04231bcf08ef8b0665e0a9c0bd1588979d77d2cebed9670a8b9d5b8406b53
papers/tables_pydepth390.tex	c3b89e7721f75d00ef375cf63df4e0096d7d5836794d5b295ba545dd541dbbc4
papers/tables_mathfix390.tex	a582314ed9ea744ca8e4752a1ab95db4ab43d1f6ce6df67201560f5c515af665
papers/tables_policy390.tex	d83dd2f3f6b52c6941df82a27f4fd5a9c1c7b52c7ce7a70853d4e37582602ddd
papers/numbers390.tex	ff5d2371b24408586b0e18ae670f0b90998192f7b8800bc03937b9a1f3920a2c
docs/ANALYSIS_PLAN_E_TPU.md	2d8fe29cc8a5ba00c89a665a6e269847009914a7448ff78dffa6be1db7bbe22a
docs/DEVIATIONS.md	3173a8791a5d5a2a3f1b66b53fb42a9b1d1273ab451a13fba4bab044c429e536
tasks _ E _ primary . d25 . jsonl(dataroot)	2143110c78a3a9f57599dce77021cf4571fc57cf2584017f16ebdd08d5acba66
spend _ ledger . jsonl(27lines , asreadfor3.9.0;alivingfile)	9f97c3bb15ddac042c10954949d0442f748edf19bd456b8bb5f165bceac37e86

Revision 3.10.0 (written by `scripts/make_tables_3100.py` when the tables were generated; living files as of that run)

results/gwenlaya_v4/eval_sets_F/manifest_F.json	025bfa6a3a9ef1acf09326779392e15c2a0fc7a3c25096aa87c2f4b41314c538
results/gwenlaya_v4/eval_sets_F/numbers_F_pm.json	a41966dc2ae6dc4383c11458c4cf8397e47934db3f3ec925aa791c6cad33d9ed
results/gwenlaya_v4/eval_sets_F/numbers_F_laya.json	1ff91bdb87dbf066cbecac46d168bc33781aa04af28ee91e2773b42021d00ac0
results/gwenlaya_v4/eval_sets_F/lean_scored/summary_a18.json	bfff95e064faaad97de0ee8a7592bf1cff427b3cfe203514b2137c05a2995f6e
results/gwenlaya_v4/eval_sets_F/tasks_F_python.jsonl	49546c3bd612d5b4cf2eb1b72fc73b7822bf67f831efd954a5d3060bc417e68e
results/gwenlaya_v4/eval_sets_F/tasks_F_math.jsonl	8277bc52e3a0f06ae809744db76bd85c83b88eca5fa75d99c0fde3073230de70
results/gwenlaya_v4/eval_sets_F/tasks_F_lean.jsonl	27ca4024f2b4bbb61986f6d4ef655c47ffc57dd9607b2e01278bf278872d097d
results/gwenlaya_v4/eval_sets_F/decontamination_report_F.json	dbb1a7635746b398b2c5fa84a9f9b831bcaa6b0d62ec82be493533fa28da5c02
results/gwenlaya_v4/eval_sets_F/laya_rows/manifest_F_laya_rows.json	d1c9194c40e9ca49d27be6094e2c903fe61410847b1a95e0aef8bfd2d4520672
results / gwenlaya _ v4 / lean _ a18 / scored_greedy_v3/summary_a18.json	f2855a34fccb65095b73325506b40db0713ab43e65878c56ab401284d8029c88
results / gwenlaya _ v4 / lean _ a18 / history / summary _ a18 _ pass1 _ 120s . json	730143f19ebbd4846cbdf7dac3654343a083ba6ab724f2ae882da70b6e138891
results / gwenlaya _ v4 / lean _ a18 / history / summary _ a18 _ pass2 _ 300s _ no_match.json	3f149d91b54558aedcdeed196e834ec546d5d9d6a0378afcd5b6394d0307eb8b
results / gwenlaya _ v4 / lean _ a18 / lean _ forbidden _ breakdown _ 3100 . json	0bcdfe927fbc7bdd872eae3144307a1cd4dd7f15bcb064dc4b010f76f223a227
results/gwenlaya_v4/edition/rust_gate_selection.json	7ebed068b2d40dbd59a03126eda5f2c9a16033b9d77ea1a88a3252e9072a08cf
results/gwenlaya_v4/laya_runtime/quant_shift_probe.json	21f6dc974b38157d7a039dcd5601c4dc33a398031efbc0cd089bf54abe0cc4c5
results/gwenlaya_v4/laya_runtime/system_replay_eprime.json	358a78f6ac2eb6b4152151a5b1000eb85e1504487a91c70683c520e53ee00485
results/gwenlaya_v4/laya_runtime/parity.json	5dafa5ca2a5e4f428bcf4946ffc5ee4a5e875b7f855916ff11f1f9a49925f0a5
results/gwenlaya_v4/laya_runtime/live_smoke.json	8e4fa2d5d182da9539467ffd0c0baa44dbbb5893866eff5a27b6ab1060c18238
results/gwenlaya_v4/laya_runtime/thresholds_C.json	ca550e1acfb6b4f88add613e1ee0ea8f86cb48d906d1ec962e9cbbe6e326d11b

results/gwenlaya_v4/laya/layout_a22.json	83ab1432e5465c25cab6ff7c4cc0c9f666a60eedb02b598b097d8154da276ca4
results/gwenlaya_v4/spend_summary_3100.json	953d0d796efe33bc7cd6f06771098ac54832964cf8da506a267ad17428f3d071
results/gwenlaya_v4/math_pot_runtime/consistency.json	7e6edddc3375b3ad50224700fb80ffd34a9968a510495e48a8aff4e85dc053a5
gwaya/laya_runtime.py	3f623574778758c6a780bb3250cca5c4389568980a88bc3f1dce98ceaa54c4a9
gwaya/llama_tier.py	34fa1b5d99d5d788b028b59536025c13351aeada60fc82206e18755e016f39a2
gwaya/math_pot_runtime.py	88707569a1106f921d1a5f89361b0791a70c53a68205b0c392d0ae032ad1617b
scripts/analyze_F.py	25b6ad979eefff2ae0bf1e704cdd9f6e1929e464ca8fe1ff8e4a389479dcd583
scripts/analyze_F_laya.py	b58ef6fb66013fb18c7575ba2e576f11761621ffd81de25143b729736f5c1540
scripts/print_F_tables.py	0ec798effbec3b8ed46cba3a675af5bd95cf83ba18eba6d75daba1abe711a857
scripts/make_tables_3100.py	bc97171682b0dd6ce999f2d5aa4890bbe12fb14fc674ed7d5cb7cc84f994745b
papers/tables_f3100.tex	4259a0b4f35f00042a3ec56342652cd13035e9c13c3758c2d00906215d0206ba
papers/tables_flaya3100.tex	0f36a06c090438c262b73020e0b5a7bf682fd5f8aa7d69b0325e4fbac0c30774
papers/tables_lean3100.tex	a1616bbb06b0d7ae01e012db072b8dfd3148cfabc6743c60239fbf16de1155da
papers/tables_rust3100.tex	62de19441d97e24d75351f5b2dc962d80f2d207b3b41881145c713cabd8e9ab6
papers/tables_runtime3100.tex	e762ce1a7b102db11077f2cdbaa77edcfce92eca2f36bfac30c37ed48e845f3c
papers/numbers3100.tex	02ee9c935a32830d35cdcae4ffa502558a3423f81b7ab1719dcfe42be9fda0a9
docs/ANALYSIS_PLAN_E_TPU.md	f6928cd99021a84e08eef07ae65d0f9a0020055a54e714b30d49751947a7f463
docs/DEVIATIONS.md	43b06c540c4d56bd3ecd51d8672bdb95911d8bdb148a6a4abb4d585bba0c2ebf
docs/LAYA_RUNTIME.md	2a003e457a5ce7d633cbc51a850e98b2e6d7ecb1d03c870d2af6c87a0ea0f3bd
docs/LAYA_FREEZE.txt	10bcc78fe4b3856dcb14220e1a47a1082d6a7df670e3905224259efbafc43111
spend_ledger.jsonl(31lines, asreadfor3.10.0;alivingfile)	9f90534921ade9801a6142601c276e96d83c4744f4cdfc328682999c08d7b8de

The sha256 values of the trained adapters, heads and the SH controls are in `docs/LAYA_FREEZE.txt`; the weights themselves are on the data disk and are not committed.

The HF dataset revisions are in the freeze addendum (MATH-500 `6e4ed1a2`, humanevalplus `d32357cf`, mbppplus `b2d74c91`, MultiPL-E `28441b60`). The E-set model weights were loaded at revision `main` and are not pinned.

Data lake (`gs://socrateai-datalake-gen-lang-client-0625573011/gwenlaya_v4/`; read-only listings).

- `night/2026-10-07/`: the night inputs.
- `tpu_smoke/`: smoke runs #1–#3.
- `tpu_runs/gwenlaya-tpu-e-gen-210955/`: the first, discarded E pass.
- `tpu_runs/gwenlaya-tpu-e-gen2-214445/`: the second E pass (raw generations, `gen_batch` logs, `status.jsonl`).
- `e_tpu_final/`: scored rows and generations of the tiers and the Coder, the overlay and the audit reports.

The night CPU rows are on the local data disk and in the repository (`results/gwenlaya_v4/night_L2fix/`). Mirroring them to `gwenlaya_v4/results/`, as the registration requires, is still open.

Commands (full E set). The environment is `PYTHONPATH`=repo, `GWAYA_DATA_ROOT`=data root, with `GWAYA_ALLOW_UNISOLATED` unset. The exact flags of each step are in the commit messages and script headers.

```
deploy/tpu/drive.sh --name e-gen2 --script deploy/tpu/run_e_gen.sh \
  --upload "repo.tgz tasks_prompts.jsonl" --download "raw_*.jsonl status.jsonl gen_*.log" \
  --max-minutes 120        # dry run unless --yes-spend
#   on the VM: run_e_gen.sh -> scripts/tpu/gen_batch.py
#   (SKIP_JAX_PRECOMPILE=1, max-seqs ladder 96 32 16 8)
python scripts/import_remote_gens.py --tasks tasks_E_primary.d25.jsonl --raw raw_<model>.jsonl \
  --served <model> --quant bf16 --out $D/etpu/tiers --accelerator "TPU v5e x1"
python scripts/run_study.py --plan experiments/night/plan_night.json --stage night_L2 --backend openai \
  --backend-url http://127.0.0.1:1/v1 --tasks tasks_E_primary.d25.jsonl \
  --models qwen3.5-2b-bf16 qwen3.5-4b-bf16 --quants bf16 \
  --arms base always_smallest always_largest gate_only raw_confidence gwenlaya \
  --mode score --no-vram --check-workers 5 --out $D/etpu/tiers
#   Coder: scripts/tpu/score_local.sh (import + score, arm base)
python scripts/tpu/recheck_failures.py --run $D/etpu/tiers --tasks ... --model <model> \
  --out recheck_<model>.json
python scripts/tpu/recheck_failures.py ... --verified --out recheck_pass_<model>.json
python scripts/tpu/serial_overlay.py --audits recheck_*.json --raw-dir $D/etpu/raw --tasks ... \
  --out $D/etpu/overlay2 --tiers qwen3.5-2b-bf16 qwen3.5-4b-bf16 --coder qwen2.5-coder-1.5b-bf16 \
  --tiers-gens $D/etpu/tiers/gens.jsonl --coder-gens $D/etpu/qwen2.5-coder-1.5b-bf16/gens.jsonl
python scripts/analyze_e_tpu.py --tiers $D/etpu/tiers --coder $D/etpu/qwen2.5-coder-1.5b-bf16 \
  --tasks tasks_E_primary.d25.jsonl --night-cpu results/gwenlaya_v4/night_L2fix/rows.jsonl \
  --night-tasks tasks_E_night.d25.jsonl --unstable-audits ... --overlay-tiers ... \
  --n-boot 10000 --seed 0 --out-numbers results/gwenlaya_v4/e_tpu/numbers_e_tpu.json \
  --out-tables papers/tables_e_tpu.tex --out-figs papers/figures_v4
```

I did not re-run `analyze_e_tpu.py` for this revision. I verified that its committed output matches the hash above, and I recomputed the per-model pass counts and truncation counts from the committed rows.

Commands (revision 3.5.0). The exact command lines of these steps were not saved; below are the shapes, taken from the helper scripts `run_mathgate.sh` and `run_nine.sh` (data disk, `etpu/`) and from each script's argument parser. D is the data root.

```
# A10: program-of-thought generation on TPU (2B, 4B), then import and score (arms incl. gate_only, gwenlaya)
python scripts/import_remote_gens.py --tasks $D/etpu/mathgate/tasks_math.jsonl --raw <answers>.jsonl \
  --served <model> --quant bf16 --out $D/etpu/mathgate --accelerator "TPU v5e x1" --seconds-from <tiers>/gens.jsonl
python scripts/import_remote_gens.py ... --raw $D/etpu/pot_raw/raw_pot_<model>.jsonl --kind pot
python scripts/run_study.py --plan experiments/night/plan_night.json --stage night_L2 --backend openai \
  --backend-url http://127.0.0.1:1/v1 --tasks $D/etpu/mathgate/tasks_math.jsonl \
  --models qwen3.5-2b-bf16 qwen3.5-4b-bf16 --quants bf16 \
  --arms base always_smallest always_largest gate_only raw_confidence gwenlaya \
  --mode score --no-vram --check-workers 5 --out $D/etpu/mathgate
# A12: 9B on v5litepod-4 (TP=4), import with --chips 4, score arms base and gate_only
python scripts/import_remote_gens.py --tasks tasks_E_primary.d25.jsonl --raw $D/etpu/nine_raw/raw_qwen3.5-9b-bf16.jsonl \
  --served qwen3.5-9b-bf16 --quant bf16 --out $D/etpu/nine --accelerator "TPU v5litepod-4 (4 chips)" --chips 4
python scripts/import_remote_gens.py ... --raw $D/etpu/nine_raw/raw_pot_qwen3.5-9b-bf16.jsonl --chips 4 --kind pot
python scripts/run_study.py ... --models qwen3.5-9b-bf16 --arms base gate_only --mode score --out $D/etpu/nine
# analyses (flags from each argparse; paths as in the scripts' headers)
python scripts/analyze_e_tpu.py --tiers $D/etpu/tiers --coder ... --tasks tasks_E_primary.d25.jsonl \
  --overlay-tiers $D/etpu/overlay2/tiers/rows.jsonl $D/etpu/mathgate/rows.jsonl --n-boot 10000 --seed 0 \
  --out-numbers results/gwenlaya_v4/e_tpu_mathgate/numbers_e_tpu_mathgate.json
python scripts/analyze_abstention.py --tiers $D/etpu/tiers --tasks ... --overlay <overlay2 rows> <mathgate rows> \
  --models qwen3.5-4b-bf16 qwen3.5-2b-bf16 --out results/gwenlaya_v4/e_tpu_mathgate/abstention_a11.json
python scripts/analyze_cascades.py --tasks ... --rows <tier rows incl. nine/rows.jsonl> --gens <gens incl. nine/gens.jsonl> \
  --out results/gwenlaya_v4/e_tpu_mathgate/cascades_a12.json
python scripts/make_tables_350.py   # paper tables of 3.5.0
```

I did not re-run the TPU generations or the three analysis scripts for this revision. The numbers in Sections 6.6–6.8 were read from the three JSON files above (hashes in the table).

Commands (revision 3.6.0). The command lines were not saved verbatim; the steps and their scripts are: P and E′ task files from `scripts/data/build_pool_p.py`; generation and scoring of the three tiers as in revision 3.5.0 (TPU via `deploy/tpu/drive.sh`); Laya rows from `scripts/data/build_laya_data.py`; training and prediction by `scripts/train_laya.py` (`deploy/tpu/run_laya_train.sh`, `run_laya_twin.sh`, `run_laya_predict.sh`); `scripts/laya_freeze.py` (isotonic map on C and the freeze file); `scripts/analyze_laya.py` (E′ analysis; flags in its parser); `scripts/laya_router_warmup.py` (no-warm-up cost ratios); `python scripts/make_tables_360.py` (tables of this revision; recomputes the control AUROCs and the spend sums). I ran only `make_tables_360.py` and independently recomputed the control AUROCs; I did not re-run any TPU job or `analyze\_laya.py`, and the numbers of Section 7 were read from the committed JSON files whose hashes are above.

Commands (revision 3.7.0). The R′ task file and report come from `scripts/data/build_rust_pool.py` (tests in `tests/test_build_rust_pool.py`); generation and scoring of the three tiers as in 3.5.0 and 3.6.0 (TPU via `deploy/tpu/drive.sh`); Laya predictions with the frozen artifacts via `deploy/tpu/run_laya_predict.sh`; `scripts/analyze_laya.py` (R′ analysis; flags in its parser; output `laya_rfresh.json`); `scripts/rust_fresh_summary.py` (per-tier summary); `scripts/rust_edition_sensitivity.py` and `scripts/rust_gate_depth.py` (A15; tests in `tests/test_rust_gate_depth.py`); `python scripts/make_tables_370.py` (tables of this revision; recomputes the gate and top-k overlap and the spend sums). The command lines were not saved verbatim. I ran only `make_tables_370.py`; I did not re-run any TPU job, `analyze\_laya.py` or `rust\_fresh\_summary.py`, and the numbers of Section 7.7 were read from the committed JSON files whose hashes are above.

Commands (revision 3.9.0). `GWAYA_DATA_ROOT=/mnt/data/home/xavkal/gwaya-data python3 scripts/make_tables_390.py` writes every generated table, macro and summary of this revision (it reads only committed result files, the E task file and the ledger). Tests run for this revision: `tests/test_make_tables_390.py`, `test_python_gate_depth.py`, `test_gate_strength.py`, `test_gwenlaya_verdict_policy.py`, `test_math_check_normalization.py` (all passed) and `tests/test_lean_gate.py` with the pinned project (94 passed; log in `lean_gate_tests_390.log`). I did not re-run `python_gate_depth.py`, `verdict_policy_exploratory.py` or `rescore_math_stored.py`; their numbers are read from the committed JSON files.

Commands (revision 3.10.0). `GWAYA_DATA_ROOT=/mnt/data/home/xavkal/gwaya-data python3 scripts/make_tables_3100.py` writes every generated table, macro and summary of this revision (it reads only committed result files and the ledger, and runs `gwaya.lean_gate.lexical_flaws` offline on the stored Lean texts). The F analyses were produced by `scripts/analyze_F.py` and `scripts/analyze_F_laya.py` (the command lines are in the script docstrings; `scripts/print_F_tables.py` prints the descriptive tables), the Lean files by `scripts/lean/score_lean.py`, A15.3 by `scripts/rust_gate_selection.py`, A19 by `scripts/laya_quant_shift_probe.py`, A20 by `scripts/replay_system_eprime.py` and A22 by `scripts/analyze_laya_layout.py`. I did not re-run any of them; the numbers are read from the committed JSON files whose hashes are above. The test suite was run with `PYTEST_DISABLE_PLUGIN_AUTOLOAD=1 python3 -m pytest -p no:cacheprovider -q`; the result for this revision was 1,018 passed, 22 failed, 16 skipped and 2 collection errors in 370 s; the 22 failures (and the 2 errors) are environment-only: the sandbox lacks `pyflakes`, `sympy` (one real-sandbox math-gate case) and `fastmcp`, and they touch no code of this revision. The four helper tests of `tests/test_make_tables_3100.py` passed.

Commands (night CPU arm). With the environment of `night/run_L2.sh`:

```
python scripts/run_study.py --plan experiments/night/plan_night.json --stage night_L2 \
  --backend openai --backend-url http://127.0.0.1:8091/v1 \
  --tasks $NIGHT/results/tasks_E_night.jsonl --models qwen3.5-2b-q4_K_M \
  --quants q4_K_M --arms base --mode generate --no-vram --cpu-pid <llama-server pid> \
  --out $NIGHT/cache/L2
cp $NIGHT/cache/L2/gens.jsonl $NIGHT/cache/L2fix/   # v3.3.1: re-score only
python scripts/run_study.py ... --mode score --out $NIGHT/cache/L2fix
python scripts/analyze_study.py --results $NIGHT/cache/L2fix/rows.jsonl $NIGHT/cache/L2fix/gens.jsonl \
  --tasks $NIGHT/results/tasks_E_night.jsonl --ledger $GWAYA_DATA_ROOT/spend_ledger.jsonl \
  --n-boot 10000 --seed 0
python scripts/data/regen_gate_payloads_d25.py --results $NIGHT/results   # D25 task files
```

13 Limitations

- **No registered-form test.** The only tests of registered hypotheses (H1, H3 on E′, 3.6.0, on the Rust set R′, 3.7.0, and on F, 3.10.0) are deviated (D48–D50, and D53–D56 for R′, and D58, and D69–D73 for F) and use the A13b metrics. Nothing here supports or refutes H2, H4–H12, H14 or H15.
- **Unregistered E-set baselines.** The full-E baselines are exploratory: single greedy sample, one chip, one engine version, bf16, two tiers, weights not pinned to a commit. Four of the plan's analysis choices were amended after the first results (Section 6.1). The Coder rows existed 10 s before the plan commit.
- **Matched coverage in A4.** The baseline's k is matched to the gate's coverage on E rather than taken from a threshold frozen on a separate split. The gate's advantage also includes executing a visible test.
- **E is spent for registered H1/H3.** E was generated and analysed before the registration's freeze (calibrator and thresholds), so H1/H3 cannot be run as registered on these generations. A registered test needs fresh generations on a new set E′, or a recorded deviation.
- **The cascade is not GL.** It has no Laya, is compared with B2 rather than B5, and is not costed in GPU-seconds. Its chip-second cost depends on engine concurrency (effective at most 11 for the 4B vs at most 54 for the 2B) and on whether the compilation chunk is counted (pooled ratio 2.021 with it, 2.023 without, with the math gate of 3.5.0 and its program charge; 1.357 and 1.243 in 3.4.0), and it excludes the CPU time of the gate.
- **Math gate (revision 3.5.0).** The math gate (program-of-thought re-execution) is now implemented (A10), replacing the 3.4.0 "not implemented" statement. It is post hoc, the program is written by the same tier as the answer (so verification is agreement of one model with itself), its 4B precision 0.943 is below the 0.95 flag point (CI includes it), and its cost is charged only to gated arms. (3.9.0) Its headline comparison with "answer everything" (0.026 against 0.388) mostly reflects abstaining on truncated non-answers; against answer-if-boxed it is 0.026 against 0.068. Math accuracy is still a lower bound, because up to 40 % of math answers (2B) were truncated at 1,024 tokens.
- **9B tier and chip-seconds (A12).** The 9B ran on a four-chip slice and its effective engine concurrency was not measured, so its cost, and every ratio against always-9B, is less certain. The "pays off" verdicts use point estimates for cost and have no multiplicity correction. The 27B was attempted in 3.6.0 and did not finish loading (Section 7.5). A11 was not run for the 9B.
- **A11 (calibrated abstention).** M3 uses the gate verdict as a feature, so its gain is calibration and not independent evidence. CIs do not include model-fit variability, and ECE percentile-bootstrap CIs can exclude their own point estimates. It is not the Laya head of 3.6.0.

- **Post hoc.** A10, A11 and A12 were all decided after the E baselines had been seen (D43–D47), on an evaluation set that is burned for the registered H1/H3.
- **Stability.** One re-run of each passing row cannot rule out unstable candidates. Contention and five genuine timeouts per model remain. Scoring ran from working trees with uncommitted changes.
- **Contamination.** All absolute scores are on public sets that Qwen3.5 has probably seen. Only paired within-model deltas are claimed by the registration. The contamination-stratified analysis asked for in review finding F12 was not done (TBD).
- **Night data.** The night math generations are invalid (D28). The Rust harness was validated after the night generations existed, on hand-written solutions (25 of 26 tasks covered). The HumanEval+ night gate payloads contained the hidden tests (D25), but no gate ran there.
- **Laya and E′ (3.6.0).** E′ has 169 Python tasks and mostly easy math and no Rust; nothing is claimed about Rust from E′ (the Rust test is R′, next item); one greedy sample; list-price cost estimates; E is in the Laya training set (D49); the calibrator receives the gate and log-prob as inputs, so its advantage over each alone may largely be a combination effect (a reading only; the 3.8.0 text-only ablation shows the text alone is predictive on E′ but does not test what the text adds to the gate and log-prob); the shuffled-label control failed and its withdrawal rests on an unproven reading (D50); H1/H3 use the A13b metrics; the backbone revision is not pinned; no generator adapter exists; the data-lake "laya" weights were not used. The router policy is exploratory with τ at the lowest grid value; the 4.0% against 30.4% confident-wrong comparison is not like for like.
- **Fresh Rust set R′ (3.7.0).** 69 tasks (wide intervals; H1 and H3 may simply be underpowered, and the null result is not evidence of no effect); the tasks come from a public repository and may be in pre-training data; they were kept by a reference and stub check with the same harness and built after the E′ results were known (D53, D54); one greedy sample; list-price costs; τ reused from C (D56). The reasons for the non-transfer of Laya's advantage (prompt style and length, 512-token truncation, Rust only from MultiPL-E in training, small n) are hypotheses and were not tested; input truncation was not measured. **Harness:** all published Rust results used edition 2015 (D55). A15.1 found at most one confirmed fail-to-pass flip per tier (E 1/507 for the 4B and the 9B, R′ 4B 1/69), so the effect is small and no number was re-analysed; the oracle default is now 2021 (D58), so future runs differ from the published ones in that respect.
- **Text-only ablation (3.8.0).** Exploratory, one seed, post hoc after the E′ and R′ results (D59, D60); the in-distribution signal may reflect source or difficulty rather than the candidate (untested); R′ has 69 tasks, so "at chance" means "not distinguishable from 0.5 here"; the reasons for the non-transfer are hypotheses; no shuffled-label control for this model; the converse ablation (no text) was not run.
- **Depth dials (A15.2, A17).** Descriptive, on stored answers of E, 4B and 9B only: Rust 505 tasks, $k \leq 3$, edition 2015, deviating from A15 ($k = 1, 2, 4$; D58); Python 524 tasks, $k \leq 8$ (3.9.0). Not run on R′ or E′. For MBPP the first visible test is the one printed in the prompt (D63), so the one-test gate partly checks a test the model saw. They say nothing about what a model would write given more visible tests, and they are not hypothesis tests.
- **Math scorer (D62, 3.9.0).** The registered math labels contain scorer false negatives on notational equivalences; the corrected scorer was designed after inspecting the stored outputs, so the re-scored accuracies and gate counts are exploratory. A4, A5, A11, A12, the Laya training labels and E′ were not re-scored, so every math-dependent number there carries the registered scorer's misses.

- **Verdict policy and Lean gate (3.9.0).** System changes without results. The weak-gate policy is off by default and has no chosen $k_{\min}$; its effect for real users is unknown. The hardened Lean gate does not compare binder annotations, accepts definitional matches, rejects some harmless constructs, takes tens of seconds per check, and has not had its external red-team review.
- **Fresh sets F (3.10.0).** Fresh relative to the Laya only; public benchmarks, probably in the generators' pre-training data, so accuracies are upper bounds. One greedy sample; a 1,024-token limit that cuts a third of the generations and almost all competition answers (the competition stratum is a floor); Python hidden suites as good as the datasets' own tests; sandbox exclusions favour self-contained problems (149 references did not verify); the one-test Python gate is a weaker check on F (precision 0.527 to 0.729) than on E; prompt-visibility labels are a heuristic; passing rows were not re-run for stability (D69); the 27B tier and CMIMC/SMT problems were not run; the F cascade no-warm-up subset is not A12's (D70). The F result for H1 is pooled over two domains with different base rates and is carried by math; within Python alone the 9B interval includes zero.
- **Laya replication (3.10.0).** One training run, one seed (the seed-to-seed spread is unmeasured); the calibrator receives the gate and log-prob as inputs and is not distinguishable from a logistic regression on them pooled (and worse on Python); the shuffled-label control still has not passed (D50); no Laya score on Lean or Rust F; the router saving (0.889 of always-9B) is a first-chunk-included reading, with no no-warm-up reading computed.
- **Lean 4 (3.10.0).** Greedy, one proof per statement, 1,024 new tokens (many generations are cut); proof rates are lower bounds under 300 s timeouts measured on a loaded machine (4 to 24 timeouts per tier and set); the hardened gate does not compare binder annotations, accepts definitional matches (none occurred), has had no completed external red-team review and has no independent second checker, so its false-accept rate is unmeasured; miniF2F is public; the $k = 8$ sampling produced no rows; the 0.8B tier is an extra; the A18 results were scored in three passes with the later passes re-running only part of the rows (D67, D68).
- **Runtime integration (3.10.0).** The Laya runtime, the PoT gate without a gold answer and the llama.cpp tier are opt-in system changes with no end-to-end evaluation: the live smoke has $n = 3$, the A19 probe 28 Python pairs on the 2B only, in-sample and across different serving stacks, the A20 replay checks orchestration only; Q4_K_M calibration, thresholds, 4B and 9B paths, concurrency and latency on a free machine are not validated (Section 7.13).
- **A15.3 and A22 (3.10.0).** Descriptive, on stored answers (A15.3: 505 E and 68 R′ tasks, edition 2021, 4B and 9B answers of prompts that mostly show no test) and on E′ and R′, which have been seen; the R′ first-versus-first-and-last count was known before the plan, so R′ confirms nothing; A22 is one seed with no shuffled-label control and without the planned token-window fractions (D74).
- **Spend.** The USD spend ($22.0055, 31 ledger lines, including the A18 Lean generation, F and A22) is a list-price estimate, not the billed amount. H13 is weak evidence (Section 8).
- **Not run.** The SFT and DPO training arms, quantization and the endpoint did not run. Lean 4 now has descriptive results (3.10.0, Section 7.11); not done there: the $k = 8$ sampling, the external red-team review, a Laya score on Lean. The Laya head was trained in 3.6.0 (Section 7); still not done: a generator adapter and the 27B tier (the probe package exists, D76).

14 Next steps

1. The test was run in 3.6.0 in a modified form (D48–D50); the registered form was not run. Run it: a split with a Laya not trained on the burned E, thresholds frozen on C, the registered CWR-at-matched-coverage metric for H1 and the cost-per-correct metric for H3.
2. Resolve the failed shuffled-label control: repeat the SH and untrained-twin runs over several seeds and with the numeric features dropped, to test the reading offered in D50, which is unproven.
3. The text-only ablation was run in 3.8.0 (one seed). Still open: the converse ablations (gate and log-prob inputs without the text; the text without the domain and tier tokens), several seeds, and a source or difficulty control (for example GSM8K against MATH) for the in-distribution signal.
4. A fresh Rust set now exists in a limited form (R′, 69 tasks, 3.7.0). Still open: harder fresh math and Python tasks (E′ has 169 Python tasks and easy math), a larger and less filtered Rust set, and a test of why Laya's advantage did not appear on Rust (input truncation share, training with Rust in the style of the Exercism prompts, a longer input window); these were not run.
5. Expose the number of visible assertions (`tests\_total`) to the calibrator as a signal. This changes Laya's input features, so it needs retraining and a new freeze; it is not done, and the frozen Laya was not changed. The depth dial now covers Python (A17, 3.9.0); still open: R′, a design in which the prompt shows no test that the gate runs, choosing a $k_{\min}$ for the weak-gate policy on held-out data, and re-running the Rust generations with the 2021 default.
6. Re-run the router policy with a τ grid extending below 0.5, and on harder tasks.
7. Try the 27B again on a slice that the per-zone quota permits, if a quota can be obtained, and a generator LoRA on a stack that trains.
8. Validate the program-of-thought math gate (A10) on fresh data in E′, with a precision target set in advance (the 3.5.0 gate is implemented but evaluated only post hoc on the burned E).
9. Decide, before seeing any new result, whether the D62 scorer becomes the scorer of future registered math analyses, and re-score A4, A5, A11, A12 and the Laya labels with it, labelled as a correction.
10. Lean: run the $k = 8$ sampling of A18 (the engine-seed fix exists, D66) with a longer generation limit (a third of the proofs are cut), re-score timeouts on an unloaded machine, complete the external red-team review of the gate and add a binder-annotation comparison and an independent second checker; decide in an addendum, before running it, whether the frozen Laya is applied to Lean (it never saw Lean).
11. Validate the integrated system on the tiers it will actually serve: A19 for the 4B and a 9B tier, a larger paired set than 28 Python pairs, accuracy and LIKELY_CORRECT precision on Q4_K_M tiers with thresholds chosen on a split and frozen before an evaluation set (not the in-sample C thresholds), and an end-to-end live run on fresh tasks.
12. A22 follow-up, if the window question matters: compute the token-window fractions with the tokenizer, repeat v2 with several seeds and a shuffled-label control, and run `middle\_truncate`; only a rule registered in advance and evaluated once on a new set would support a claim.
13. A15.3 follow-up: a Rust set with the degenerate test not first, and a design in which the prompt shows no test the gate runs; choose a rule and $k_{\min}$ on held-out data before using

either.

14. Replicate the F cascade and router with the corrected D62 scorer, a 2,048-token limit, a no-warm-up reading and a measured 9B concurrency; add CMIMC and SMT.
15. Pin model revisions.
16. Report cost at equal engine concurrency, including a measured 9B concurrency, and read the 27B tier if the budget allows.
17. Read the billed TPU amount.
18. Regenerate the night math on CPU.

A null or negative registered result will be reported in the same form as this one.

References

[1] Anastasios N. Angelopoulos and Stephen Bates. A gentle introduction to conformal prediction and distribution-free uncertainty quantification. *arXiv preprint arXiv:2107.07511*, 2021.

[2] Xavier Callens. GWAYA v3.1: Fail-closed gate & real toolchain verification for LLM code generation. Zenodo, `https://doi.org/10.5281/zenodo.23121788`, 2026. Version 3.1.0; its +8.0 point claim is retracted.

[3] Federico Cassano, John Gouwar, Daniel Nguyen, Sydney Nguyen, Luna Phipps-Costin, Donald Pinckney, Ming-Ho Yee, Yangtian Zi, Carolyn Jane Anderson, Molly Q Feldman, Arjun Guha, Michael Greenberg, and Abhinav Jangda. MultiPL-E: A scalable and polyglot approach to benchmarking neural code generation. *IEEE Transactions on Software Engineering*, 49(7):3675–3691, 2023.

[4] Yonatan Geifman and Ran El-Yaniv. Selective classification for deep neural networks. In *Advances in Neural Information Processing Systems*, 2017.

[5] ggml-org. llama.cpp. `https://github.com/ggml-org/llama.cpp`, 2026. Release b11476, CPU build.

[6] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *International Conference on Machine Learning*, 2017.

[7] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. In *NeurIPS Datasets and Benchmarks Track*, 2021.

[8] Sture Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70, 1979.

[9] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, 2023.

[10] Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let's verify step by step. In *International Conference on Learning Representations*, 2024.

[11] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. Is your code generated by ChatGPT really correct? rigorous evaluation of large language models for code generation. In *Advances in Neural Information Processing Systems*, 2023.

[12] Quinn McNemar. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157, 1947.

A Claim audit of v3.3.1 (summary of `docs/CLAIM_AUDIT.md`)

The audit was run on 2026-10-08 on branch `bench/tpu-e-eval`, independently of the author. The auditor changed no paper, number, code or data file. Every number was recomputed from raw rows, generations, the ledger and logs (or the audit row says it was not), without using `numbers_v4.json` as a source.

Status	Count	What changed in 3.4.0
VERIFIED	61	nothing needed
VERIFIED_WITH_NOTE	29	notes on binning (A5), run #2 (D12, B8), assert counting (B3), registration timing (D11), L2 wall time (A30), self-reported timers (A24), the LoRA scope (B14), the Laya model (C14) and MultiPL-E metadata (C2) are now in the text (Section 2); bootstrap Monte-Carlo noise on two ratio upper bounds (A10, A14) needs no change
NOT_VERIFIABLE	2	B9 (cause of the run #3 TPU-host Python failure) now stated as unrecorded; C16 (pretraining exposure) is hedged and unchanged
WRONG	4	A32, A33 (hashes), A28 (stale spend), C12 (bib title): all fixed
OVERSTATED	3	B15 (H13), B12 (CPU-vs-TPU): reworded with the audit's text; C17 concerned the README and was fixed there
Total	99	

By class: A, own quantitative results (45 rows), all 14 pass/fail counts correct, and re-running the checkers on the stored generations reproduced every stored status (0 mismatches over 80 CPU and 240 TPU items); B, code and system (20); C, external facts and citations (19); D, logic and statistics (15).

Where this revision differs from the audit. The audit gives the commit of tag `v3.3.0` as `ebd032f`. `git rev-parse v3.3.0^{commit}` gives `200c7f4`, the merge commit that contains `ebd032f`. The file hash the audit asked for (`fccb1695...`) is the same at both commits, so the fix is unaffected. I found no other disagreement.

B Machine-generated tables of the night arm

The tables on the next page are `papers/tables_v4.tex` as written by `scripts/analyze_study.py` from `papers/numbers_v4.json` (content unedited, font reduced). "CPU-s total" is server CPU-seconds, which include idle-spinning worker threads (D14, D21). "acc." and "pass rate" are the PASS_HIDDEN rate, not the registered gate VERIFIED. CWR counts every UNDECIDED answer as wrong, so it is an upper bound. Since v3.3.1, math is scored boxed-only by default (D28), so the two math rows and the two pooled rows are equal.

Table 58: Cached generations on the E-night set (greedy, one seed). Tokens are mean completion tokens.

model	domain	n	mean tok.	CPU-s total	finish reasons
qwen3.5-2b-q4_K_M	math	26	371.0	9911	length:4, stop:22
qwen3.5-2b-q4_K_M	python	28	118.8	3625	stop:27, length:1
qwen3.5-2b-q4_K_M	rust	26	98.5	2963	stop:26

Table 59: Single unregistered arm `base`, per domain (descriptive, not a test). Pass = passed the hidden checks (PASS_HIDDEN), not the registered gate VERIFIED. Confidence = raw exp(mean log-prob). 95% cluster bootstrap CI. Cost unit: cpu_seconds. TBD = undefined (e.g. no passing item).

domain	status	n	pass rate	mean conf.	ECE	AUROC	cost / pass
math	INVALID prompt path (D24/D28); boxed-only	26	0.000 [0.000, 0.000]	0.832 [0.807, 0.857]	0.832 [0.807, 0.857]	TBD [TBD, TBD]	TBD [TBD, TBD]
python	headline	28	0.393 [0.214, 0.571]	0.896 [0.875, 0.917]	0.519 [0.392, 0.689]	0.717 [0.483, 0.917]	330 [174, 726]
rust	validated harness (D27)	26	0.231 [0.077, 0.385]	0.894 [0.882, 0.906]	0.675 [0.549, 0.813]	0.875 [0.709, 0.990]	494 [261, 1552]
math (boxed-only)	INVALID prompt path; registered boxed-only	26	0.000 [0.000, 0.000]	0.832 [0.807, 0.857]	0.832 [0.807, 0.857]	TBD [TBD, TBD]	TBD [TBD, TBD]
pooled (boxed-only math)	sensitivity	80	0.212 [0.125, 0.300]	0.875 [0.863, 0.886]	0.662 [0.585, 0.741]	0.806 [0.675, 0.917]	970 [629, 1648]
pooled	sensitivity (mixed)	80	0.212 [0.125, 0.300]	0.875 [0.863, 0.886]	0.662 [0.585, 0.741]	0.806 [0.675, 0.917]	970 [629, 1648]

Table 60: Arms on the paired E-night items (percent; 95% cluster bootstrap CI). Cost unit: cpu_seconds. TBD = not run or not available.

arm	acc.	coverage	CWR	ans. acc.	cost / correct
base	21.2 [12.5, 30.0]	100.0 [100.0, 100.0]	78.8 [70.0, 87.5]	21.2 [12.5, 30.0]	970.5 [629.2, 1647.9]

Table 61: Hypothesis status for the night run. Verdicts follow the pre-registered rules; primary Holm with m = 2.

H	role	result	detail
H1	primary	TBD	scored rows exist but only for unregistered single arm(s) ['base']: H1/H2/H3/H7 need paired registered arms (GL, B1, B2, B3, B5)
H2	exploratory	TBD	scored rows exist but only for unregistered single arm(s) ['base']: H1/H2/H3/H7 need paired registered arms (GL, B1, B2, B3, B5)
H3	primary	TBD	scored rows exist but only for unregistered single arm(s) ['base']: H1/H2/H3/H7 need paired registered arms (GL, B1, B2, B3, B5)
H4	secondary	TBD	no cross-fitted scores file (no scores file)
H5	secondary	TBD	no cross-fitted scores file (no scores file)
H6	secondary	TBD	no cross-fitted scores file (no scores file)
H7	exploratory	TBD	scored rows exist but only for unregistered single arm(s) ['base']: H1/H2/H3/H7 need paired registered arms (GL, B1, B2, B3, B5)
H8	exploratory_only_if_G1	TBD	needs G1 (no GPU slot)
H9	deferred	deferred	deferred (D8)
H10	dropped	dropped	dropped for this run (docs/DEVIATIONS.md D8)
H11	dropped	dropped	dropped for this run (docs/DEVIATIONS.md D8)
H12	dropped	dropped	dropped for this run (docs/DEVIATIONS.md D8)
H13	run	not refuted	
H14	only_if_E1	TBD	needs E1 (D12)
H15	offline_test	TBD	offline kill/resume test, see tests/test_run_study.py